

Agradecimientos

Estoy eternamente agradecido a todas aquellas personas (y mascotas) que me han acompañado estos últimos cuatro años. Mi esfuerzo y sus frutos son vuestros.

Primero y principal, a mis acompañantes felinos, Boni y Bicho, por hacerme la mejor compañía en las largas y solitarias jornadas de trabajo.

A mi tutor Germán Francisco Vidal Oriola, por su paciencia, guía y compromiso. Con la esperanza de que el trabajo esté a la altura de un tutor como tú.

A Denis y Pau, por ser grandes amistades y haber hecho de la universidad un sitio mucho más agradable, divertido y acogedor.

A los de toda la vida, por estar conmigo siempre, con un amor profundo, sincero e incondicional. A Mario, por hacerme reír como nadie puede; a Cuchieman, por estar ahí siempre que se le necesita, dispuesto a ayudar; a Karla, por su enorme corazón y cariño; a Dani, por haber compartido conmigo tantísimas cosas y por haberlas hecho todas mejores simplemente con su forma de ser; y a Pablo, el gran colaborador de esta memoria, eterno compinche y compañero de mil y una aventuras. Es todo un orgullo crecer con vosotros.

A Aitana, por haberme abierto las puertas de su amistad y confianza. Me hace muy feliz poder compartir mi vida con personas tan brillantes, peculiares y de buen corazón como tú.

A Alicia, por haber convertido estos cuatro años de una pesadilla en un sueño. Por haber sido la mejor compañía que podría haber deseado, por haberme querido y respetado con todas mis luces y sombras, y por ser la grandísima persona que eres.

Y por último, a mi familia que, por disfuncional que pueda ser, es la raíz que me ha nutrido para ser la persona que soy hoy, y no puedo estar más orgulloso y agradecido por ello.

Os quiero mucho a todos.

Resumen

Votar es una de las formas más sencillas y justas de tomar una decisión en grupo: cada persona expresa su preferencia, todas las opiniones valen lo mismo y el recuento da un resultado claro. Sin embargo, las herramientas digitales disponibles para recoger la opinión de un grupo, como los formularios genéricos o los dispositivos físicos de valoración, o bien son demasiado generales, o bien resultan caras y rígidas, y en ambos casos atan cada código QR a una única encuesta que queda obsoleta en cuanto termina.

Este Trabajo de Fin de Grado presenta Krate, una plataforma web de votaciones pensada para la toma de decisiones colaborativas. Un gestor crea un catálogo de opciones, define votaciones a partir de él y las asigna a puntos de votación, cada uno con un código QR fijo que puede imprimirse una sola vez: lo que se muestra al escanearlo lo decide el sistema en cada momento, de modo que las votaciones pueden cambiar sin reimprimir nada. Los participantes votan desde su propio teléfono sin registrarse ni instalar aplicaciones, con tres tipos de votación (opción única, varias opciones o clasificación por orden de preferencia), y el gestor consulta los resultados de forma centralizada. El caso de uso que dio origen al proyecto es el de una red de máquinas expendedoras cuyos usuarios eligen los productos que quieren ver en ellas.

Más allá del producto, el trabajo se ha planteado como un ejercicio completo de ingeniería del software: se sigue una metodología ágil con revisiones periódicas con un cliente externo, se elabora una especificación de requisitos completa, se diseña una arquitectura formada por un servidor y dos aplicaciones web independientes, se implementa con Spring Boot y Angular, se verifica con pruebas automáticas y pruebas de carga y se despliega en la nube mediante contenedores y un flujo de integración y despliegue continuos. El resultado es un producto funcional y desplegado, y una memoria que documenta las decisiones tomadas y las desviaciones respecto a lo planificado.

Palabras clave: Votación; Implicación del usuario; Código QR; API REST; CI/CD

Resum

Votar és una de les maneres més senzilles i justes de prendre una decisió en grup: cada persona expressa la seua preferència, totes les opinions valen el mateix i el recompte dona un resultat clar. No obstant això, les eines digitals disponibles per a recollir l'opinió d'un grup, com els formularis genèrics o els dispositius físics de valoració, o bé són massa generals, o bé resulten cares i rígides, i en tots dos casos lliguen cada codi QR a una única encuesta que queda obsoleta tan bon punt acaba.

Aquest Treball de Fi de Grau presenta Krate, una plataforma web de votacions pensada per a la presa de decisions col·laboratives. Un gestor crea un catàleg d'opcions, defineix votacions a partir d'aquest i les assigna a punts de votació, cadascun amb un codi QR fix que pot imprimir-se una sola vegada: allò que es mostra en escanejar-lo ho decideix el sistema en cada moment, de manera que les votacions poden canviar sense reimprimir res. Els participants voten des del seu propi telèfon sense registrar-se ni instal·lar aplicacions, amb tres tipus de votació (opció única, diverses opcions o classificació per ordre de preferència), i el gestor consulta els resultats de manera centralitzada. El cas d'ús que va donar origen al projecte és el d'una xarxa de màquines expendedores els usuaris de les quals trien els productes que volen veure-hi.

Més enllà del producte, el treball s'ha plantejat com un exercici complet d'enginyeria del programari: se segueix una metodologia àgil amb revisions periòdiques amb un client extern, s'elabora una especificació de requisits completa, es dissenya una arquitectura formada per un servidor i dues aplicacions web independents, s'implementa amb Spring

Boot i Angular, es verifica amb proves automàtiques i proves de càrrega i es desplega al núvol mitjançant contenidors i un flux d'integració i desplegament continuus. El resultat és un producte funcional i desplegat, i una memòria que documenta les decisions preses i les desviacions respecte d'allò planificat.

Paraules clau: Votació; Implicació de l'usuari; Codi QR; API REST; CI/CD

Abstract

Voting is one of the simplest and fairest ways for a group to make a decision: each person expresses a preference, every opinion carries the same weight and the count yields a clear result. However, the digital tools available for gathering the opinion of a group, such as generic online forms or physical feedback devices, are either too general or too expensive and rigid, and in both cases they tie each QR code to a single survey that becomes obsolete as soon as it ends.

This Bachelor's Thesis presents Krate, a web voting platform designed for collaborative decision-making. A manager creates a catalogue of options, defines votings from it and assigns them to voting points, each with a fixed QR code that only needs to be printed once: what is shown when the code is scanned is decided by the system at every moment, so votings can change without reprinting anything. Participants vote from their own phone without signing up or installing any application, using one of three voting types (single choice, multiple choice or ranking by order of preference), and the manager reviews the results from a single place. The use case that gave rise to the project is a network of vending machines whose customers choose the products they want to find in them.

Beyond the product itself, the project has been conceived as a complete software engineering exercise: an agile methodology with regular reviews with an external client is followed, a complete requirements specification is produced, an architecture made up of a server and two independent web applications is designed, the system is implemented with Spring Boot and Angular, it is verified with automated tests and load tests, and it is deployed to the cloud using containers and a continuous integration and deployment pipeline. The outcome is a working, deployed product and a report that documents the decisions taken and the deviations from the initial plan.

Key words: Voting; User engagement; QR code; REST API; CI/CD

Índice general

Índice general

VII

1	Introducción	1
1.1	Motivación personal	1
1.2	Objetivos	2
1.3	Estructura de la memoria	3
2	Estado del arte	5
2.1	Análisis de soluciones existentes en el mercado	5
2.1.1	Plataformas de formularios genéricos (SaaS)	5
2.1.2	Sistemas de captura de feedback por hardware dedicado	6
2.1.3	Sistemas de votación digitales académicos y corporativos	6
2.2	Crítica al estado del arte	6
2.3	Propuesta tecnológica del trabajo	7
2.4	Conclusiones sobre el estado del arte	7
3	Metodología y planificación	9
3.1	Prácticas ágiles implementadas	9
3.1.1	Gestión del backlog y el flujo de trabajo	10
3.1.2	Gestión del conocimiento y registro diario	10
3.1.3	Reuniones periódicas	11
3.2	Planificación inicial del proyecto	11
3.3	Conclusiones del uso de prácticas ágiles	12
4	Identificación y especificación de requisitos	13
4.1	Actores	13
4.1.1	Modelo Personas	14
4.2	Casos de uso	15
4.2.1	Gestión de acceso	15
4.2.2	Configuración logística de la red de votación	16
4.2.3	Ciclo de vida del proceso de votación	18
4.3	Historias de usuario	20
4.3.1	Aplicación de gestión	20
4.3.2	Interfaz de votación	21
4.4	Requisitos funcionales	22
4.4.1	Autenticación y acceso	22
4.4.2	Gestión de ítems	22
4.4.3	Gestión de puntos de votación	23
4.4.4	Gestión de votaciones	23
4.4.5	Interfaz de votación	24
4.4.6	Seguridad	25
4.4.7	Estadísticas	25
4.5	Requisitos no funcionales	25
4.5.1	Usabilidad	25
4.5.2	Rendimiento	26
4.5.3	Seguridad	26

4.6	Matrices de trazabilidad	27
4.6.1	Trazabilidad entre requisitos funcionales y casos de uso	27
4.6.2	Trazabilidad entre requisitos funcionales e historias de usuario	27
4.7	Modelo de dominio	30
4.8	Conclusiones tras la especificación de requisitos	30
5	Diseño y arquitectura del software	33
5.1	Arquitectura del sistema	33
5.1.1	Visión general y componentes	33
5.1.2	Despliegue	34
5.1.3	Integración y despliegue continuo	36
5.1.4	Principios transversales	36
5.2	Diseño detallado	36
5.2.1	Organización interna del backend	36
5.2.2	Modelo de datos	38
5.2.3	Tipos de votación	39
5.2.4	Seguridad y autenticación	40
5.2.5	Ciclo de vida de una ronda y resolución del código QR	41
5.2.6	Diseño de los frontends	42
5.3	Conclusiones del diseño	43
6	Tecnologías utilizadas	45
6.1	Backend	45
6.1.1	Java	45
6.1.2	Spring Boot	46
6.1.3	Spring Data JPA e Hibernate	46
6.1.4	Spring Security	46
6.1.5	Flyway	47
6.1.6	springdoc-openapi	47
6.1.7	Maven	47
6.2	Frontend	47
6.2.1	TypeScript	47
6.2.2	Angular	47
6.2.3	CSS propio y recursos empaquetados	48
6.2.4	qrcode	48
6.3	Base de datos	48
6.3.1	PostgreSQL	48
6.4	Despliegue e infraestructura	48
6.4.1	Docker y Docker Compose	48
6.4.2	nginx	49
6.4.3	Caddy	49
6.4.4	GitHub, GitHub Actions y GitHub Container Registry	49
6.4.5	Hetzner Cloud y Ubuntu Server	50
6.4.6	DuckDNS	50
6.5	Pruebas	50
6.5.1	JUnit 5, AssertJ y MockMvc	50
6.5.2	Testcontainers	50
6.5.3	JaCoCo	51
6.5.4	Vitest	51
6.5.5	k6	51
6.6	Entorno y herramientas de desarrollo	52
6.6.1	Sistemas operativos y entornos de desarrollo	52
6.6.2	Herramientas de apoyo	52
6.6.3	Asistentes basados en modelos de lenguaje	52

6.7	Conclusiones de la selección tecnológica	52
7	Desarrollo e implementación de la solución	55
7.1	Sprint 0: lanzamiento	55
7.2	Sprint 1: ingeniería de requisitos	56
7.3	Sprint 2: cierre de la especificación y arquitectura	56
7.4	Sprint 3: núcleo del backend	57
7.5	Sprint 4: panel de gestión	58
7.6	Sprint 5: interfaz de votación, rondas y códigos QR	59
7.7	Sprint 6: tipos de votación y estadísticas	60
7.8	Sprint 7: pruebas, seguridad y despliegue continuo	61
7.9	Sprint 8: estabilización y cierre	62
7.10	Recorrido de uso de la aplicación	63
7.10.1	El gestor: de la cuenta a las estadísticas	63
7.10.2	El votante: del código QR a la papeleta	66
7.11	Conclusiones del desarrollo	67
8	Pruebas y calidad	69
8.1	Estrategia y automatización	69
8.2	Pruebas unitarias del backend	70
8.3	Pruebas de integración del backend	70
8.4	Pruebas de los frontends y comprobaciones manuales	72
8.5	Validación con el usuario	73
8.6	Pruebas de carga	73
8.7	Cobertura de código	74
8.8	Limitaciones	75
8.9	Conclusiones de las pruebas	75
9	Conclusiones y trabajo futuro	77
9.1	Relación con los contenidos del grado	77
9.2	Trabajo futuro	78
	Bibliografía	79
	Apéndices	
A	Objetivos de Desarrollo Sostenible	81
B	Glosario de términos	83
C	Informe de la prueba de carga	87

CAPÍTULO 1

Introducción

El voto es uno de los derechos fundamentales recogidos por las Naciones Unidas [15]. Y no es ninguna minucia: ya desde la antigua Grecia son conocidos el poder y el valor de la acción de votar. En la misma Edad de Oro ateniense, también conocida como Pentecontecia (490-431 a. C.), se instauró la llamada *democracia radical* bajo el gobierno de Pericles [2]. Este sistema se basaba en el voto de todos los ciudadanos de la Asamblea a la hora de tomar las decisiones de gobierno y decidir las leyes que gobernarían la polis.

Hoy, más de dos mil años después y tras muchos sistemas fallidos de por medio, se ha vuelto al voto como sistema de gobierno. Esto no es casualidad; el voto es una manera notablemente eficaz de tomar decisiones y llegar a consensos cuando se está tratando con grupos. Permite a cada individuo que participa en la votación expresar su opinión y postura de forma rápida y directa, mientras se asegura que la opinión de todos los votantes valga lo mismo. Además, un recuento de votos permite obtener los resultados de forma clara y sencilla.

La efectividad y el poder del voto no solo se ven en los grandes procesos electorales que deciden los gobiernos democráticos. Tomar decisiones votando es algo que se hace constantemente: a la hora de planificar una reunión en el trabajo, en la junta de vecinos cuando se debe decidir si repintar el portal e incluso a la hora de decidir a dónde ir de viaje con amigos. La vida está repleta de momentos en los que se debe decidir algo en grupo y para estos casos una votación suele ser la mejor solución.

Pero para que una votación sea efectiva debe ser efectuada correctamente y con las herramientas adecuadas. Y ese es el tema que se tratará en este trabajo: la construcción de una plataforma de votación, una aplicación de gestión de dichas votaciones y los procesos de ingeniería del software involucrados en la creación de un producto software de calidad.

El producto que se desarrolla a lo largo de este trabajo recibe el nombre de Krate. El nombre procede de la palabra griega *kratos*, «poder» o «gobierno», la misma raíz que da lugar al término «democracia», el gobierno del pueblo, y juega a la vez con la palabra inglesa *crate*, «caja», en alusión a la urna en la que tradicionalmente se depositan los votos.

1.1 Motivación personal

La idea de este proyecto nace de la fusión de dos elementos. El primero es la creencia personal de que los avances tecnológicos actuales tienen la capacidad de transformar la democracia representativa actual en una democracia directa y digital [13]. Aunque este trabajo no va a abordar esa problemática concreta, puesto que resulta demasiado

ambiciosa para las capacidades actuales del autor y el alcance de este proyecto, es un tema que interesa al autor desde hace varios años. El hecho de que este proyecto sea algo similar a las herramientas necesarias para esta democracia digital, aunque a una escala razonablemente menor, motiva a llevarlo adelante.

El segundo de estos elementos es la colaboración con una amistad personal del autor. La idea nace en el momento en que esta persona decide comprar unas máquinas expendedoras y busca un elemento que separe sus máquinas del resto. Esto llevó a la idea de permitir que los usuarios pudiesen votar los productos que querían ver en las máquinas. Esta idea resultó interesante por varios motivos. El primero es que este sistema genera una relación con el usuario final: si este ve que sus decisiones y votaciones se reflejan en las máquinas, aumentan las probabilidades de que quiera volver a comprar en ellas. Otro de los motivos es la posibilidad de crear una herramienta software generalizada y que no solo sirviese para su caso concreto. Y el último motivo es el desafío de ser capaz de aplicar los procesos de ingeniería del software al completo en el ciclo de desarrollo de un producto. No hay mejor manera de terminar los estudios en la rama de Ingeniería del Software que plasmando los conocimientos adquiridos en todas las fases del desarrollo, puesto que lo que diferencia a un ingeniero de software de otros perfiles profesionales del sector no es su capacidad de programar, sino su capacidad de aplicar conocimientos científicos en el diseño, construcción, operación, documentación y mantenimiento del software [3].

1.2 Objetivos

Este Trabajo de Fin de Grado tiene como objetivo principal construir un producto software de calidad que permita llevar a cabo votaciones, gestionirlas y acceder a ellas mediante códigos QR estáticos. A continuación se enumeran los objetivos de forma específica.

En primer lugar, estos son los objetivos a nivel de producto:

1. Construir una plataforma para llevar a cabo votaciones.
2. Construir una aplicación que permita crear y gestionar estas votaciones de forma centralizada.
3. Permitir que los gestores accedan a las estadísticas de las votaciones.
4. Permitir que los gestores puedan designar distintos grupos o puntos de votación que actúen como entidades separadas, permitiendo que se lancen distintas votaciones simultáneas.
5. Desarrollar la tecnología que permita que cada punto de votación tenga un código QR estático que permita acceder a la votación de dicho punto.

A continuación se enumeran los objetivos a nivel de ingeniería del software.

1. Llevar a cabo un proceso de identificación y especificación de requisitos completo.
2. Implementar una metodología de trabajo compuesta por distintos elementos de las metodologías ágiles.
3. Implementar un sistema de empaquetado y aislamiento de la aplicación que garantice la consistencia del software y la paridad entre los entornos de desarrollo y ejecución.

4. Establecer pruebas que controlen la integridad del código y el rendimiento del producto.
5. Implementar un flujo de integración y despliegue continuos que se encargue de ejecutar las pruebas de forma automática durante el desarrollo.
6. Desplegar el producto en la nube, listo para el uso en entornos reales.

Con estos objetivos se busca construir un producto software de calidad y plasmar todos los procesos que lo hacen posible.

1.3 Estructura de la memoria

Esta memoria está escrita de forma que refleja el proceso de desarrollo lógico. Sin embargo, al haber seguido un enfoque metodológico ágil, el orden de los capítulos no representa una línea temporal cronológica.

- **Capítulo 2 – Estado del arte:** Se realiza una exploración de las tecnologías existentes y las alternativas posibles al producto que se busca desarrollar, culminando con la justificación y presentación de la solución propuesta.
- **Capítulo 3 – Metodología y planificación:** Se describe la metodología de trabajo adoptada, los flujos de trabajo seguidos y la planificación inicial del proyecto.
- **Capítulo 4 – Identificación y especificación de requisitos:** Se recogen y detallan los requisitos funcionales y no funcionales del sistema, junto con la definición de actores, casos de uso, historias de usuario y las matrices de trazabilidad correspondientes.
- **Capítulo 5 – Diseño y arquitectura del software:** Se presenta el diseño de la solución en dos niveles: la arquitectura del sistema y su despliegue, y el diseño detallado del *backend*, el modelo de datos, la seguridad y el ciclo de vida de una ronda de votación.
- **Capítulo 6 – Tecnologías utilizadas:** Se exponen las herramientas, tecnologías y *frameworks* utilizados en el desarrollo del proyecto.
- **Capítulo 7 – Desarrollo e implementación de la solución:** Se detalla, *sprint a sprint*, la implementación llevada a cabo, los patrones de diseño empleados y se recogen los principales problemas encontrados durante el desarrollo. Adicionalmente, se narra un ejemplo del uso de la aplicación, junto con imágenes del producto real.
- **Capítulo 8 – Pruebas y calidad:** Se describen las pruebas diseñadas para verificar y validar el sistema junto con las pruebas de rendimiento, así como los resultados obtenidos tras su ejecución.
- **Capítulo 9 – Conclusiones y trabajo futuro:** Se recogen las conclusiones derivadas del trabajo realizado, valorando los resultados obtenidos y planteando posibles líneas de trabajo futuro.

Adicionalmente, cada capítulo cuenta con unas breves conclusiones finales escritas tras la finalización del proyecto. En estas se expresa cómo el trabajo realizado ha acabado aportando valor al proyecto junto con los cambios y problemas encontrados.

Por último, al final del documento se incluyen tres anexos: la relación del trabajo con los **Objetivos de Desarrollo Sostenible**; un **glosario de términos**, de especial utilidad para aquellos lectores que no estén familiarizados con la terminología específica del área, y al que se puede recurrir en cualquier momento para aclarar el significado de conceptos técnicos empleados a lo largo del texto; y el **informe de la prueba de carga**.

CAPÍTULO 2

Estado del arte

Antes de embarcarse en un proyecto es necesario conocer el ecosistema tecnológico en el que este se va a desarrollar. Conocer las alternativas presentes en el mercado, sus debilidades y fortalezas permite determinar mejor las oportunidades que ofrece el entorno. Por esto, en este capítulo se van a estudiar las soluciones existentes para la recogida de información de usuarios mediante interfaces de votación y su gestión centralizada. Tras el estudio se realizará una propuesta de valor y una comparativa con las opciones existentes.

2.1 Análisis de soluciones existentes en el mercado

La recogida de información y retroalimentación (*feedback*) del usuario final no es un concepto nuevo. Existen cientos de soluciones disponibles, cada una con sus características y cualidades concretas. Por ello, llevar a cabo el análisis de una selección concreta no resulta práctico en este dominio. Sin embargo, se han detectado tres grandes patrones bajo los que se puede agrupar la gran mayoría de soluciones disponibles. Cada uno de estos enfoques presenta ciertas ventajas y desventajas frente al resto, ocupando cada uno de estos un lugar diferenciado en el mercado.

2.1.1. Plataformas de formularios genéricos (SaaS)

Soluciones consolidadas como *Typeform*¹, *Google Forms*² o *Slido*³ permiten la creación ágil de encuestas digitales con interfaces genéricas. Estas herramientas ofrecen una gran flexibilidad en el diseño de preguntas; sin embargo, carecen por completo de una capa de lógica de negocio orientada a la reutilización de elementos comunes a través de distintas encuestas. Tampoco permiten asociar encuestas de forma dinámica a una red jerárquica de puntos físicos, obligando al gestor a generar, imprimir y sustituir códigos QR independientes cada vez que se desea rotar la encuesta de un punto concreto.

En definitiva, el enfoque SaaS tiene como mayor desventaja su punto más fuerte: ser genérico. Al no decantarse por lógicas de negocio complejas y buscar acomodarse a la mayor cantidad de casos posibles, estas soluciones acaban siendo muy flexibles, pero a la vez muy poco profundas.

¹Página oficial de Typeform: <https://www.typeform.com/es>.

²Página oficial de Google Forms: <https://workspace.google.com/intl/es/products/forms/>.

³Página oficial de Slido: <https://www.slido.com/>.

2.1.2. Sistemas de captura de feedback por hardware dedicado

Dispositivos físicos basados en botoneras mecánicas o pantallas táctiles monopregunta (cuyo máximo exponente comercial es el sistema *HappyOrNot*⁴) se despliegan en entornos de venta minorista y servicios públicos. Si bien ofrecen una fricción casi nula para el usuario, presentan un coste de adquisición e infraestructura muy elevado. Además, su capacidad semántica suele ser extremadamente limitada, reduciéndose normalmente a métricas ordinales de satisfacción. Esto los vuelve inviables para procesos de decisión colaborativa complejos, como la elección de ítems específicos de un catálogo o el lanzamiento de votaciones parametrizadas.

Este enfoque se inclina de manera clara hacia la conexión entre las encuestas virtuales y el mundo físico, pero de manera excesiva para la mayoría de casos. Es el caso opuesto al enfoque anterior: es excesivamente específico y restrictivo. Además, la barrera de entrada es notablemente alta al requerir la instalación de hardware específico.

2.1.3. Sistemas de votación digitales académicos y corporativos

Existen plataformas orientadas a la toma de decisiones formal o al sufragio electrónico (como *Evoting*⁵ o sistemas basados en *blockchain*). Estas soluciones priorizan el blindaje criptográfico y la verificación unívoca de la identidad del votante. No obstante, este enfoque requiere obligatoriamente procesos de registro complejos, verificación de credenciales o instalación de aplicaciones dedicadas. Estos compromisos necesarios en el acceso resultan incompatibles con entornos en los que se busca una interacción rápida y generar la menor fricción posible con el usuario.

2.2 Crítica al estado del arte

Tras el análisis de las soluciones disponibles se identifican las siguientes lagunas tecnológicas no abordadas:

1. **Acoplamiento rígido entre el identificador QR y el recurso:** Las soluciones tradicionales asocian el código QR a una URL estática de una encuesta. Al finalizar el periodo de votación, el soporte físico impreso queda obsoleto. Existe una laguna en el diseño de arquitecturas que propongan un **QR persistente**, donde el *token* físico actúe como un puntero abstracto en base de datos hacia el estado activo de un punto de votación.
2. **Ausencia de un modelo analítico centralizado y reutilizable:** Los sistemas analizados procesan las respuestas de manera aislada. No existe un nexo de unión entre el catálogo de ítems global de una organización y el rendimiento histórico de dichos ítems cuando estos aparecen repetidamente en distintas votaciones.
3. **Vulnerabilidad al fraude en entornos anónimos:** Las herramientas web que prescinden de inicio de sesión suelen delegar el control de duplicados a mecanismos débiles basados exclusivamente en *cookies* de cliente, fácilmente eludibles. El reto tecnológico radica en diseñar un sistema de seguridad perimetral ligero mediante *tokens* efímeros combinados con restricciones de red que mitiguen el fraude sin comprometer el anonimato ni la usabilidad extrema.

⁴Página oficial de HappyOrNot: <https://www.happy-or-not.com/>.

⁵Página oficial de Evoting: <https://evoting.es/>.

Tras el análisis se obtiene la siguiente tabla comparativa entre los diferentes enfoques:

Enfoque	Fricción	Flexibilidad	Seguridad	Desacoplamiento
Plataformas SaaS	Muy baja	Alta	Muy baja	Inexistente
Hardware dedicado	Nula	Nula	Alta	Total
Sistemas académicos	Muy alta	Baja	Muy alta	Nulo
Propuesta TFG	Muy baja	Alta	Media-Alta	Total

2.3 Propuesta tecnológica del trabajo

Como respuesta a las carencias identificadas, este Trabajo de Fin de Grado propone el desarrollo de una plataforma web modular y reutilizable que cierra la brecha entre la inmediatez física y el control analítico centralizado.

- **Desacoplamiento mediante abstracción física:** Implementación de un modelo en el que los códigos QR son fijos e invariables para cada localización, delegando en el *backend* la resolución dinámica de qué votación y qué ítems se renderizan en la interfaz de votación en función de reglas de negocio horarias o activaciones manuales.
- **Seguridad asimétrica sin fricción:** Mitigación del fraude y la repetición de votos mediante un sistema que no requiera la creación de cuentas por parte de los usuarios y que sea capaz de soportar intentos básicos de vulneración.
- **Centralización analítica relacional:** Un panel de gestión que trace el rendimiento de un catálogo de ítems a través de múltiples puntos con votaciones independientes, permitiendo la toma de decisiones basada en datos históricos de consumo.

2.4 Conclusiones sobre el estado del arte

El estudio del estado del arte ha permitido cerrar la idea de negocio de forma clara y estratégica. Observar los enfoques más comunes ha permitido encontrar el nicho de mercado que la solución propuesta busca cubrir. También ha permitido conocer los puntos fuertes de la tecnología existente para establecerlos como estándares a los que aspirar y conocer los puntos débiles y fallos como errores a evitar.

En definitiva, el estudio del estado del arte no solo ha aportado contexto teórico, sino que ha tenido un impacto práctico sobre el diseño final del sistema, ayudando a definir una propuesta tecnológica diferenciada y alineada con necesidades reales no resueltas de forma eficiente por las soluciones actuales.

CAPÍTULO 3

Metodología y planificación

Para realizar el desarrollo de un producto software se llevan a cabo distintos procesos, desde el diseño teórico de la arquitectura de la solución hasta la implementación de las pruebas. La clave para gestionar los diversos procesos de manera estructurada y eficiente es adoptar una metodología de desarrollo de software. Las metodologías no son algo exclusivo de la ingeniería del software, pero sí que hay metodologías concretas para el desarrollo de software de calidad. En especial se destacan dos enfoques principales: el tradicional y el ágil [12].

Para el desarrollo que se documenta en esta memoria se ha elegido el enfoque ágil por su orientación hacia la adaptabilidad y la flexibilidad¹. Gracias a esto, el producto no estará atado a las decisiones tomadas en el inicio del proyecto y estará abierto a cambios para adaptarse a revisiones en el diseño, nuevos requisitos y barreras técnicas que vayan surgiendo durante el desarrollo. Este enfoque resulta especialmente útil en desarrollos llevados a cabo por una sola persona, ya que es altamente improbable que todas las decisiones tomadas al inicio del proyecto, como los requisitos o la arquitectura a seguir, sean completamente correctas y óptimas. Esto se puede deber a varios motivos, ya sea la falta de conocimiento a nivel teórico del dominio del problema, la falta de conocimiento técnico en alguna de las tecnologías utilizadas o una estimación del esfuerzo incorrecta. El enfoque ágil permitirá que los cambios que vayan surgiendo no rompan el proyecto desde la base y permitirá un desarrollo más fluido.

3.1 Prácticas ágiles implementadas

En las asignaturas de Proceso de Software y Proyecto de Ingeniería de Software, se desmiente el mito de que, para llevar a cabo un desarrollo ágil, se deba implementar al pie de la letra una de las metodologías existentes. En lugar de limitarse a elegir un marco popular como *Scrum*, *Kanban* o *Extreme Programming*², la clave reside en entender qué prácticas ágiles conforman cada una de estas metodologías y aplicar solo aquellas que se adapten al proyecto y al equipo de desarrollo en concreto. A continuación se detallan las prácticas que se han llevado a cabo a lo largo del proyecto.

¹Para más información sobre los valores y principios de este enfoque, véase el Manifiesto Ágil: <https://agilemanifesto.org/iso/es/manifesto.html>.

²Breve presentación de las principales metodologías ágiles y principales diferencias: <https://www.wrike.com/project-management-guide/faq/what-are-the-different-types-of-agile-methodologies/>.

3.1.1. Gestión del backlog y el flujo de trabajo

Para la planificación y el seguimiento del estado del proyecto se ha utilizado *Trello*³ como herramienta de gestión. A través de ella se han implementado las siguientes dinámicas.

Sprints y tablero Kanban

El desarrollo se ha estructurado en nueve *sprints* de una semana de duración. El objetivo de tener *sprints* cortos es evaluar el progreso de manera constante y pivotar rápidamente ante imprevistos técnicos gracias a la flexibilidad que otorga tener bloques de trabajo pequeños. El estado de estos *sprints* se visualiza mediante un tablero *Kanban*, lo que permite organizar el esfuerzo realizado y por realizar de cada *sprint* de manera visual e inmediata.

Unidades de trabajo y flujos de trabajo

El trabajo se ha categorizado en diferentes tipos de elementos para mantener un orden claro en el *backlog*:

- **Épicas:** Grandes bloques de trabajo. Suelen estar divididas en historias de usuario o tareas.
- **Historias de usuario:** Funcionalidades descritas desde la perspectiva del usuario final, que sirven para guiar el desarrollo enfocado en el valor que aporta el producto final.
- **Tareas:** Unidades de trabajo técnicas necesarias para el proyecto pero que no aportan valor directo al usuario de forma aislada.
- **Bugs:** Informes de fallos encontrados durante las pruebas que requieren corrección.

En el tablero, cada tarjeta lleva una etiqueta de color que indica su tipo, y todas siguen un mismo flujo de trabajo que define los estados en los que se pueden encontrar las distintas unidades de trabajo y que, por lo tanto, se corresponden con las columnas presentes en el tablero *Kanban*. El flujo es el siguiente: *Por hacer*, *En progreso* y *Completada*. Este flujo de trabajo permite organizar el progreso en las diferentes actividades dentro de cada *sprint*. El resto de tareas que aún no han sido añadidas a algún *sprint* se encuentran almacenadas en el *backlog*.

3.1.2. Gestión del conocimiento y registro diario

Dada la naturaleza individual del proyecto, la gestión de la información es crítica, por lo que se ha utilizado la herramienta *Obsidian*⁴. Esta ha permitido almacenar diagramas, documentos y diferentes tipos de notas, desde ideas fugaces hasta la especificación de requisitos del proyecto al completo.

Además, se ha mantenido un registro diario del trabajo realizado, los bloqueos encontrados, las soluciones aplicadas y las tareas pendientes para el día siguiente. Esto no solo ha facilitado la agilidad a la hora de retomar el trabajo, sino que ha servido como base de conocimiento y cuaderno de bitácora para la posterior redacción de esta memoria.

³Página oficial de Trello: <https://trello.com/>.

⁴Página oficial de Obsidian: <https://obsidian.md/>.

3.1.3. Reuniones periódicas

Aunque el desarrollo ha sido llevado a cabo por una sola persona, se han realizado reuniones con dos actores externos para mantener el rumbo del proyecto:

- **Tutor del TFG:** Ha actuado como asesor técnico y metodológico de alto nivel, asegurando el cumplimiento de los objetivos académicos. Las reuniones se han dado en puntos clave del proyecto, principalmente el inicio y el final de este.
- **Product Owner externo:** La amistad personal mencionada en el apartado 1.1 ha asumido el rol de *Product Owner*. Mediante reuniones al finalizar cada *sprint*, ha actuado como cliente externo para evaluar el producto, revisar las historias de usuario terminadas y priorizar el *backlog* de cara al siguiente ciclo.

3.2 Planificación inicial del proyecto

Una de las claves del éxito de cualquier proyecto es la planificación, especialmente en la ingeniería del software. En el caso particular de las metodologías ágiles, la planificación suele realizarse al inicio del proyecto, pero no tiene tanto peso como en metodologías tradicionales como puede ser el desarrollo en cascada, ya que la planificación inicial se realiza con la flexibilidad en mente. La planificación que se muestra a continuación ha funcionado más como una guía orientativa que como una serie de directrices que seguir al pie de la letra.

Los nueve *sprints* se organizan conceptualmente en cuatro fases principales según los objetivos generales que abordan:

Fase 0: Lanzamiento y requisitos (Sprints 0 y 1) Estas dos primeras semanas se plantearon como el núcleo estratégico del proyecto. El *sprint* 0 se planificó exclusivamente para la preparación logística, el aprovisionamiento de los entornos de desarrollo locales y la configuración inicial de las herramientas de gestión. Acto seguido, el *sprint* 1 se concibió para la ingeniería de requisitos, orientándose a la definición del sistema mediante la obtención y especificación de requisitos, la estructuración del *backlog* inicial y el diseño del modelo de dominio.

Fase 1: Núcleo del sistema y administración (Sprints 2 y 3) El arranque del desarrollo técnico se proyectó sobre la construcción de los cimientos arquitectónicos del software. Para el *sprint* 2 se priorizó la definición de la infraestructura del *backend*, la inicialización de la base de datos relacional mediante contenedores y el desarrollo de las operaciones CRUD esenciales junto a un flujo de integración continua básico. Con esta base consolidada, el *sprint* 3 se destinó al desarrollo del panel de gestión, dotando al sistema de su interfaz de administración.

Fase 2: Funcionalidad cliente y analítica (Sprints 4 y 5) Las semanas centrales se reservaron para el canal del usuario final y las capacidades avanzadas de la aplicación. El *sprint* 4 se diseñó para abordar la implementación de la lógica e interfaz de las votaciones junto con la lógica de generación y permanencia de códigos QR. Posteriormente, el *sprint* 5 se planificó para expandir el sistema añadiendo valor analítico a través de un *dashboard* visual de estadísticas.

Fase 3: Calidad, despliegue y cierre (Sprints 6, 7 y 8) El último bloque de la planificación inicial se orientó hacia la robustez, la puesta en producción y los entregables académicos. El *sprint* 6 se concibió para el refuerzo de la seguridad y la ejecución de pruebas de rendimiento. Durante el *sprint* 7 se previó la automatización del despliegue continuo hacia la nube y la ejecución del *testing* final. Por último, el *sprint* 8 se reservó como un margen de estabilización para la corrección de defectos menores y la puesta a punto final.

3.3 Conclusiones del uso de prácticas ágiles

Tal y como es habitual en este tipo de proyectos, la planificación inicial no representó con precisión el orden real de la implementación, ya que diversos factores técnicos y de aprendizaje desviaron el desarrollo por un camino distinto. Fue en este escenario de incertidumbre donde las prácticas ágiles resultaron cruciales para gestionar los cambios sin comprometer el avance del proyecto.

Por un lado, la segmentación del esfuerzo en *sprints* breves de una semana otorgó la flexibilidad necesaria para reorganizar los bloques de trabajo tras cada imprevisto. Por otro lado, el mantenimiento de un *backlog* estructurado en épicas e historias de usuario impidió la dispersión, manteniendo siempre visibles los objetivos críticos del sistema. Asimismo, las sesiones de revisión con el *Product Owner* y el tutor actuaron como un mecanismo de validación frecuente, asegurando que las desviaciones del plan original siguieran aportando valor real al producto. Finalmente, la toma de notas y el registro del trabajo diario no solo mantuvieron las ideas estructuradas durante el desarrollo, sino que han resultado de gran utilidad a la hora de documentar el proceso en esta memoria.

El desglose completo de cómo se ejecutó cada uno de los *sprints*, las modificaciones que sufrió el plan inicial y la materialización técnica del producto se exponen y analizan en profundidad en el capítulo 7.

CAPÍTULO 4

Identificación y especificación de requisitos

Para guiar el diseño de la solución es necesario conocer el dominio del problema que se aborda. Con ese conocimiento se podrán acotar unos objetivos y requisitos concretos que deberá cumplir el producto final. En la ingeniería del software, el proceso de conocer y especificar dichos requisitos se conoce como «ingeniería de requisitos». Es un paso de suma importancia puesto que marca los objetivos que se seguirán durante todo el proyecto. Si el proceso de ingeniería de requisitos se realiza incorrectamente, las decisiones y diseños de la solución que partan de este probablemente también sean incorrectos y el coste de solucionar estos problemas escala exponencialmente conforme el proyecto va madurando [19]. Por esto, en la mayoría de las metodologías de desarrollo se incentiva a no escatimar en la especificación inicial y realizar revisiones periódicas de estos requisitos conforme va avanzando el proyecto, cambian las necesidades de los *stakeholders* y va aumentando el conocimiento en el dominio del problema.

A continuación se van a exponer los resultados de la especificación de requisitos realizada.

4.1 Actores

El primer paso en la obtención de los requisitos es identificar a los distintos actores que interactúan en los procesos que se llevan a cabo.

AC-01 — Usuario no autenticado: Cualquier persona que accede a la plataforma web pero que aún no se ha identificado. Este actor interactúa exclusivamente con los flujos de creación de cuenta (Registrarse) y validación de identidad (Iniciar sesión). Una vez que el sistema valida sus credenciales de manera correcta, este actor cambia de rol y pasa a ser un gestor.

AC-02 — Gestor: Es el actor principal de la aplicación de gestión. Representa a cualquier usuario u organización autenticados que administran la plataforma. Tiene permisos para crear y configurar votaciones, gestionar el catálogo de ítems, definir los puntos de votación y consultar las estadísticas globales.

AC-03 — Votante: Usuario final que participa en una votación activa. Su interacción con el sistema se realiza de forma anónima accediendo directamente a través de un enlace web o escaneando un código QR en un punto físico. No requiere registro ni inicio de sesión previo e interactúa exclusivamente con la interfaz de emisión de voto.

4.1.1. Modelo Personas

Con el objetivo de modelar el comportamiento, las necesidades y las restricciones de los usuarios finales del sistema, se ha empleado la metodología de *User Personas* propuesta originalmente por Alan Cooper [6]. Este enfoque de diseño permite sintetizar los datos de los usuarios arquetípicos en fichas operativas, facilitando que el desarrollo técnico responda de manera directa a las motivaciones y frustraciones reales de las personas que interactuarán con el producto final.

Para este proyecto se han considerado dos perfiles claramente diferenciados, el gestor y el votante:

P-01 — El gestor: Pablo Domínguez	
Edad	38 años
Ocupación	Responsable de operaciones en una empresa de <i>vending</i> .
Contexto	Gestiona una red de 20 máquinas expendedoras repartidas por oficinas y centros de trabajo de la ciudad.
Motivación	Quiere saber qué productos gustan más en cada localización para optimizar el <i>restock</i> y reducir el desperdicio. Además, busca generar implicación y fidelidad con los usuarios de sus máquinas, ofreciéndoles un canal de participación que les haga sentir escuchados, los haga más propensos a volver a comprar y diferencie a sus máquinas de la competencia.
Frustraciones	Los métodos actuales (preguntar directamente, formularios en papel) son lentos, poco fiables y difíciles de consolidar. Las alternativas digitales no permiten tener un QR estático en cada máquina.
Objetivos	Lanzar votaciones periódicas sin esfuerzo, ver resultados en tiempo real y tomar decisiones basadas en datos.
Habilidades técnicas	Nivel medio. Cómodo con herramientas web, pero no tiene perfil técnico.
Dispositivo principal	Ordenador de escritorio en oficina.
P-02 — El votante: Alicia de Felipe	
Edad	26 años
Ocupación	Empleada de oficina.
Contexto	Usa la máquina expendedora del trabajo a diario. Ve un código QR pegado en la máquina.
Motivación	Le gustaría que hubiera un sabor concreto de una bebida energética, pero nunca ha tenido un canal para expresarlo.
Frustraciones	No quiere registrarse en ninguna plataforma ni instalar ninguna aplicación solo para votar.
Objetivos	Participar de forma rápida y anónima desde su móvil.
Habilidades técnicas	Nivel alto. Usa el móvil con fluidez para todo.
Dispositivo principal	<i>Smartphone</i> .

4.2 Casos de uso

A continuación se detallan las especificaciones de los casos de uso que componen el sistema, estructurados en tres grupos lógicos. Se muestran en primer lugar los diagramas UML de cada grupo seguidos de las especificaciones de los casos de uso correspondientes. El modelo de especificación de los casos de uso adoptado sigue los principios propuestos por Alistair Cockburn en su libro «*Writing Effective Use Cases*» [5], pero optando por un punto medio entre los enfoques *casual* y *fully dressed*. De esta forma la especificación utilizada cuenta con los campos de identificador, actor, descripción, precondition, postcondición y flujo principal. El resto de apartados que llevarían la especificación hacia el enfoque *fully dressed* que presentaba Cockburn no se han considerado necesarios para el nivel de análisis requerido.

4.2.1. Gestión de acceso

Este grupo engloba los casos de uso que conciernen a los procesos de autenticación, registro y control de seguridad para el acceso a la plataforma. En él se especifican las interacciones que permiten a los usuarios no autenticados darse de alta en el sistema o identificarse de manera segura, así como el posterior cierre de su sesión activa. En la figura 4.1 se ilustra el diagrama conceptual de este módulo.

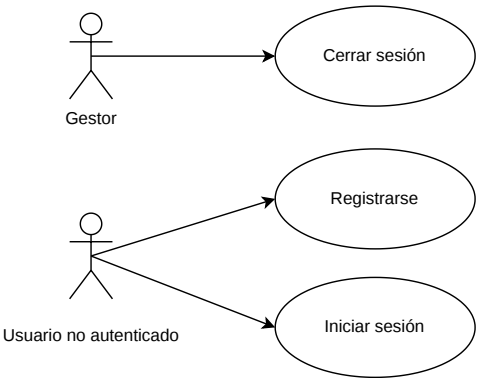


Figura 4.1: Diagrama de casos de uso correspondiente al módulo de gestión de acceso.

Campo	Detalle
Caso de uso	CU-01 — Registrarse como gestor
Actor	Usuario no autenticado
Descripción	Un usuario se registra en la plataforma para obtener acceso a la aplicación de gestión.
Precondición	El usuario no tiene cuenta registrada en la plataforma.
Postcondición	Habr� una cuenta creada con los datos indicados en caso de que sean v�licos.
Flujo principal	1. El usuario accede a la p�gina de registro. 2. Introduce sus datos (nombre, correo y contrase�a). 3. El sistema valida los datos y crea la cuenta. 4. El sistema redirige al usuario a la aplicaci�n de gesti�n.

Campo	Detalle
Caso de uso	CU-02 — Iniciar sesión
Actor	Usuario no autenticado
Descripción	El usuario se autentica en la aplicación de gestión mediante sus credenciales.
Precondición	El usuario tiene una cuenta registrada en la plataforma.
Postcondición	El usuario accederá a la aplicación con su cuenta de gestor si los datos introducidos son correctos.
Flujo principal	1. El usuario accede a la página de inicio de sesión. 2. Introduce su correo y contraseña. 3. El sistema valida las credenciales. 4. El usuario accede a la aplicación de gestión.

Campo	Detalle
Caso de uso	CU-03 — Cerrar sesión
Actor	Gestor
Descripción	El gestor cierra su sesión activa en la aplicación de gestión.
Precondición	El gestor tiene una sesión activa.
Postcondición	Se cerrará la sesión activa.
Flujo principal	1. El gestor selecciona la opción de cerrar sesión. 2. El sistema redirige al gestor a la página de inicio de sesión.

4.2.2. Configuración logística de la red de votación

Este grupo de casos de uso comprende la definición y organización de los componentes esenciales que articulan la plataforma para los gestores. En él se detallan las acciones que permiten al gestor administrar el catálogo de ítems que pueden someterse a votación y dar de alta los diferentes puntos físicos de votación, vinculándolos a códigos QR específicos para su posterior acceso. Asimismo, este módulo integra las herramientas de supervisión y análisis que consolidan las métricas de participación a través de gráficas estadísticas. En la figura 4.2 se ilustra el diagrama de casos de uso de este módulo.

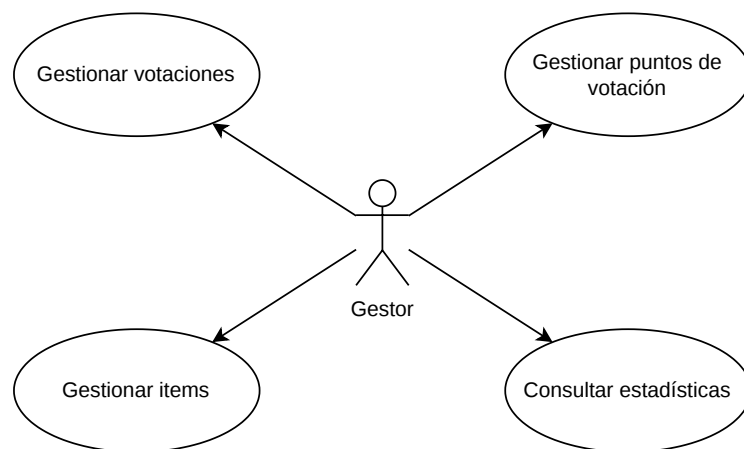


Figura 4.2: Diagrama de casos de uso correspondiente al módulo de configuración logística de la red de votación.

Campo	Detalle
Caso de uso	CU-04 — Gestionar ítems
Actor	Gestor
Descripción	El gestor crea, edita o elimina ítems del catálogo de la plataforma.
Precondición	El gestor tiene una sesión activa.
Postcondición	Los cambios hechos por el gestor en los ítems se reflejarán en la aplicación y la base de datos.
Flujo principal	1. El gestor accede a la sección de gestión de ítems. 2. El sistema muestra el catálogo de ítems existentes. 3. El gestor crea un nuevo ítem introduciendo nombre, descripción e imagen; o selecciona uno existente para editarlo o eliminarlo. 4. El sistema guarda los cambios y actualiza el catálogo.

Campo	Detalle
Caso de uso	CU-05 — Gestionar puntos de votación
Actor	Gestor
Descripción	El gestor crea, edita o elimina puntos de votación, y descarga el código QR asociado a cada uno.
Precondición	El gestor tiene una sesión activa.
Postcondición	Los cambios hechos por el gestor en los puntos de votación se reflejarán en la aplicación y la base de datos.
Flujo principal	1. El gestor accede a la sección de puntos de votación. 2. El sistema muestra los puntos existentes con su estado actual. 3. El gestor crea un nuevo punto introduciendo nombre y descripción; o selecciona uno existente para editarlo o eliminarlo. 4. El sistema genera un código QR persistente asociado al punto. 5. El gestor puede obtener el QR para su impresión o compartir el enlace directo.

Campo	Detalle
Caso de uso	CU-06 — Gestionar votaciones
Actor	Gestor
Descripción	El gestor crea, edita o elimina votaciones, y las asigna a uno o varios puntos de votación.
Precondición	El gestor tiene una sesión activa y al menos un ítem y un punto de votación creados.
Postcondición	Los cambios hechos por el gestor en las votaciones se reflejarán en la aplicación y la base de datos.
Flujo principal	1. El gestor accede a la sección de votaciones. 2. El sistema muestra las votaciones existentes. 3. El gestor crea una nueva votación definiendo nombre, descripción, ítems participantes y número máximo de votos por usuario; o selecciona una existente para editarla o eliminarla. 4. El gestor asigna la votación a uno o varios puntos de votación. 5. El sistema guarda la configuración.

Campo	Detalle
Caso de uso	CU-07 — Consultar estadísticas
Actor	Gestor
Descripción	El gestor consulta el panel de estadísticas para analizar los resultados de las votaciones.
Precondición	El gestor tiene una sesión activa y existe al menos una votación con votos registrados.
Postcondición	El sistema muestra las gráficas y métricas filtradas según las peticiones del gestor.
Flujo principal	1. El gestor accede al panel de estadísticas. 2. El sistema muestra los resultados globales de todas las votaciones e ítems. 3. El gestor puede filtrar por votación concreta, punto de votación o ítem para obtener información más detallada.

4.2.3. Ciclo de vida del proceso de votación

Este bloque engloba los casos de uso que gobiernan el estado, la ejecución y el comportamiento de las votaciones dentro de la plataforma. En él se especifican los mecanismos críticos de apertura y cierre de las votaciones, así como el flujo de interacción mediante el cual los usuarios escanean los enlaces o códigos QR para acceder a la votación y emiten su voto, aplicando los controles técnicos necesarios para garantizar la integridad y la unicidad del voto. El diagrama conceptual de este proceso se detalla en la figura 4.3.

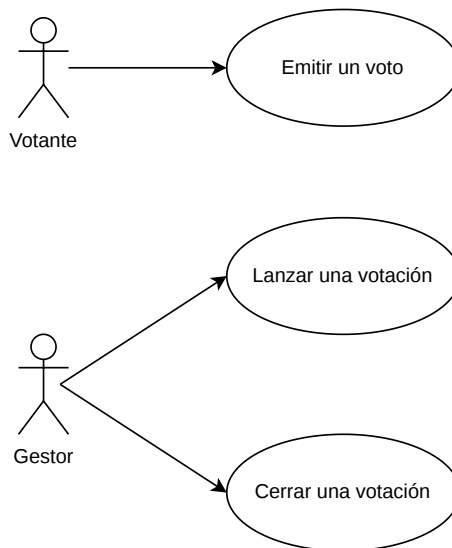


Figura 4.3: Diagrama de casos de uso correspondiente al módulo del ciclo de vida del proceso de votación.

Campo	Detalle
Caso de uso	CU-08 — Lanzar una votación
Actor	Gestor
Descripción	El gestor activa una votación.
Precondición	El gestor tiene una sesión activa y la votación está creada y asignada a al menos un punto de votación.
Postcondición	La votación pasará a estar en estado activo en los puntos de votación correspondientes.
Flujo principal	1. El gestor lanza manualmente una votación. 2. El sistema activa la votación en los puntos asignados en el momento indicado. 3. La votación activa se asigna a la dirección del punto de votación, asegurándose de que el QR sigue siendo válido.

Campo	Detalle
Caso de uso	CU-09 — Cerrar una votación
Actor	Gestor
Descripción	El gestor cierra una votación activa.
Precondición	Existe una votación activa en al menos un punto de votación.
Postcondición	La votación pasará a estar en estado inactivo y se detendrá la recepción de nuevos votos.
Flujo principal	1. El gestor cierra la votación manualmente. 2. El sistema marca la votación como cerrada. 3. La interfaz de votación de los puntos afectados muestra el mensaje de votación inactiva.

Campo	Detalle
Caso de uso	CU-10 — Emitir un voto
Actor	Votante
Descripción	El votante accede a la interfaz de votación y emite su voto sobre los ítems disponibles.
Precondición	Existe una votación activa en el punto de votación al que accede el votante.
Postcondición	El voto queda registrado de forma segura y se emite la restricción para evitar duplicados.
Flujo principal	1. El votante escanea el código QR o accede al enlace directo del punto de votación. 2. El sistema comprueba si el votante ya ha participado en la votación activa. Si es así, muestra un mensaje informativo y finaliza. 3. El sistema muestra los ítems de la votación activa. 4. El votante selecciona uno o varios ítems respetando el límite configurado y confirma su voto. 5. El sistema registra el voto y muestra una confirmación al votante.

4.3 Historias de usuario

Los casos de uso son una herramienta de gran utilidad para guiar el diseño del producto, pero a la hora de la gestión y planificación interna las metodologías ágiles aportan una opción más conveniente, las historias de usuario. En este proyecto se ha empleado la estructura estándar analizada por Jeff Patton [16]. Este estándar presenta la estructura según la cual una historia de usuario debe seguir el patrón «Como [rol], quiero [acción], para [beneficio]». Además, cada historia incorpora sus respectivos criterios de aceptación, los cuales acotan el alcance técnico del requisito y servirán como base para el diseño del plan de pruebas funcionales del sistema.

Estas historias de usuario se han organizado en el *backlog* del proyecto en dos épicas, dependiendo de a qué sección del producto hacen referencia.

4.3.1. Aplicación de gestión

En esta épica se han englobado aquellas historias de usuario que involucran a la aplicación de gestión. Esto incluye los procesos de acceso y autenticación, la gestión de los diferentes elementos a disposición del gestor y el acceso a las estadísticas.

HU-01: Registro de cuenta

Enunciado: Como gestor, quiero poder registrarme en la plataforma con mi nombre, correo y contraseña, para obtener acceso a la aplicación de gestión.

Criterios de aceptación:

- El sistema valida que el correo no esté ya registrado.
- La contraseña cumple unos requisitos mínimos de seguridad.
- Tras el registro, el gestor es redirigido a la aplicación de gestión.

HU-02: Inicio y cierre de sesión

Enunciado: Como gestor, quiero poder iniciar y cerrar sesión en la plataforma, para acceder a mis recursos de forma segura.

Criterios de aceptación:

- El sistema valida las credenciales de inicio de sesión.
- Al cerrar sesión se invalidan las credenciales.
- Las rutas de gestión son inaccesibles sin una sesión activa con credenciales válidas.

HU-03: Gestión de ítems

Enunciado: Como gestor, quiero poder crear, editar y eliminar ítems en mi catálogo, para mantener actualizada la oferta disponible en mis votaciones.

Criterios de aceptación:

- Se puede añadir un ítem con nombre, descripción e imagen.
- Se puede modificar cualquier campo de un ítem existente.
- Se puede eliminar un ítem del catálogo.
- El catálogo muestra todos los ítems creados.

HU-04: Gestión de puntos de votación

Enunciado: Como gestor, quiero poder crear, editar y eliminar puntos de votación, para representar cada grupo lógico o localización física donde se realizarán las votaciones.

Criterios de aceptación:

- Se puede crear un punto con nombre y descripción.
- Se puede editar o eliminar un punto existente.
- Al crear un punto se genera automáticamente un código QR persistente.

HU-05: Acceso al QR y enlace del punto

Enunciado: Como gestor, quiero poder descargar el código QR de cada punto de votación y compartir su enlace directo, para facilitar el acceso a los votantes.

Criterios de aceptación:

- El QR se puede descargar en un formato apto para impresión.
- El enlace directo se puede copiar con un solo clic.

HU-06: Gestión de votaciones

Enunciado: Como gestor, quiero poder crear, editar y eliminar votaciones, para configurar las encuestas que se lanzarán en mis puntos de votación.

Criterios de aceptación:

- Se puede crear una votación con nombre, descripción, ítems y límite de votos.
- Se puede editar una votación que aún no ha comenzado.
- Se puede eliminar una votación existente.
- Se puede consultar el listado completo de votaciones creadas.

HU-07: Lanzamiento y finalización de votaciones

Enunciado: Como gestor, quiero poder lanzar y finalizar votaciones, para controlar cuándo están activas en cada punto de votación.

Criterios de aceptación:

- Se puede asignar una votación a uno o varios puntos de votación.
- Se puede lanzar una votación en los puntos asignados.
- Se puede terminar una votación activa.

HU-08: Consulta de estadísticas

Enunciado: Como gestor, quiero consultar un panel de estadísticas con los resultados de mis votaciones, para tomar decisiones basadas en datos.

Criterios de aceptación:

- Se muestran estadísticas globales de todas las votaciones e ítems.
- Se pueden filtrar los resultados por votación, punto de votación o ítem.
- Se puede consultar el historial de rendimiento de cada ítem.

4.3.2. Interfaz de votación

En esta época se han englobado aquellas historias de usuario que involucran directamente la interfaz de votación y los procesos que ocurren en esta.

HU-09: Acceso a la votación activa

Enunciado: Como votante, quiero acceder a la votación activa de un punto escaneando el QR o usando el enlace directo, para participar sin necesidad de registro.

Criterios de aceptación:

- El acceso no requiere registro ni inicio de sesión.
- Si hay votación activa se muestran los ítems disponibles.

- Si no hay votación activa se muestra un mensaje informativo.
- Si el votante ya ha participado se muestra un mensaje indicándolo.

HU-10: Emisión del voto

Enunciado: Como votante, quiero seleccionar mis ítems preferidos y confirmar mi voto, para hacer llegar mi opinión al gestor de forma sencilla.

Criterios de aceptación:

- El votante puede seleccionar uno o varios ítems respetando el límite.
- El voto se confirma con una sola acción.
- El sistema muestra una confirmación tras registrar el voto.
- El proceso completo se realiza en menos de tres interacciones.

4.4 Requisitos funcionales

En esta sección se describen los requisitos funcionales del sistema, que comprenden las capacidades y servicios que el sistema debe proporcionar de manera obligatoria para satisfacer las necesidades de los usuarios y asegurar el correcto comportamiento de la aplicación. Los requisitos se presentan clasificados en grupos lógicos que representan los diferentes módulos funcionales del sistema. La especificación de los requisitos funcionales se ha llevado a cabo siguiendo los principios del estándar ISO/IEC/IEEE 29148 [10], aunque con un nivel de detalle menos minucioso debido a las necesidades del proyecto.

4.4.1. Autenticación y acceso

Este módulo engloba los requisitos destinados a garantizar la seguridad de la plataforma, gestionando el ciclo de vida de las sesiones administrativas de los gestores y facilitando un canal de entrada directo, ágil y sin fricciones de registro para los usuarios con rol de votante.

ID	Nombre	Descripción	Actor	Prioridad
RF-01	Inicio de sesión del gestor	El gestor puede autenticarse en la aplicación de gestión mediante usuario y contraseña.	Gestor	Alta
RF-02	Cierre de sesión	El gestor puede cerrar su sesión de forma segura.	Gestor	Alta
RF-03	Acceso público a votación	El votante puede acceder a la interfaz de votación mediante un código QR sin necesidad de registro ni inicio de sesión.	Votante	Alta
RF-04	Registro de gestores	Un usuario puede acceder a la página y, en caso de no tener cuenta o querer crear otra, puede registrarse con sus datos.	Gestor	Alta

4.4.2. Gestión de ítems

Agrupar las funciones encargadas del mantenimiento integral de los elementos tangibles o abstractos que serán objeto de escrutinio. El sistema debe proveer los mecanismos

necesarios para el ciclo de vida clásico de estos datos (creación, lectura, modificación y borrado) dentro del catálogo maestro.

ID	Nombre	Descripción	Actor	Prioridad
RF-05	Crear ítem	El gestor puede añadir nuevos ítems al catálogo, especificando nombre, descripción e imagen.	Gestor	Alta
RF-06	Editar ítem	El gestor puede modificar los datos de un ítem existente.	Gestor	Media
RF-07	Eliminar ítem	El gestor puede eliminar un ítem del catálogo.	Gestor	Alta
RF-08	Listar ítems	El gestor puede consultar el catálogo completo de ítems disponibles.	Gestor	Alta

4.4.3. Gestión de puntos de votación

Este bloque define las capacidades logísticas de la aplicación para estructurar la red física o digital de votación. Regula cómo se definen los puntos de votación y la forma en que el sistema expone los puntos de entrada interactivos mediante enlaces dinámicos y códigos QR.

ID	Nombre	Descripción	Actor	Prioridad
RF-09	Crear punto de votación	El gestor puede crear un punto de votación asignándole un nombre y una descripción.	Gestor	Alta
RF-10	Editar punto de votación	El gestor puede modificar los datos de un punto de votación existente.	Gestor	Media
RF-11	Eliminar punto de votación	El gestor puede eliminar un punto de votación.	Gestor	Alta
RF-12	Listar puntos de votación	El gestor puede consultar todos los puntos de votación con su estado actual.	Gestor	Alta
RF-13	Generar código QR persistente	Cada punto de votación tiene asociado un código QR fijo que siempre redirige a la votación activa en ese momento.	Sistema	Alta
RF-14	Descargar QR de cada punto	El gestor puede descargar el QR de cada punto de votación de forma individual en un formato que facilite su impresión.	Gestor	Baja
RF-15	Compartir enlace de votación	El gestor puede copiar y compartir el enlace directo a la votación activa de un punto concreto, como alternativa al acceso por QR.	Gestor	Alta

4.4.4. Gestión de votaciones

Este bloque agrupa los requisitos enfocados en el motor de negocio de las jornadas electorales. Detalla las herramientas de parametrización temporal, asignación de depen-

dencias (ítems y puntos) y los mecanismos tanto imperativos (ejecución manual) como declarativos (planificación automática) para alterar el estado operativo de los escrutinios.

ID	Nombre	Descripción	Actor	Prioridad
RF-16	Crear votación	El gestor puede crear una votación definiendo nombre, descripción, ítems participantes, número máximo de votos y duración.	Gestor	Alta
RF-17	Editar votación	El gestor puede modificar los parámetros de una votación que aún no ha comenzado.	Gestor	Media
RF-18	Eliminar votación	El gestor puede eliminar una votación existente.	Gestor	Alta
RF-19	Listar votaciones	El gestor puede consultar todas las votaciones que ha creado.	Gestor	Alta
RF-20	Asignar votación a punto/s	El gestor puede asociar una votación a uno o varios puntos de votación.	Gestor	Alta
RF-21	Lanzar votación	El gestor puede iniciar una votación en un punto de votación.	Gestor	Alta
RF-22	Cerrar votación	El gestor puede cerrar una votación activa en un punto de votación.	Gestor	Alta

4.4.5. Interfaz de votación

Este módulo define las capacidades de la interfaz pública expuesta al votante. Rige la visualización dinámica del contenido en función del estado de la red logística, la emisión del voto y las restricciones que previenen que un usuario vote más veces de las permitidas.

ID	Nombre	Descripción	Actor	Prioridad
RF-23	Mostrar votación activa	La interfaz de votación muestra los ítems de la votación activa en el punto de votación correspondiente.	Votante	Alta
RF-24	Emitir voto	El votante puede seleccionar uno o varios ítems y emitir su voto, respetando el límite máximo configurado.	Votante	Alta
RF-25	Confirmación de voto	Tras votar, el sistema muestra una confirmación al votante.	Sistema	Media
RF-26	Mostrar votación inactiva	Si no hay votación activa en el punto de votación, la interfaz muestra un mensaje informativo al votante.	Sistema	Media
RF-27	Control de voto múltiple	El sistema impide que un mismo usuario emita más votos de los permitidos, mostrando un mensaje informativo.	Sistema	Media

4.4.6. Seguridad

Abarca las restricciones operacionales aplicadas transversalmente sobre la infraestructura. Se encarga de aislar el núcleo administrativo y las funciones críticas de configuración frente a cualquier intento de acceso no autorizado.

ID	Nombre	Descripción	Actor	Prioridad
RF-28	Protección de rutas de gestión	Las funcionalidades de gestión solo son accesibles para usuarios autenticados en el sistema.	Sistema	Alta

4.4.7. Estadísticas

Este último bloque funcional se centra en la capa de visualización de los datos. El gestor deberá ser capaz de organizar, filtrar y acceder a la información de las votaciones que haya lanzado.

ID	Nombre	Descripción	Actor	Prioridad
RF-29	Panel de estadísticas general	El gestor puede consultar un panel con estadísticas globales de todas las votaciones e ítems.	Gestor	Media
RF-30	Estadísticas por votación	El gestor puede consultar los resultados detallados de una votación concreta.	Gestor	Media
RF-31	Estadísticas por punto de votación	El gestor puede filtrar las estadísticas por punto de votación.	Gestor	Media
RF-32	Estadísticas por ítem	El gestor puede consultar el rendimiento histórico de un ítem concreto a lo largo de las votaciones.	Gestor	Media

4.5 Requisitos no funcionales

En esta sección se especifican los requisitos no funcionales del sistema. Estos requisitos definen las propiedades, restricciones de diseño, directrices de calidad y criterios de comportamiento que la aplicación debe cumplir transversalmente. A diferencia de los funcionales, los requisitos no funcionales no describen qué hace el sistema, sino cómo lo hace. La especificación se ha llevado a cabo siguiendo el mismo esquema basado en el estándar ISO/IEC/IEEE 29148 [10] que los requisitos funcionales, pero sin incluir la columna de actor. Para su clasificación, se han agrupado en categorías inspiradas en las estipuladas por el estándar internacional de calidad del software ISO/IEC 25010 [17], con el objetivo de que la plataforma resulte robusta, segura, eficiente y fácil de mantener.

4.5.1. Usabilidad

Este bloque agrupa las restricciones de calidad orientadas a la experiencia del usuario final (UX). Define los criterios que garantizan que las interfaces de votación pública y de gestión sean accesibles, intuitivas e independientes del dispositivo de acceso.

ID	Nombre	Descripción	Prioridad
RNF-01	Interfaz de votación simple	La interfaz de votación debe permitir a cualquier usuario emitir su voto en menos de tres interacciones desde que accede al enlace del QR.	Alta
RNF-02	Diseño <i>responsive</i>	Tanto la interfaz de votación como la aplicación de gestión deben adaptarse correctamente a dispositivos móviles y escritorio.	Baja
RNF-03	Compatibilidad con navegadores	La plataforma debe funcionar correctamente en las últimas versiones estables de <i>Chrome</i> , <i>Firefox</i> , <i>Safari</i> y <i>Edge</i> .	Alta
RNF-04	Interfaz en español	La plataforma debe estar disponible en español como idioma principal de la aplicación.	Alta

4.5.2. Rendimiento

Establece los límites cuantitativos y los umbrales temporales tolerables para la carga del sistema y las transferencias en la red. Regula la eficiencia del *backend* y la velocidad de respuesta de las vistas operacionales, asegurando una experiencia fluida tanto bajo condiciones normales como en picos de alta concurrencia.

ID	Nombre	Descripción	Prioridad
RNF-05	Tiempo de carga de la votación	La interfaz de votación debe cargarse en menos de dos segundos en condiciones normales de red.	Alta
RNF-06	Tiempo de respuesta de la API	La API REST debe responder a las peticiones entrantes en menos de 800 ms bajo condiciones de carga normal.	Media
RNF-07	Tiempo de respuesta bajo carga alta	Bajo condiciones de carga elevada o picos de concurrencia, el tiempo de respuesta de la API no debe superar los 2500 ms.	Media

4.5.3. Seguridad

Regula las directrices obligatorias para salvaguardar la confidencialidad, la integridad de los datos de los escrutinios y el control de acceso perimetral. Comprende las tecnologías de cifrado en tránsito, la persistencia protegida de sesiones de administración y las contramedidas lógicas para evitar el fraude electoral local.

ID	Nombre	Descripción	Prioridad
RNF-08	Autenticación robusta para el gestor	El acceso a la aplicación de gestión debe estar protegido de forma que solo se pueda acceder iniciando sesión.	Alta
RNF-09	Comunicación cifrada	Todas las comunicaciones entre el cliente y el servidor deben realizarse de forma obligatoria mediante el protocolo HTTPS.	Media
RNF-10	Protección contra voto múltiple	El sistema debe utilizar alguna tecnología para evitar que un mismo usuario supere el límite de votos permitido, sin requerir registro previo.	Alta

4.6 Matrices de trazabilidad

Las matrices de trazabilidad o RTM (*Requirements Traceability Matrix*) son herramientas de ingeniería del software que permiten correlacionar los distintos elementos de la especificación de requisitos. Estas forman parte del estándar ISO/IEC/IEEE 29148 [10] según el cual la trazabilidad es una propiedad indispensable para la gestión de cambios, el análisis de impacto y la verificación del sistema.

En este proyecto resultan relevantes las dos matrices de trazabilidad que se muestran a continuación.

4.6.1. Trazabilidad entre requisitos funcionales y casos de uso

La presente matriz establece la correspondencia entre los requisitos funcionales (RF) y los casos de uso (CU) descritos en las secciones anteriores. Al estructurar los requisitos en las filas del eje vertical, se facilita la verificación de la cobertura funcional del sistema, permitiendo constatar visualmente que el 100 % de las necesidades técnicas del software cuenta con un flujo de comportamiento diseñado para su ejecución.

4.6.2. Trazabilidad entre requisitos funcionales e historias de usuario

Esta matriz conecta la especificación técnica de los requisitos funcionales con las historias de usuario (HU) del *backlog* de desarrollo. Esta traza es fundamental para validar la consistencia metodológica del proyecto, vinculando el «qué hace el sistema» (enfoque de ingeniería tradicional) con el «qué valor aporta al usuario» (enfoque ágil).

ID de requisito	Nombre del requisito	Caso de uso
RF-01	Inicio de sesión del gestor	CU-02
RF-02	Cierre de sesión	CU-03
RF-03	Acceso público a votación	CU-10
RF-04	Registro de gestores	CU-01
RF-05	Crear ítem	CU-04
RF-06	Editar ítem	CU-04
RF-07	Eliminar ítem	CU-04
RF-08	Listar ítems	CU-04
RF-09	Crear punto de votación	CU-05
RF-10	Editar punto de votación	CU-05
RF-11	Eliminar punto de votación	CU-05
RF-12	Listar puntos de votación	CU-05
RF-13	Generar código QR persistente	CU-05, CU-08
RF-14	Descargar QR de cada punto	CU-05
RF-15	Compartir enlace de votación	CU-05
RF-16	Crear votación	CU-06
RF-17	Editar votación	CU-06
RF-18	Eliminar votación	CU-06
RF-19	Listar votaciones	CU-06
RF-20	Asignar votación a punto/s	CU-06
RF-21	Lanzar votación	CU-08
RF-22	Cerrar votación	CU-09
RF-23	Mostrar votación activa	CU-10
RF-24	Emitir voto	CU-10
RF-25	Confirmación de voto	CU-10
RF-26	Mostrar votación inactiva	CU-09, CU-10
RF-27	Control de voto múltiple	CU-10
RF-28	Protección de rutas de gestión	CU-02, CU-03
RF-29	Panel de estadísticas general	CU-07
RF-30	Estadísticas por votación	CU-07
RF-31	Estadísticas por punto de votación	CU-07
RF-32	Estadísticas por ítem	CU-07

ID de requisito	Nombre del requisito	Historia de usuario
RF-01	Inicio de sesión del gestor	HU-02
RF-02	Cierre de sesión	HU-02
RF-03	Acceso público a votación	HU-09
RF-04	Registro de gestores	HU-01
RF-05	Crear ítem	HU-03
RF-06	Editar ítem	HU-03
RF-07	Eliminar ítem	HU-03
RF-08	Listar ítems	HU-03
RF-09	Crear punto de votación	HU-04
RF-10	Editar punto de votación	HU-04
RF-11	Eliminar punto de votación	HU-04
RF-12	Listar puntos de votación	HU-04
RF-13	Generar código QR persistente	HU-04
RF-14	Descargar QR de cada punto	HU-05
RF-15	Compartir enlace de votación	HU-05
RF-16	Crear votación	HU-06
RF-17	Editar votación	HU-06
RF-18	Eliminar votación	HU-06
RF-19	Listar votaciones	HU-06
RF-20	Asignar votación a punto/s	HU-07
RF-21	Lanzar votación	HU-07
RF-22	Cerrar votación	HU-07
RF-23	Mostrar votación activa	HU-09
RF-24	Emitir voto	HU-10
RF-25	Confirmación de voto	HU-10
RF-26	Mostrar votación inactiva	HU-09
RF-27	Control de voto múltiple	HU-09
RF-28	Protección de rutas de gestión	HU-02
RF-29	Panel de estadísticas general	HU-08
RF-30	Estadísticas por votación	HU-08
RF-31	Estadísticas por punto de votación	HU-08
RF-32	Estadísticas por ítem	HU-08

4.7 Modelo de dominio

Tras la realización de la especificación de requisitos primaria se pueden empezar a esbozar las relaciones entre las distintas entidades que forman el dominio del problema. Para organizar estas estructuras de manera visual se ha construido un modelo de dominio¹.

El modelo realizado busca mostrar las entidades principales que conforman el sistema y a su vez declarar relaciones entre ellas y sus restricciones. El modelo de dominio no es una declaración literal de cómo se relacionan los datos en el producto final; para ello se ha realizado el diseño detallado del capítulo 5, donde se discute la arquitectura de la aplicación.

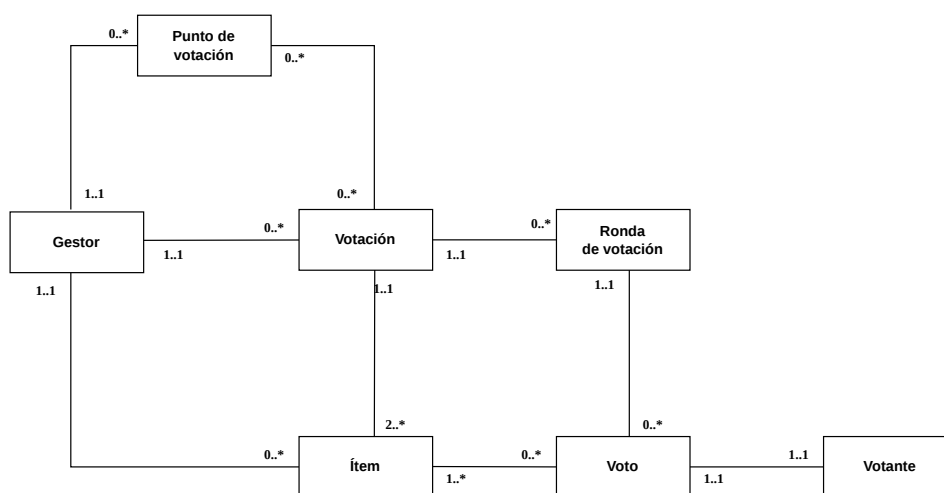


Figura 4.4: Modelo de dominio.

La figura 4.4 expresa claramente cómo se relacionan los diferentes elementos del dominio, que son el gestor, la votación, la ronda de votación, el ítem que se vota, el votante, el voto y el punto de votación. Entre estos términos el que destaca es la ronda de votación, ya que es el único que no es inmediatamente evidente o no es algo que se haya mencionado previamente. Pero es justo en casos como este donde el valor de la realización de un modelo de dominio reluce. Al modelar las relaciones entre el voto y la votación se ha observado que era necesario un punto intermedio que permitiese distinguir entre diferentes instancias de votaciones y es justo ahí donde nace la ronda de votación.

4.8 Conclusiones tras la especificación de requisitos

La realización de una especificación de requisitos completa y en profundidad ha aportado un gran valor al proyecto. Ha permitido tener claros los objetivos desde el inicio del desarrollo, priorizar las tareas y tener una visión clara y completa del producto. Esto último en especial ha resultado de vital importancia, puesto que gracias a tener una visión completa del producto que se buscaba construir se han podido tomar decisiones más informadas y precisas a la hora de elegir y maqueter la arquitectura de la aplicación.

Los requisitos han evolucionado a lo largo del desarrollo: se han creado, eliminado, refinado y priorizado los requisitos iniciales hasta llegar a la lista final presentada en la

¹Definición de IBM de *modelo de dominio*: <https://www.ibm.com/docs/es/ida/9.1.2?topic=types-domain-models>.

memoria. Gracias a la adopción de las metodologías ágiles y las reuniones cada *sprint* con el *Product Owner* no se ha perdido el foco del proyecto y se ha podido adaptar a los cambios en los requisitos sin problemas mayores.

Un último aspecto a destacar en la especificación de requisitos es la calidad de los estándares presentes. Estos han pintado una clara hoja de ruta que seguir, con plantillas y métodos claros que han ayudado a que la identificación y especificación de requisitos sea un proceso de ingeniería preciso.

CAPÍTULO 5

Diseño y arquitectura del software

Una vez identificados y especificados los requisitos del sistema, el siguiente paso del proceso de ingeniería consiste en decidir cómo se va a construir la solución. Este capítulo recoge el diseño de Krate en los dos niveles de abstracción habituales. En primer lugar se presenta la arquitectura del sistema: los grandes bloques que componen la solución, cómo se comunican entre sí y cómo se despliegan. En segundo lugar se desciende al diseño detallado: la organización interna de cada bloque, el modelo de datos, el mecanismo de seguridad y el ciclo de vida de una votación. En ambos niveles cada decisión se justifica a partir de los requisitos del capítulo 4 y del modelo de dominio de la figura 4.4, y se indica qué alternativas se han descartado y por qué.

Conviene fijar antes la terminología. En el modelo de dominio se introdujo la ronda de votación como el concepto que distingue cada lanzamiento de una votación en un punto concreto. En el código y en la base de datos ese mismo concepto recibe el nombre de instancia (`VotingInstance`, tabla `voting_instances`), ya que es un término más utilizado en contextos técnicos; sin embargo, en esta memoria se sigue hablando de rondas de votación, porque resulta un término más natural y sencillo de entender. Del mismo modo, la participación de un dispositivo en una ronda se denomina papeleta, y cada ítem elegido dentro de esa papeleta, voto. Esta distinción no existía en el modelo de dominio inicial y resulta necesaria para dar soporte a los distintos tipos de votación, como se explica en el apartado 5.2.3.

5.1 Arquitectura del sistema

5.1.1. Visión general y componentes

Krate sigue una arquitectura cliente-servidor en la que toda la lógica de negocio y todo el estado residen en un único servicio de *backend* que expone una API REST [7], y en la que los clientes son aplicaciones web de una sola página (SPA) que se ejecutan íntegramente en el navegador. Sobre esa base, la decisión estructural más visible es la separación de la solución en tres componentes independientes, que se compilan, prueban y despliegan por separado: el panel de gestión, la interfaz de votación y la API. Los tres se apoyan en una única base de datos relacional *PostgreSQL*, tal y como muestra la figura 5.1.

La existencia de dos *frontends* separados responde directamente a los dos actores identificados en el capítulo 4. El gestor trabaja desde un ordenador de escritorio, inicia sesión, navega por varias pantallas y rellena formularios. El votante llega escaneando un código QR con su teléfono o pulsando un enlace, no tiene cuenta, dispone de unos segundos de atención y puede encontrarse en una red de mala calidad; los requisitos RNF-01 y RNF-05 exigen que pueda votar en menos de tres interacciones y que la pantalla cargue en me-

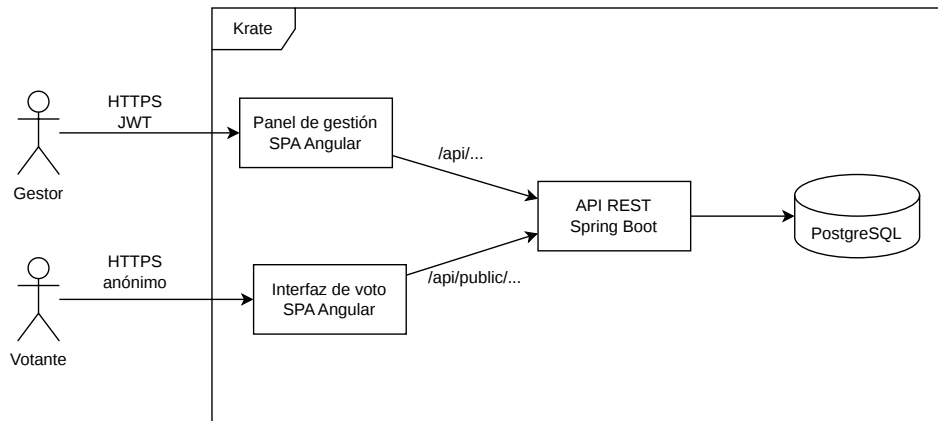


Figura 5.1: Componentes del sistema y actores que los utilizan.

nos de dos segundos. Una única aplicación habría obligado al votante a descargar todo el código del panel, que nunca va a utilizar, y habría mezclado en un mismo despliegue rutas públicas y privadas. Separarlas permite que la interfaz de votación sea mínima, que ambas evolucionen de forma independiente y, como se verá en el apartado 5.1.2, que se sirvan desde dominios distintos. Ambas se comunican con el *backend* exclusivamente a través de la API, y la interfaz de votación consume únicamente su parte pública, lo que reduce la superficie expuesta a los usuarios anónimos a dos operaciones: consultar el estado de un punto y enviar una papeleta.

La aplicación tampoco depende de ningún servicio de terceros en tiempo de ejecución: no hay proveedor de identidad, servicio de correo ni almacenamiento en la nube, y la tipografía y los iconos van empaquetados en las propias aplicaciones, de modo que el navegador del votante solo se comunica con Krate. Los puntos de votación son localizaciones físicas cuya conectividad no se controla, y cualquier llamada a terceros desde ese navegador añadiría puntos de fallo ajenos. Los servicios externos que aparecen en el despliegue público descrito en el apartado 5.1.2, el alojamiento, la resolución de nombres y la emisión de certificados, pertenecen a la infraestructura y no al sistema: son intercambiables, y la misma pila puede ejecutarse en una red local sin salida a internet, en cuyo caso la interfaz de votación recurre a la alternativa que ya contempla para generar el identificador del votante sin HTTPS.

5.1.2. Despliegue

La contenerización de todos los componentes es uno de los objetivos de ingeniería fijados en el capítulo 1. El sistema se ejecuta como una pila de contenedores *Docker* orquestada con *Docker Compose*, idéntica en la máquina de desarrollo y en el servidor de producción salvo por un fichero de sobrescritura que ajusta puertos, imágenes y variables. La figura 5.2 representa la pila de producción: dos contenedores *nginx* sirven los ficheros estáticos de cada SPA, un tercero ejecuta la API sobre la máquina virtual de Java, el cuarto es *PostgreSQL* y el quinto, presente solo en producción, es *Caddy*, un *proxy* inverso que termina las conexiones TLS. Los datos, las imágenes subidas y los certificados residen en volúmenes persistentes, de modo que los contenedores pueden destruirse y recrearse sin pérdida de información.

Tres decisiones de este nivel merecen explicación. La primera es que los *frontends* nunca conocen la dirección del *backend*: ambas aplicaciones llaman a rutas relativas bajo */api/*, y es el *nginx* que las sirve quien redirige esas peticiones al contenedor de la API

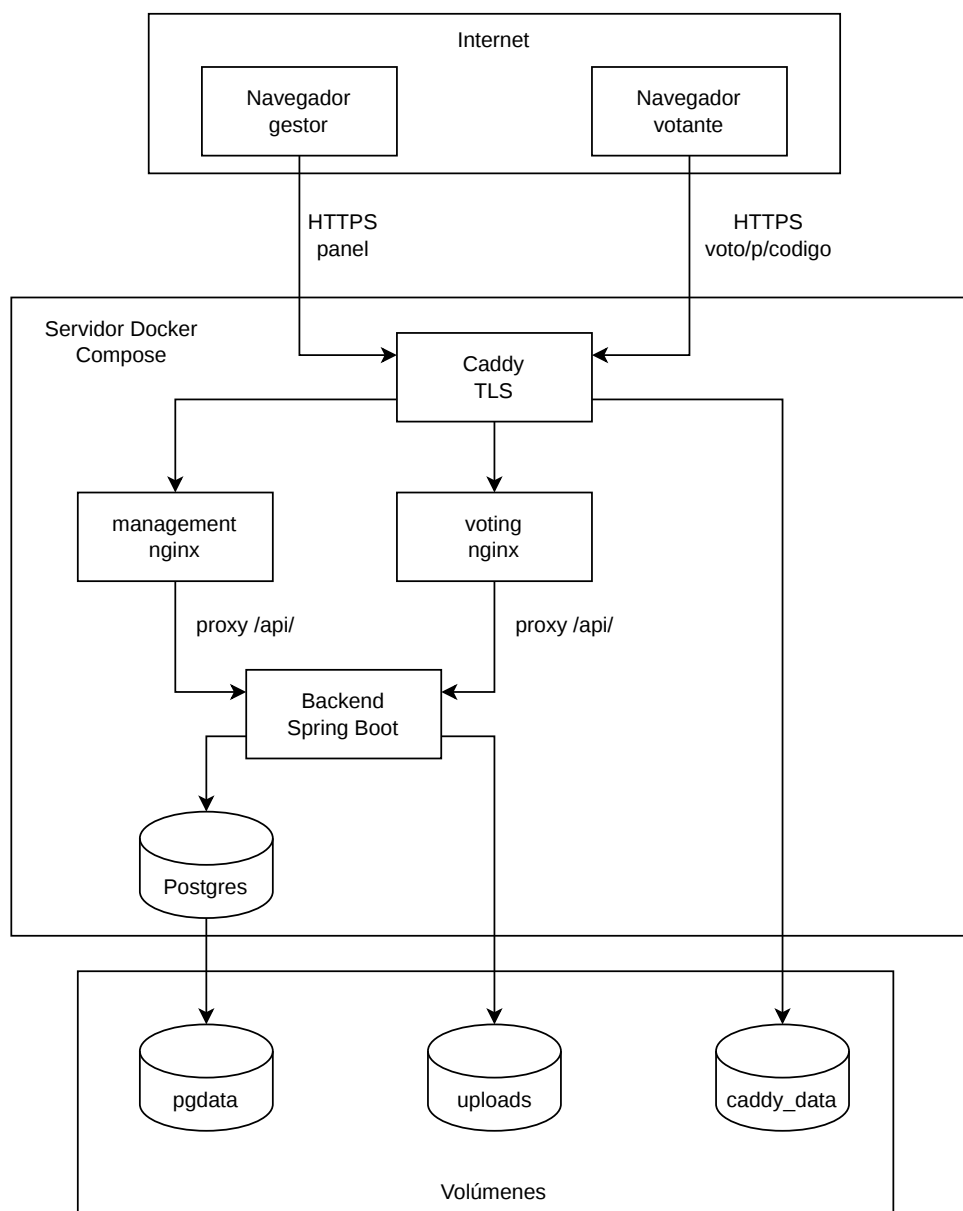


Figura 5.2: Diagrama de despliegue de la pila de producción.

por la red interna de *Compose*. La consecuencia es que una misma imagen de cada *frontend* sirve para cualquier entorno sin reconstruirla ni inyectarle configuración, la imagen que llega a producción es exactamente la que se probó en la integración continua y, como el navegador ve un único origen para la página y para la API, el mecanismo de CORS (*Cross-Origin Resource Sharing*)¹ no interviene.

La segunda es que *Caddy* es el único contenedor accesible desde internet; el resto solo publica puertos en la red interna, y el cortafuegos del servidor únicamente admite SSH, HTTP y HTTPS. *Caddy* obtiene y renueva de forma automática los certificados de *Let's Encrypt* y redirige todo el tráfico HTTP a HTTPS, con lo que se cumple el requisito RNF-

¹Para saber en detalle qué es CORS, véase <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/CORS>.

09 sin gestionar certificados a mano. Aquí HTTPS no es solo una medida de seguridad, sino una necesidad funcional: la API del navegador que genera el identificador anónimo del votante, descrita en el apartado 5.2.4, solo está disponible en contextos seguros.

La tercera es el uso de dos subdominios, uno para la interfaz de votación y otro para el panel. El primero aparece en los enlaces y en los códigos QR impresos, por lo que interesa que sea corto y estable; el segundo es de uso interno del gestor. El *backend* construye los enlaces públicos de los puntos a partir del primero, de forma que el panel, aunque se sirva desde otro dominio, genera enlaces que apuntan al lugar correcto.

5.1.3. Integración y despliegue continuo

El flujo de integración y despliegue continuo completa la arquitectura de despliegue y responde al quinto objetivo de ingeniería. Cada cambio integrado en la rama principal dispara un flujo de *GitHub Actions* con tres trabajos encadenados: se ejecutan todas las pruebas del *backend* y de los dos *frontends*; si pasan, se construyen las tres imágenes de contenedor y se publican en el registro de *GitHub* etiquetadas con el identificador del *commit*; por último, el flujo se conecta por SSH al servidor, descarga las imágenes nuevas y reinicia solo los contenedores cuya imagen ha cambiado. Si una etapa falla, las siguientes no se ejecutan y el servidor conserva la versión anterior. De este modo el servidor no compila nada, solo ejecuta imágenes que ya han superado las pruebas, y volver a una versión anterior se reduce a fijar la etiqueta de una imagen y reiniciar la pila. La configuración del flujo se describe en el capítulo 7 y las pruebas que ejecuta, en el capítulo 8.

5.1.4. Principios transversales

Varias decisiones atraviesan todos los componentes y conviene enunciarlas antes de descender al detalle. El *backend* no guarda estado de sesión: la autenticación se basa en *tokens* firmados que el servidor valida sin consultar ningún almacén, y toda la configuración se lee de variables de entorno, siguiendo la metodología de los doce factores para aplicaciones en contenedores [20]. Los datos son privados para cada gestor: toda entidad de gestión referencia a su propietario, toda consulta filtra por él y un recurso de otro gestor responde con «no encontrado» en lugar de «prohibido», para no revelar siquiera su existencia. El borrado es lógico: los ítems, las votaciones y los puntos se marcan como eliminados en lugar de destruirse, porque las estadísticas históricas los referencian y perderlos falsearía los resultados. Las invariantes críticas del dominio se garantizan en la propia base de datos y no solo en el código, ya que una comprobación en el servicio puede fallar ante dos peticiones simultáneas y una restricción del gestor de base de datos no. Y la infraestructura es reproducible y desechable: el servidor se prepara con un único guion idempotente y no contiene nada configurado a mano, por lo que puede destruirse en caso de querer buscar otra alternativa y volver a levantarse en otro lugar repitiendo los mismos pasos.

5.2 Diseño detallado

5.2.1. Organización interna del backend

El *backend* es una aplicación *Spring Boot* organizada por dominios funcionales y no por capas técnicas. En lugar de agrupar todos los controladores en un paquete, todos los servicios en otro y todas las entidades en un tercero, existe un paquete por concepto de negocio y dentro de cada uno conviven su entidad, su repositorio, su servicio, sus objetos

de transferencia y su controlador. Así, el código que cambia junto está junto, y quien lee el proyecto encuentra en cada carpeta un concepto del modelo de dominio y no una categoría técnica. La tabla 5.1 resume la responsabilidad de cada paquete.

Paquete	Responsabilidad
config	Configuración de seguridad, de la firma de <i>tokens</i> , de la documentación OpenAPI y de las propiedades de la aplicación.
common	Traducción de excepciones a respuestas de error, obtención del gestor autenticado y construcción de enlaces públicos.
auth, user	Registro e inicio de sesión de gestores y emisión de <i>tokens</i> .
item	Catálogo de ítems y almacenamiento de sus imágenes en disco.
voting	Votaciones, con su lista ordenada de ítems, su tipo y su configuración.
voting.strategy	Estrategias de los tres tipos de votación (véase el apartado 5.2.3).
votingpoint	Puntos de votación y generación de su código público.
instance	Rondas: lanzamiento y cierre.
vote	Papeletas y votos.
stats	Resultados por ronda, por votación y por ítem, calculados según el tipo de votación.
publicapi	Las dos operaciones que consume la interfaz de votación.

Tabla 5.1: Paquetes del *backend* y su responsabilidad.

Dentro de cada paquete se repite la misma estructura en tres niveles, en la línea de los patrones de capa de servicio y repositorio descritos por Fowler [8]. El controlador se limita a traducir HTTP: recibe la petición, valida la forma de los datos y delega. El servicio contiene toda la lógica de negocio y define los límites de las transacciones; es donde viven reglas como «no se puede borrar una votación mientras tiene una ronda activa». El repositorio abstrae el acceso a la base de datos mediante interfaces cuyas consultas se generan a partir del nombre de sus métodos, de modo que el filtrado por propietario y la exclusión de lo borrado quedan codificados en el propio nombre de cada consulta. Las entidades de persistencia nunca salen por la API: cada operación tiene sus propios objetos de transferencia de datos, inmutables y con las reglas de validación declaradas sobre sus campos, lo que desacopla el contrato REST del modelo de datos. Los errores siguen el formato estándar de detalles de problema para APIs HTTP [14], con un mensaje legible y, en las validaciones, un mapa de errores por campo que los *frontends* muestran junto a cada control. La figura 5.3 representa esta estructura.

El recorrido de una petición de gestión ilustra cómo encajan las piezas. La petición atraviesa primero la cadena de filtros de *Spring Security*, que extrae el *token* de la cabecera de autorización, verifica su firma y su caducidad y rechaza con un 401 (*Unauthorized*) la que no lo supere. El controlador recibe una petición ya autenticada, valida los datos de entrada y obtiene el identificador del gestor a partir del *token*. El servicio ejecuta la lógica dentro de una transacción, filtrando siempre por ese identificador y excluyendo lo borrado, y el repositorio traduce las operaciones a SQL. Si en cualquier punto se lanza una excepción de dominio, un manejador global la convierte en la respuesta de error correspondiente: 404 (*Not Found*) si el recurso no existe o pertenece a otro gestor, 409 (*Conflict*) si la operación viola una regla de negocio y 400 (*Bad Request*) si los datos son inválidos.

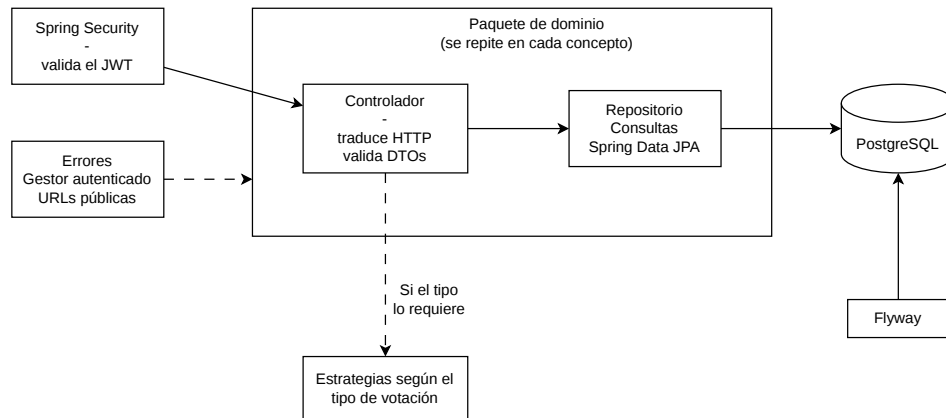


Figura 5.3: Estructura en capas que repite cada paquete de dominio del *backend*.

5.2.2. Modelo de datos

El esquema de la base de datos materializa el modelo de dominio de la figura 4.4 en ocho tablas, que se muestran en la figura 5.4. El gestor se corresponde con la tabla de usuarios; los ítems, las votaciones y los puntos de votación tienen su tabla propia, cada una con una clave ajena hacia su propietario, y la ronda de votación se convierte en la tabla de instancias. El votante, en cambio, no tiene tabla: al no existir cuentas, un votante es únicamente el identificador anónimo de su dispositivo, que queda registrado en cada papeleta.

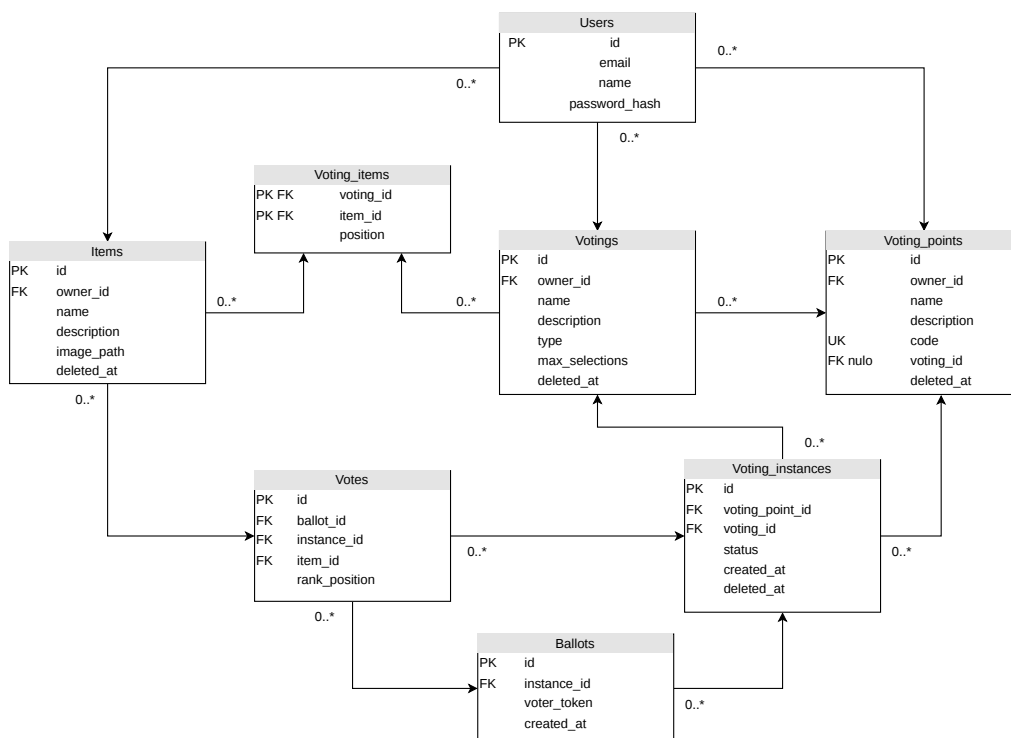


Figura 5.4: Modelo de datos de la aplicación.

Las decisiones que este esquema encierra y que no se deducen a simple vista son cinco. La primera es que la relación entre una votación y sus ítems se resuelve con una tabla de unión que incluye la posición de cada ítem, porque el orden en que el gestor los dis-

pone es el orden en que el votante los ve y debe conservarse. La segunda es que la ronda guarda una referencia a la votación que se lanzó, aunque podría deducirse del punto al que pertenece: si el gestor asigna más tarde otra votación al mismo punto, cada ronda histórica sigue sabiendo qué votación se celebró y los resultados antiguos no se reinterpretan a la luz de la configuración nueva. La tercera es la separación entre la papeleta, que registra que un dispositivo participó en una ronda, y los votos, que registran qué ítems eligió y, en el *ranking*, en qué posición. Con un único tipo de votación bastaría una tabla, pero dar cabida a papeletas con uno, varios o todos los ítems ordenados sin cambiar el esquema exige que la participación y su contenido sean filas distintas. La cuarta es que dos de las reglas más importantes del dominio están escritas como restricciones de la base de datos: un índice único parcial sobre las rondas, aplicado solo a las que están activas, garantiza que nunca existan dos rondas activas en el mismo punto, y una restricción de unicidad sobre el par ronda e identificador de dispositivo garantiza que cada dispositivo tenga como máximo una papeleta por ronda. Ambas reglas se comprueban también en los servicios para devolver un mensaje claro, pero si dos peticiones llegasen en el mismo instante, la base de datos rechazaría la segunda y el manejador de errores lo traduciría a un 409 (*Conflict*). La quinta es que todas las claves ajenas impiden el borrado en cascada: como el borrado de la aplicación es lógico, nunca se ejecuta un borrado físico desde el código y el histórico permanece íntegro por construcción.

El esquema no lo genera el mapeador objeto-relacional a partir de las entidades, sino que se define en guiones SQL versionados que *Flyway* aplica en orden en cada arranque; el mapeador se limita a comprobar que el esquema coincide con las entidades y se niega a arrancar si no es así. Esto permite expresar construcciones propias de *PostgreSQL*, como el índice parcial mencionado, y deja constancia en el repositorio de cada cambio del esquema junto con la migración de datos que requiera.

5.2.3. Tipos de votación

El requisito RF-24 pide que el votante pueda elegir uno o varios ítems respetando un límite configurable. Durante el desarrollo esa necesidad se concretó en los tres tipos de votación de la tabla 5.2. En el voto único el votante elige exactamente un ítem y el resultado es el recuento de papeletas por ítem. En los votos limitados elige entre uno y un máximo fijado por el gestor, y el resultado es el número de papeletas que incluyen cada ítem. En el *ranking* ordena los ítems que desee de mejor a peor y el resultado se calcula con el recuento de Borda [4]: en una votación con M ítems, la posición p otorga $M - p + 1$ puntos.

Tipo	Papeleta válida	Resultado por ítem
Voto único	Exactamente un ítem.	Número de papeletas que lo eligen y porcentaje sobre el total.
Votos limitados	Entre uno y N ítems distintos, con N fijado por el gestor.	Número de papeletas que lo incluyen y porcentaje sobre el total.
<i>Ranking</i>	Al menos un ítem, sin repetir, en orden de preferencia.	Puntos de Borda, posición media y número de primeros puestos.

Tabla 5.2: Tipos de votación y sus reglas.

Los tres tipos varían juntos en cuatro comportamientos y en nada más: cómo se valida la configuración de la votación, cómo se valida una papeleta, cómo se puntúan los resultados y qué instrucciones se muestran al votante. Repartir esos comportamientos en

condicionales por tipo dentro de cada servicio habría dispersado la misma decisión por cuatro lugares y habría hecho que añadir un tipo nuevo obligase a tocar todos ellos. Por eso se ha aplicado el patrón *Strategy* [9]: una interfaz declara las cuatro operaciones, cada tipo las implementa en una clase propia sin estado y un registro entrega la estrategia adecuada a partir del tipo almacenado en la votación. Los servicios programan contra la interfaz y desconocen las clases concretas; el registro recibe todas las implementaciones por inyección de dependencias y falla al arrancar si falta alguna, con lo que un error de configuración se detecta antes de atender la primera petición. Añadir un cuarto tipo consiste en escribir una clase nueva y un valor en la enumeración de tipos, sin modificar el código existente. Es el único patrón de diseño de objetos que se introduce de forma deliberada, porque es el único lugar donde existe una variabilidad real con varias implementaciones; el resto de patrones sobre los que se apoya la arquitectura se analizan en el capítulo 7.

5.2.4. Seguridad y autenticación

El diseño de seguridad distingue dos poblaciones con necesidades distintas: los gestores, que deben autenticarse (RF-01, RF-28, RNF-08), y los votantes, que no deben registrarse, pero tampoco poder votar más veces de las permitidas (RF-03, RF-27, RNF-10).

Los gestores se registran con correo electrónico, nombre y contraseña, que se almacena con el algoritmo *bcrypt* [18], diseñado para que el cálculo sea deliberadamente costoso y resista ataques por fuerza bruta. Al iniciar sesión, el *backend* emite un *JSON Web Token* [11] firmado con HMAC-SHA256 mediante una clave que solo el servidor conoce, con el identificador del gestor y una caducidad de veinticuatro horas; el panel lo guarda en el almacenamiento local del navegador y un interceptor lo añade a todas las peticiones y cierra la sesión si el servidor responde con un 401 (*Unauthorized*). Se ha elegido este mecanismo frente a la alternativa clásica de una sesión mantenida en el servidor y referenciada por una *cookie* porque deja al servidor sin estado, en coherencia con los principios enunciados, se resuelve con el soporte nativo de *Spring Security* sin librerías ni almacenes de sesión adicionales, y funciona sin fricción con dos *frontends* en dominios distintos, escenario en el que las *cookies* exigen una configuración cuidadosa. Esta decisión tiene un coste: el cierre de sesión es una acción local del cliente, y el *token* sigue siendo técnicamente válido hasta su caducidad. Para el perfil de riesgo de esta aplicación, en la que un gestor solo puede afectar a sus propios datos, se ha considerado aceptable mitigarlo con una caducidad corta en lugar de introducir una lista de revocación que devolvería el estado al servidor. Un proveedor de identidad externo se descartó para evitar depender de terceros.

Los votantes no tienen cuenta. La primera vez que la interfaz de votación se abre en un navegador, genera un identificador aleatorio mediante la API criptográfica del propio navegador, con una alternativa menos robusta para los contextos en los que esa API no está disponible, y lo guarda en el almacenamiento local. Ese identificador viaja en una cabecera al consultar el estado de un punto, para saber si el dispositivo ya ha participado, y en el cuerpo de la petición al enviar la papeleta, y la restricción de unicidad descrita en el modelo de datos impide que un mismo dispositivo registre dos papeletas en la misma ronda. Este mecanismo cumple lo que el requisito RNF-10 pide (evitar el voto múltiple sin exigir registro y resistir intentos básicos de vulneración): protege frente a la recarga de la página, el doble envío y la repetición ingenua, pero no frente a quien borra el almacenamiento del navegador o usa varios dispositivos. Se ha considerado que ese equilibrio es el correcto para un producto en el que la ausencia de fricción para el votante es un objetivo de primer orden y el incentivo para el fraude es bajo, y se ha preferido

dejarlo claro a prometer una protección que ningún sistema sin identificación de este nivel puede ofrecer.

En cuanto a la autorización, no existen roles: todos los gestores tienen los mismos permisos sobre sus propios datos y ninguno sobre los ajenos. La separación se consigue filtrando toda consulta por el identificador del propietario, que se toma del *token* y nunca de la petición, de modo que no es posible acceder a un recurso ajeno manipulando un identificador en la URL.

5.2.5. Ciclo de vida de una ronda y resolución del código QR

El requisito RF-13 y la propuesta del capítulo 2 establecen que el código QR de cada punto de votación es fijo y que es el servidor quien decide en cada momento qué se muestra tras él. El código QR no codifica una votación sino la dirección pública del punto, formada por el subdominio de la interfaz de votación y un código de ocho caracteres, único e inmutable, que se genera al crear el punto evitando los caracteres que se confunden al leerlos. Cuando un votante abre esa dirección, la interfaz de votación pregunta a la API por el estado del punto, y la API resuelve en ese instante si existe una ronda activa. Si la hay, devuelve la votación con sus ítems y las instrucciones del tipo; si no, devuelve un estado inactivo y la interfaz muestra un mensaje informativo (RF-26). Como la resolución ocurre en el servidor en cada petición, el gestor puede cambiar la votación asignada a un punto, lanzarla y detenerla tantas veces como quiera sin que el QR impreso pierda vigencia. La figura 5.5 recoge los estados por los que pasa un punto.

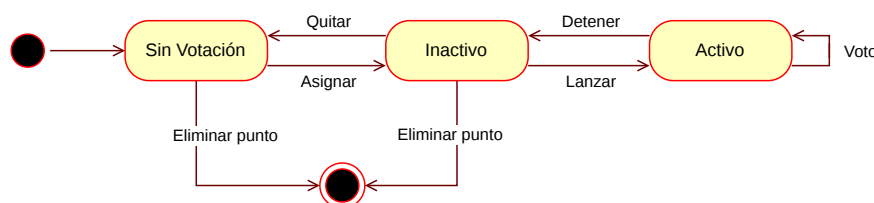


Figura 5.5: Estados de un punto de votación y transiciones entre ellos.

Un punto recién creado no tiene votación asignada y su enlace muestra el estado inactivo. Al asignarle una votación (RF-20) queda listo para lanzarse. El lanzamiento (RF-21) crea una ronda activa, tras comprobar que el punto pertenece al gestor, que tiene una votación asignada con al menos un ítem y una configuración válida según su estrategia, y que no existe ya una ronda activa en ese punto. Mientras la ronda está activa, el sistema bloquea con un 409 (*Conflict*) las operaciones que alterarían el significado de los votos que se están recogiendo: borrar la votación o el punto, cambiar la lista de ítems, el tipo o el máximo de selecciones, borrar ítems que participan en ella o asignar otra votación al punto; el nombre y la descripción sí pueden editarse, porque no afectan al resultado. Al detener la ronda (RF-22), esta pasa al estado cerrado, se registra el instante de fin y el punto vuelve a estar inactivo; la ronda cerrada solo existe para las estadísticas. Por último, el borrado lógico de un punto hace que su código responda con un 404 (*Not Found*) sin destruir el histórico de sus rondas.

La figura 5.6 resume la interacción entre el gestor, la API y el votante a lo largo de una ronda. Al consultar el punto, la API comprueba si el dispositivo ya tiene papeleta en la ronda activa y lo indica en la respuesta, de modo que la interfaz muestra directamente el agradecimiento a quien ya ha votado. Al recibir una papeleta, el servicio valida que el punto existe y tiene ronda activa, delega en la estrategia del tipo la validación del contenido y registra la papeleta con un voto por ítem en una única transacción. Si la restricción de unicidad rechaza la inserción, el votante recibe un 409 (*Conflict*) y la interfaz, en lugar

de suponer qué ha ocurrido, vuelve a consultar el estado real del punto. Este detalle es una decisión consciente: entre la carga de la papeleta y su envío el gestor puede haber detenido la ronda, y la interfaz trata siempre la respuesta del servidor como la única fuente de verdad.

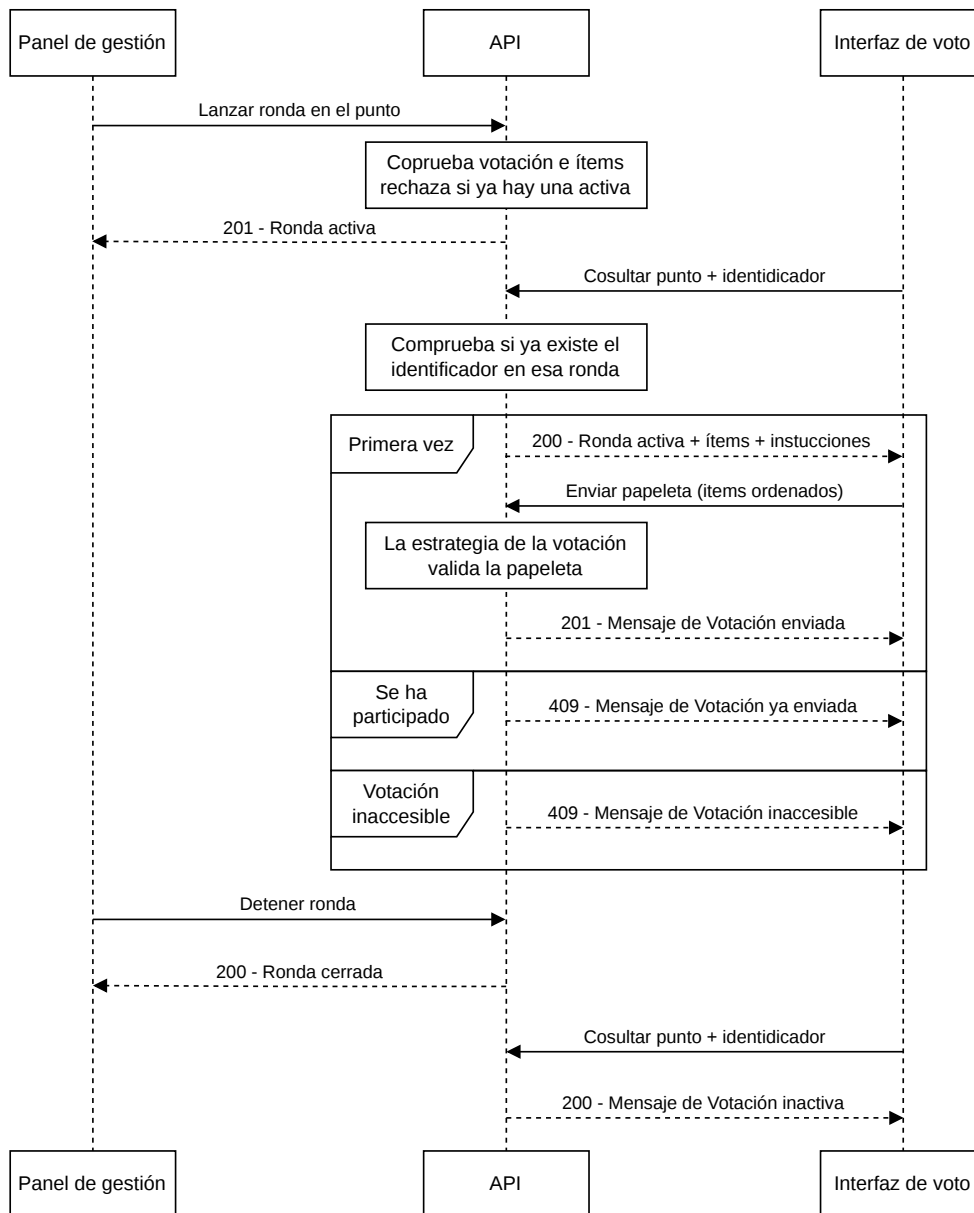


Figura 5.6: Secuencia de lanzamiento, votación y cierre de una ronda.

5.2.6. Diseño de los frontends

Las dos aplicaciones cliente comparten tecnología y convenciones, pero su diseño responde a propósitos distintos. El panel de gestión se organiza en tres áreas. El área central agrupa los servicios sin interfaz: un cliente tipado de la API que expone cada operación como un método y oculta rutas, cabeceras y formato de las peticiones; el servicio de autenticación, que mantiene el *token* y el usuario en señales sincronizadas con el alma-

cenamiento local; el interceptor que añade la autorización a cada petición; dos guardas de ruta que exigen sesión para el panel y su ausencia para las pantallas de acceso; y los modelos, que reflejan uno a uno los objetos de transferencia del *backend*, de modo que un cambio en el contrato de la API se detecta en tiempo de compilación. El área compartida contiene componentes de presentación sin lógica de negocio, que reciben sus datos por parámetros y no conocen la API; es el caso del componente de clasificación con barras, que recibe los candidatos ya puntuados y sirve tanto a los resultados de una ronda como a los agregados de una votación. El área de funcionalidades contiene una carpeta por sección del panel, con sus páginas de lista y de formulario, y es la única que carga datos y mantiene estado; sus rutas se cargan de forma diferida, de modo que el navegador descarga cada sección la primera vez que se visita.

La interfaz de votación es deliberadamente mínima: una sola pantalla útil, sin menú ni sesión, pensada para un teléfono y para una interacción de pocos segundos. Su diseño gira en torno a una máquina de estados explícita: la página mantiene en una señal el estado de la vista, que puede ser de carga, punto no encontrado, ronda inactiva, papeleta disponible, voto completado o error, y decide cuál mostrar exclusivamente a partir de la respuesta de la API. Esta representación se ha preferido a una combinación de indicadores booleanos porque cada pantalla posible tiene nombre y así se evitan las combinaciones inconsistentes. Dentro del estado de papeleta, el comportamiento depende del tipo de votación que la API indica: en el voto único, tocar un ítem sustituye la selección; en los votos limitados, la alterna y, al llegar al máximo, el resto de ítems deja de responder, de modo que para elegir otro hay que anular antes uno de los ya seleccionados; en el *ranking*, cada toque añade el ítem al final del orden con su número de posición y un segundo toque lo retira. En todos los casos la selección se guarda como una lista ordenada de identificadores, que es lo que la API espera, y el botón de confirmación se activa cuando la papeleta es válida según las reglas del tipo. La validación se repite en el servidor, que es la única fuente de verdad; la del cliente existe solo para dar una respuesta inmediata al votante.

5.3 Conclusiones del diseño

El trabajo de diseño recogido en este capítulo ha aportado al proyecto una base estable sobre la que la implementación pudo avanzar sin rehacer decisiones importantes. Partir de los requisitos y del modelo de dominio permitió que las piezas centrales, el código QR fijo resuelto en el servidor, la separación en dos interfaces y la ronda de votación como entidad intermedia entre el punto y los votos, incorporada al modelo en el primer *sprint* de desarrollo, encajaran desde el principio y no tuvieran que replantearse después. Del mismo modo, haber fijado desde el inicio que las invariantes críticas viven en la base de datos y que el borrado es lógico evitó dos categorías enteras de errores que habrían sido costosos de corregir con datos reales.

No todo se anticipó. El modelo de dominio inicial contemplaba un único tipo de votación y un único voto por participación; la aparición de tres tipos con reglas distintas obligó a separar la papeleta de sus votos y a introducir el patrón *Strategy*, que no estaba previsto. Este cambio se considera una confirmación del enfoque ágil seguido más que un fallo del diseño: la abstracción se introdujo cuando apareció una variabilidad real, y no antes, y el resto de la arquitectura la absorbió sin mayores modificaciones. Por otro lado, la planificación temporal de las rondas, que los requisitos iniciales de gestión de votaciones apuntaban al hablar de duración, se sustituyó por un lanzamiento y un cierre manuales desde el panel. Finalmente, las limitaciones que el diseño asume, en particular la validez del *token* del gestor hasta su caducidad y el alcance real de la protección contra

el voto múltiple, son decisiones tomadas de forma consciente junto con el *Product Owner* en las reuniones periódicas, tras valorar los pros y los contras de las distintas opciones.

CAPÍTULO 6

Tecnologías utilizadas

Una vez descrito el diseño de la solución, este capítulo recoge las tecnologías con las que se ha construido. El proyecto tiene como objetivo recorrer casi todo el ciclo de vida de la ingeniería del software, desde la especificación de requisitos hasta el despliegue en producción con integración y despliegue continuos. Esa amplitud hace que el número de lenguajes, *frameworks*, librerías, servicios y herramientas implicados sea elevado, y explicar cada uno en profundidad ocuparía un espacio desproporcionado. Por esto, este capítulo se ha estructurado de la siguiente manera: de cada tecnología se indica brevemente qué es, qué papel desempeña en el proyecto y, cuando existen alternativas relevantes, por qué se ha escogido.

La pila base, *Angular* para los *frontends*, Java con *Spring Boot* para el *backend*, *PostgreSQL* como base de datos y *Docker* para la contenerización, fue elegida al inicio del proyecto y no fue objeto de comparación. Al ser las tecnologías que se dominaban antes de empezar, y dado que el proyecto iba a exigir aprender un número considerable de herramientas nuevas en despliegue, infraestructura y pruebas, se prefirió no añadir a ese esfuerzo el aprendizaje de un lenguaje o un *framework* distintos. El resto de decisiones se han tomado dentro de ese marco con dos criterios constantes, coherentes con los principios enunciados en el capítulo 5: no incorporar ninguna dependencia sin una necesidad concreta que la justifique, y que el sistema funcione sin servicios externos en tiempo de ejecución, puesto que la interfaz de votación debe operar en redes cuya conectividad no se controla. Las tecnologías se han agrupado a continuación según la parte del sistema en la que actúan, siguiendo la misma división que la arquitectura del capítulo 5.

6.1 Backend

6.1.1. Java

Java¹ es un lenguaje de propósito general, orientado a objetos y con tipado estático, que se compila a un código intermedio ejecutado por la máquina virtual de Java (JVM). En Krates es el lenguaje de todo el *backend*, y se han aprovechado características recientes del lenguaje, en particular los *records* para los objetos de transferencia de datos, inmutables por construcción, en línea con la decisión del capítulo 5 de que las entidades de persistencia nunca salgan por la API. Alternativas como Kotlin, sobre la misma máquina virtual, o plataformas ajenas a ella como *.NET*, *Go* o *Node.js* se descartaron por el motivo expuesto en la introducción.

¹Página oficial de Java: <https://www.java.com/es/>.

6.1.2. Spring Boot

*Spring Boot*² es el *framework* que empaqueta el ecosistema *Spring* (inyección de dependencias, web, seguridad y acceso a datos) con configuración automática, un servidor embebido y un único ejecutable. Es el esqueleto de la API: aporta el contenedor de inyección de dependencias sobre el que se apoyan los servicios y el registro de estrategias de votación del apartado 5.2.3, la configuración mediante variables de entorno que pide la metodología de los doce factores y el arranque como un jar autocontenido dentro de su contenedor.

De sus módulos, dos merecen mención. *Spring Web MVC* implementa los controladores REST sobre el modelo clásico de *servlets*, con conversión automática a JSON y traducción de los errores al formato de detalles de problema; se ha preferido a su variante reactiva, *Spring WebFlux*, porque la carga es moderada y las operaciones son cortas y transaccionales, un escenario en el que el modelo bloqueante resulta más sencillo de escribir, depurar y probar. *Jakarta Bean Validation* valida de forma declarativa los DTOs de entrada mediante anotaciones sobre sus campos; las validaciones que dependen del estado, como que el máximo de selecciones no supere el número de ítems, se dejan a las estrategias, donde existe contexto para decidir.

6.1.3. Spring Data JPA e Hibernate

JPA es la especificación estándar de mapeo objeto-relacional en Java e *Hibernate*³ su implementación más extendida; *Spring Data JPA* añade repositorios cuyas consultas se generan a partir del nombre de sus métodos. En Krate se encargan del mapeo de las entidades, de los repositorios con consultas de dominio, que codifican en su nombre el filtrado por propietario y la exclusión de lo borrado, y de las transacciones declarativas de los servicios. El modelo tiene relaciones que se recorren en ambos sentidos, como la votación y su lista ordenada de ítems o la papeleta y sus votos, y JPA las resuelve con poco código, mientras que alternativas más simples como *Spring Data JDBC* o *jOOQ* habrían obligado a escribir a mano esa navegación. Cabe destacar que se han evitado a propósito las funciones automáticas de *Hibernate* para el borrado lógico, para controlar qué consultas filtran lo eliminado y cuáles, como las de estadísticas, no deben hacerlo.

6.1.4. Spring Security

*Spring Security*⁴ es el *framework* de seguridad de *Spring*: una cadena de filtros que atraviesa toda petición, utilidades de autenticación y autorización y funciones criptográficas. Su módulo *OAuth2 Resource Server* valida *tokens* JWT en cada petición y proporciona además el codificador y el decodificador de *tokens*, de modo que el mecanismo descrito en el apartado 5.2.4, con firma HMAC-SHA256, contraseñas con *bcrypt*, rutas públicas para la interfaz de votación y configuración de CORS, se resuelve sin ninguna librería adicional. Por eso no se ha recurrido a librerías de JWT independientes como *jjwt*, y los proveedores de identidad externos, como *Keycloak*, se descartaron por el criterio de no depender de terceros.

²Página oficial de Spring Boot: <https://spring.io/projects/spring-boot>.

³Página oficial de Hibernate: <https://hibernate.org/>.

⁴Página oficial de Spring Security: <https://spring.io/projects/spring-security>.

6.1.5. Flyway

*Flyway*⁵ es una herramienta de migraciones de base de datos versionadas: guiones SQL numerados que se aplican en orden en cada arranque y quedan registrados en una tabla de control. Como se explicó en el capítulo 5, el esquema de Krate no lo genera el mapeador, sino dos migraciones, la inicial y la que introdujo los tipos de votación separando las papeletas de sus votos y migrando los datos existentes. Frente a *Liquibase*, que describe los cambios en XML o YAML y ofrece más funciones de reversión, se ha preferido *Flyway* porque los guiones SQL planos son fáciles de leer y revisar y permiten expresar exactamente lo que *PostgreSQL* necesita, como el índice único parcial que garantiza una sola ronda activa por punto.

6.1.6. springdoc-openapi

*springdoc-openapi*⁶ genera la especificación OpenAPI de la API a partir de los controladores de *Spring* y sirve la interfaz *Swagger UI*, desde la que se puede introducir el *token* de un gestor y probar cada operación. Su valor está en que la documentación no requiere mantenimiento: se deriva del código y siempre está actualizada, mientras que una especificación escrita a mano habría envejecido con cada cambio del contrato.

6.1.7. Maven

*Maven*⁷ es la herramienta de construcción y gestión de dependencias del *backend*: compila, ejecuta las pruebas, empaqueta el jar y actúa como primera etapa de la imagen *Docker*. Se incluye su *wrapper* en el repositorio para no exigir una instalación local ni en la integración continua. *Gradle* es más flexible y rápido, pero la configuración de este proyecto es puramente declarativa, para lo que *Maven*, que es además lo que genera *Spring Initializr* por defecto, resulta suficiente.

6.2 Frontend

6.2.1. TypeScript

TypeScript⁸ es un superconjunto de JavaScript con tipado estático que se compila a JavaScript. Es el lenguaje de los dos *frontends* y una exigencia de *Angular*, que está escrito en él. Más allá de la obligación, el tipado ha demostrado su utilidad: los modelos de cada aplicación reflejan los DTOs del *backend*, de forma que cuando el contrato de la API cambió al introducir los tipos de votación, el compilador señaló cada punto que había que adaptar.

6.2.2. Angular

*Angular*⁹ es el *framework* de aplicaciones de una sola página mantenido por Google, con componentes, inyección de dependencias, enrutador, formularios y cliente HTTP integrados. Con él están contruidos el panel de gestión y la interfaz de votación, cuyo

⁵Página oficial de Flyway: <https://www.red-gate.com/products/flyway/>.

⁶Página oficial de springdoc-openapi: <https://springdoc.org/>.

⁷Página oficial de Maven: <https://maven.apache.org/>.

⁸Página oficial de TypeScript: <https://www.typescriptlang.org/>.

⁹Página oficial de Angular: <https://angular.dev/>.

diseño se describe en el capítulo 5. La librería *RxJS*, interfaz de su cliente HTTP, se usa de forma mínima, convirtiendo los observables en promesas. Se ha elegido frente a otras opciones populares como *React* o *Vue* por la familiaridad con el *framework*, tal y como se expresa en la introducción.

6.2.3. CSS propio y recursos empaquetados

Las dos aplicaciones se estilizan con hojas de estilo estándar, sin preprocesador ni *framework* de utilidades. Un único fichero por aplicación define, mediante *custom properties*, los *tokens* del sistema de diseño (colores, radios, sombras y tipografía) y las clases base, y cada componente añade solo sus estilos locales. Se descartaron librerías de componentes como *Angular Material* y utilidades como *Tailwind* porque se quería una interfaz ligera y sencilla, algo que beneficia especialmente a la interfaz de votación. Dos recursos completan la presentación, y ambos se empaquetan en el *build* en lugar de cargarse de un CDN para cumplir el criterio de funcionar sin internet: la tipografía *Nunito*, incluida mediante el paquete de *Fontsource*, y los iconos de *Lucide* a través de *lucide-angular*, un conjunto de SVG del que solo se importan los iconos utilizados.

6.2.4. qrcode

qrcode es una librería JavaScript que genera códigos QR en un *canvas* del navegador. En el panel de gestión produce el código del enlace público de cada punto de votación y permite descargarlo como PNG para imprimirlo. Genera el código localmente, sin enviar la URL a ningún servicio externo. La alternativa de generarlo en el servidor con una librería Java como *ZXing*, contemplada inicialmente, se descartó por innecesaria: el QR no contiene más que una dirección que el panel ya conoce, y generarlo en el cliente evita un *endpoint* y una dependencia en el *backend*.

6.3 Base de datos

6.3.1. PostgreSQL

*PostgreSQL*¹⁰ es un sistema gestor de bases de datos relacional de código abierto y es la única fuente de datos de Krate. Además de formar parte de la pila base, el dominio es claramente relacional, con integridad referencial entre ocho tablas, y *PostgreSQL* ofrece las construcciones que el diseño del capítulo 5 exige para que las reglas críticas no dependan solo del código, en particular los índices únicos parciales con los que se garantiza que no existan dos rondas activas en un mismo punto.

6.4 Despliegue e infraestructura

6.4.1. Docker y Docker Compose

*Docker*¹¹ empaqueta cada aplicación con todo lo necesario para ejecutarla de forma reproducible, y *Docker Compose* define y arranca conjuntos de contenedores a partir de un fichero YAML. Krate construye tres imágenes propias en dos etapas: en la primera se compila y en la segunda solo queda el resultado sobre una imagen mínima, de modo que

¹⁰Página oficial de PostgreSQL: <https://www.postgresql.org/>.

¹¹Página oficial de Docker: <https://www.docker.com/>.

las imágenes finales son pequeñas y no contienen herramientas de compilación. *Compose* orquesta esas imágenes junto con la oficial de *PostgreSQL*, la red interna, los volúmenes y las variables de entorno; en producción se combina con un fichero de sobrescritura que sustituye la construcción local por las imágenes publicadas, retira los puertos expuestos y añade *Caddy*, tal como muestra la figura 5.2. Con un único servidor y cinco servicios, *Compose* cubre la necesidad con un fichero legible. Se eligió sobre *Kubernetes* por familiaridad con la tecnología.

6.4.2. nginx

*nginx*¹² es un servidor web y *proxy* inverso ligero. Un contenedor por cada *frontend* sirve sus ficheros estáticos, dirige cualquier ruta al documento principal de la SPA para que el enrutador de *Angular* la resuelva y reenvía las peticiones bajo */api/* al *backend*, con las consecuencias que se explicaron en el apartado 5.1.2. Su configuración ocupa pocas líneas y es el estándar *de facto* para este uso; servir los *frontends* desde el propio *Spring Boot* habría acoplado los tres despliegues en uno.

6.4.3. Caddy

*Caddy*¹³ es un servidor web y *proxy* inverso que obtiene y renueva certificados TLS de forma automática mediante el protocolo ACME de *Let's Encrypt*. En producción es el único contenedor expuesto a internet: con dos bloques de configuración de una línea, uno por subdominio, emite un certificado para cada uno, dirige HTTP a HTTPS y reenvía el tráfico a los *nginx*. HTTPS es imprescindible por los motivos funcionales expuestos en el apartado 5.1.2. Las alternativas eran *nginx* con *certbot*, que exige renovar y recargar los certificados con tareas programadas, *Traefik*, con más configuración, y *Cloudflare Tunnel*, que depende de un tercero. *Caddy* resuelve el problema sin gestión manual y, al ser un contenedor más de la pila, sin instalar nada en el servidor.

6.4.4. GitHub, GitHub Actions y GitHub Container Registry

El código de los tres componentes se gestiona con *Git* en un único repositorio alojado en *GitHub*¹⁴, que aporta además los dos servicios sobre los que se apoya el despliegue. *GitHub Actions* es su servicio de integración y despliegue continuos: ejecuta flujos definidos en YAML sobre máquinas virtuales efímeras cuando ocurre un evento del repositorio, y en *Krate* cada integración en la rama principal dispara el flujo de tres trabajos descrito en el capítulo 5. *GitHub Container Registry* (GHCR) es el registro de imágenes de contenedor asociado al repositorio, donde se publican las tres imágenes etiquetadas con el identificador del *commit*, lo que permite fijar o retroceder a una versión concreta cambiando una etiqueta. Se eligieron por estar integrados con el repositorio, sin cuentas adicionales, y por ser gratuitos para un proyecto de este tamaño; los ejecutores incluyen *Docker*, con lo que las pruebas con *Testcontainers* se ejecutan igual que en la máquina de desarrollo. Construir las imágenes en el propio servidor habría obligado a instalar *Maven* y *Node.js* en él y a dimensionarlo para compilar; con el registro, el servidor solo ejecuta.

¹²Página oficial de *nginx*: <https://nginx.org/>.

¹³Página oficial de *Caddy*: <https://caddyserver.com/>.

¹⁴Página oficial de *GitHub*: <https://github.com/>.

6.4.5. Hetzner Cloud y Ubuntu Server

El servidor de producción es un servidor virtual privado (VPS) de *Hetzner Cloud*¹⁵, un proveedor europeo con facturación por horas y centros de datos en la Unión Europea. Ejecuta *Ubuntu Server*, distribución *Linux* de soporte extendido para la que *Docker* publica paquetes oficiales. Toda la preparación del servidor la realiza un único guion idempotente que instala *Docker*, restringe el acceso mediante el cortafuegos y un usuario sin privilegios, clona el repositorio y programa una copia de seguridad nocturna de la base de datos y de las imágenes subidas. Frente a plataformas gestionadas como *Railway* o *Render*, que no dan control sobre el anfitrión, un VPS convencional permite ejecutar la misma pila que en local con un coste proporcional al tiempo que permanece encendido.

6.4.6. DuckDNS

*DuckDNS*¹⁶ es un servicio gratuito de DNS dinámico que ofrece subdominios bajo su dominio apuntando a la dirección IP que se indique. Krate utiliza dos, uno para la interfaz de votación, que es el que aparece en los enlaces y en los códigos QR, y otro para el panel, y ambos son los nombres para los que *Caddy* solicita certificados. Registrar un dominio propio no se consideró necesario, y los servicios que derivan el nombre de la propia IP producen direcciones poco legibles en un enlace o QR.

6.5 Pruebas

6.5.1. JUnit 5, AssertJ y MockMvc

*JUnit 5*¹⁷ es el *framework* de pruebas estándar en Java, *AssertJ* aporta aserciones fluidas y legibles, y *MockMvc*, parte de *Spring Test*, permite ejecutar peticiones HTTP contra los controladores dentro del mismo proceso, sin levantar un servidor. Los tres vienen con *Spring Boot*. Con ellos se han escrito las pruebas unitarias de las estrategias de votación, que no necesitan el contenedor de *Spring*, y las pruebas de integración de la API completa, detalladas en el capítulo 8.

6.5.2. Testcontainers

*Testcontainers*¹⁸ es una librería que arranca servicios reales en contenedores *Docker* efímeros durante las pruebas. Todas las pruebas de integración del *backend* se ejecutan contra un *PostgreSQL* real, de modo que validan también las migraciones de *Flyway* y las restricciones de la base de datos. La alternativa habitual, una base de datos *H2* en memoria, no soporta los índices únicos parciales ni se comporta como *PostgreSQL*, por lo que habría dejado sin probar precisamente las invariantes que el diseño confía a la base de datos.

¹⁵Página oficial de Hetzner Cloud: <https://www.hetzner.com/cloud/>.

¹⁶Página oficial de DuckDNS: <https://www.duckdns.org/>.

¹⁷Página oficial de JUnit: <https://junit.org/>.

¹⁸Página oficial de Testcontainers: <https://testcontainers.com/>.

6.5.3. JaCoCo

*JaCoCo*¹⁹ es una herramienta de cobertura de código para Java que registra qué partes del programa se ejecutan mientras corren las pruebas y genera después un informe navegable por paquete, clase y línea. Su valor en Krate es poder medir hasta dónde llegan las pruebas unitarias y de integración definidas para el *backend* y localizar las partes que ninguna de ellas ejercita, algo que la simple cuenta de pruebas superadas no revela. Se ha elegido por ser el estándar *de facto* en los proyectos Java contruidos con *Maven* y por integrarse en el ciclo de construcción sin configuración adicional; la cobertura que ofrece el propio IDE se descartó porque no deja un informe reproducible fuera de él. Solo se mide en el *backend*: los *frontends* no tienen cobertura. La configuración, las cifras obtenidas y su lectura se recogen en el capítulo 8.

6.5.4. Vitest

*Vitest*²⁰ es el ejecutor de pruebas para JavaScript y TypeScript que *Angular* configura por defecto. Ejecuta las pruebas unitarias de los *frontends* sobre un navegador simulado y arranca en menos de un segundo. No se ha añadido ninguna herramienta de pruebas de extremo a extremo como *Playwright*, que figuraba en el planteamiento inicial; la comprobación visual de las pantallas se hizo de forma manual sobre la pila de contenedores, como se explica en el capítulo 8.

6.5.5. k6

*k6*²¹ es una herramienta de pruebas de carga de código abierto. Una prueba de carga somete a un sistema a un número elevado de peticiones simultáneas para medir cómo responde: *k6* lanza un número configurable de usuarios virtuales que ejecutan en paralelo un guion escrito en JavaScript, donde se describe la secuencia de peticiones HTTP que realiza cada uno, y recoge los tiempos de respuesta, la tasa de errores y las peticiones atendidas por segundo. Su principal valor es que en el propio guion se declaran umbrales de aceptación, como un tiempo de respuesta máximo o una tasa de errores tolerable, y la ejecución termina con error si no se cumplen, de modo que la prueba de carga deja de ser una simple medición y pasa a ser una comprobación con resultado verificable. En Krate se emplea sobre la API pública que consume la interfaz de votación, la consulta del estado de un punto y el envío de la papeleta, que son las únicas operaciones que pueden recibir muchas peticiones simultáneas, como ocurre cuando un grupo de votantes escanea el mismo código QR a la vez. Se ha preferido a *JMeter* y *Gatling*, más pesados y orientados a una interfaz gráfica o a otro lenguaje, y a utilidades como *ab* o *wrk*, que miden peticiones aisladas, pero no permiten reproducir el flujo de consultar y votar con un identificador distinto por usuario; además, los guiones se escriben en el mismo lenguaje que los *frontends* y la herramienta se instala como un único ejecutable. El diseño de las pruebas, su forma de ejecución y los resultados obtenidos se recogen en el capítulo 8.

¹⁹Página oficial de JaCoCo: <https://www.jacoco.org/>.

²⁰Página oficial de Vitest: <https://vitest.dev/>.

²¹Página oficial de k6: <https://k6.io/>.

6.6 Entorno y herramientas de desarrollo

6.6.1. Sistemas operativos y entornos de desarrollo

El desarrollo se ha llevado a cabo en un ordenador con *CachyOS*²², una distribución *Linux* basada en *Arch Linux*. Trabajar en *Linux* tiene la ventaja de que el entorno local coincide con el del servidor y con el de las imágenes de contenedor, de modo que *Docker* y los guiones de despliegue se comportan igual en todos ellos. Como equipo de apoyo se ha utilizado un portátil con *Windows* para comprobar que las aplicaciones y la puesta en marcha del repositorio funcionan también en ese sistema.

Se han utilizado tres editores según la tarea: *IntelliJ IDEA*²³ para el *backend* en Java y para los *frontends* en *Angular*, por su integración con *Maven*, *Spring* y *TypeScript* en un mismo entorno; *Visual Studio Code*²⁴ para el resto de ficheros del proyecto, como la configuración de *Docker*, los flujos de *GitHub Actions* y los guiones de despliegue; y *TeXstudio*²⁵ para la redacción de esta memoria en $\text{\LaTeX}$. Los *frontends* requieren además *Node.js* 22, que ejecuta la interfaz de línea de órdenes de *Angular*, y *npm* como gestor de paquetes.

6.6.2. Herramientas de apoyo

La gestión del proyecto y el registro del conocimiento se describieron en el capítulo 3. Como tablero *Kanban* se ha utilizado *Trello*, una herramienta más sencilla que las *suites* de gestión de proyectos y suficiente para un equipo de una persona, y como aplicación de notas, *Obsidian*. Por último, todos los diagramas que ilustran esta memoria se han elaborado con *draw.io*²⁶, una herramienta gratuita de diagramación que exporta a PDF vectorial para su inclusión en el documento.

6.6.3. Asistentes basados en modelos de lenguaje

A lo largo del proyecto se han utilizado asistentes de inteligencia artificial basados en modelos de lenguaje de gran tamaño (LLM) como herramienta de apoyo en tres ámbitos concretos. El primero es la investigación: han servido para explorar y comparar tecnologías, arquitecturas y alternativas antes de tomar las decisiones que se documentan en este capítulo y en el capítulo 5, contrastando después la información con las fuentes oficiales. El segundo es la escritura de las partes más repetitivas del código, especialmente la capa visual de los *frontends*. El tercero es la depuración: ante un error o un comportamiento inesperado, han ayudado a interpretar trazas y mensajes y a acotar la causa con mayor rapidez. En todos los casos la responsabilidad sobre las decisiones y sobre el código entregado ha recaído en el autor, que ha revisado y adaptado cada propuesta antes de incorporarla al proyecto.

6.7 Conclusiones de la selección tecnológica

La selección tecnológica ha cumplido su objetivo principal: permitir que el esfuerzo del proyecto se concentrara en las fases de ingeniería y no en el aprendizaje de la pila. Partir de lenguajes y *frameworks* ya conocidos dejó margen para incorporar con garantías

²²Página oficial de CachyOS: <https://cachyos.org/>.

²³Página oficial de IntelliJ IDEA: <https://www.jetbrains.com/idea/>.

²⁴Página oficial de Visual Studio Code: <https://code.visualstudio.com/>.

²⁵Página oficial de TeXstudio: <https://www.texstudio.org/>.

²⁶Página oficial de draw.io: <https://www.drawio.com/>.

las tecnologías que sí eran nuevas, en especial las de despliegue, infraestructura y pruebas de integración. Los dos criterios de selección se han mantenido hasta el final: la lista de dependencias de los tres componentes es corta, cada una responde a una necesidad identificable y el sistema no realiza ninguna llamada a terceros en tiempo de ejecución.

Como se ha mencionado en la introducción del capítulo, el número de tecnologías es bastante elevado. Esto era de esperar en un proyecto que abarca y busca mostrar tantas fases del proceso de construcción de software de calidad. En cualquier caso, no se ha hecho un uso muy exhaustivo de la mayoría de tecnologías, pero aun así el gran número de estas ha resultado uno de los mayores retos de todo el proyecto. Afortunadamente, los asistentes de inteligencia artificial han aligerado la carga de aprendizaje de estas, permitiendo recopilar documentación, hacer muestras de uso y dar indicaciones generales a la hora de utilizar las herramientas, *frameworks* y librerías que previamente resultaban desconocidas.

Respecto a lo previsto al inicio, varias piezas cambiaron. La gestión del proyecto se hizo con *Trello* en lugar de *Jira*, porque la funcionalidad de este último excedía lo que un equipo de una persona necesitaba. La generación de códigos QR pasó del servidor, donde se había pensado usar *ZXing*, al navegador del panel, que es donde la información necesaria ya está disponible. *Playwright*, previsto para pruebas de extremo a extremo, no llegó a incorporarse, decisión que se valora en el capítulo 8. Ninguno de estos cambios afectó a la arquitectura, lo que confirma que las decisiones del capítulo 5 no dependían de herramientas concretas.

CAPÍTULO 7

Desarrollo e implementación de la solución

Los capítulos anteriores describen qué se quería construir y cómo se diseñó; este capítulo describe cómo se construyó. Para ello se sigue el mismo orden cronológico en el que se ejecutó el proyecto, con una sección por cada uno de los nueve *sprints* planificados en el capítulo 3. En cada una se expone qué estaba previsto en la planificación inicial, qué se hizo realmente, qué patrones de diseño fueron apareciendo, qué dificultades surgieron y cómo se resolvieron. La planificación se redactó antes de escribir la primera línea de código y no se modificó después, de modo que sirvió como referencia fija contra la que medir cada desviación. Por ese motivo el contenido previsto y el realizado no coinciden en varios *sprints*, y precisamente en esas diferencias reside buena parte del interés de este capítulo.

Conviene precisar algunos aspectos prácticos de la metodología que el capítulo 3 no detalla. Cada *sprint* contó con una dedicación de entre veinte y veinticinco horas repartidas de lunes a viernes, y en cada cierre se consideró terminada únicamente la funcionalidad probada de extremo a extremo, no la simplemente escrita. Al tratarse de un único desarrollador no se emplearon ramas de larga duración: se trabajó directamente sobre la rama principal con la regla de no dejar nunca el proyecto sin compilar al final del día. Las revisiones al cierre de cada *sprint* con el *Product Owner* fueron el mecanismo por el que entraron y salieron del alcance las funcionalidades que se comentan a continuación.

En cuanto a la terminología, se mantiene la del capítulo 5: la ronda de votación recibe el nombre de instancia en el código, pero en este capítulo se habla siempre de rondas. Las pantallas de la aplicación a las que se hace referencia se muestran en el apartado 7.10, al final del capítulo, y el detalle de las pruebas automáticas, de carga y de cobertura se reserva para el capítulo 8. Apenas se incluye código fuente, ya que el peso de la explicación recae en las decisiones tomadas y en su justificación.

7.1 Sprint 0: lanzamiento

El primer *sprint* se dedicó, tal y como estaba previsto, a la preparación logística. Se configuró el entorno de desarrollo sobre *Windows*, que era el sistema habitual del ordenador de desarrollo: *Java*, *Node.js*, *Docker Desktop* y los dos entornos de desarrollo integrados. Se creó el repositorio en *GitHub* con una configuración que excluye del control de versiones los secretos, los ficheros de los editores y las salidas de compilación, y se generaron los esqueletos de las tres aplicaciones. Para la gestión se preparó un proyecto en *Jira* con las cuatro fases como épicas y los nueve *sprints* creados por adelantado, y

se mantuvo una primera reunión con el *Product Owner* para fijar el alcance general y el calendario de revisiones.

En este primer *sprint* también se decidió la metodología a seguir y se elaboró la planificación inicial, junto con los objetivos de ingeniería del software que se querían alcanzar en general. Cabe destacar que la idea general del producto a desarrollar se conocía desde antes de empezar los *sprints*, lo que permitió hacerse una idea previa que sirvió para plantear la planificación inicial y la toma de algunas de las decisiones más tempranas.

La única desviación fue de esfuerzo: la configuración de *Jira* consumió más del doble del tiempo estimado por la cantidad de opciones que hubo que ajustar para un proyecto de una sola persona, lo que ya anticipaba el cambio de herramienta del *sprint* siguiente.

7.2 Sprint 1: ingeniería de requisitos

El segundo *sprint* estaba reservado a la ingeniería de requisitos, y en él se tomó la decisión de elaborar una especificación completa, como la que recoge el capítulo 4, en lugar de un simple listado de funcionalidades. La intención era que sirviera a la vez de punto en común con el *Product Owner* y de base para esta memoria. Tras dos reuniones con él se analizó el caso de uso de referencia, las votaciones en máquinas expendedoras, y se recogieron también escenarios en los que el punto de votación no es un lugar sino un canal, como una clase o un grupo. A partir de ahí se definieron los dos actores y sus modelos de persona, los requisitos funcionales y no funcionales, los casos de uso y, derivadas de estos, las historias de usuario que pasaron a formar el *backlog*.

Durante este *sprint* se produjeron dos cambios de herramienta. Al cargar las historias en *Jira* se hizo evidente que añadía fricción sin aportar nada a un equipo de una persona, con campos obligatorios, estimaciones en puntos que nadie iba a comparar e informes que no se consultarían, así que el *backlog* se migró a un tablero de *Trello*. El segundo cambio fue el del sistema operativo: el desarrollo se mudó de *Windows* a *Linux*, el sistema con el que se trabaja según se indica en el capítulo 6, en parte porque *Docker* corre de forma nativa en él y en parte por una transición personal a ese sistema operativo.

Dos dificultades conceptuales marcaron la especificación. La primera fue la identidad del votante: no existe forma fiable de identificar a una persona que no se registra, por lo que se asumió de forma explícita que el objetivo es disuadir el voto repetido casual y no impedir el fraude deliberado, límite que el capítulo 5 documenta. La segunda fue el alcance de la «plataforma generalizada»: cada ejemplo concreto del *Product Owner* empujaba hacia requisitos específicos de las máquinas expendedoras. La desviación fue que la especificación no terminó en la semana prevista: faltaban las matrices de trazabilidad, la revisión con el *Product Owner* y el modelo de dominio. Se decidió no recortarla y continuar en el *sprint* siguiente, asumiendo que toda la planificación posterior se desplazaría.

7.3 Sprint 2: cierre de la especificación y arquitectura

La planificación reservaba este *sprint* para la infraestructura del *backend*, la base de datos en contenedor, las operaciones CRUD y un flujo de integración continua básico. En su lugar se dedicó por completo a cerrar la especificación y a fijar la arquitectura. Fue la mayor desviación del proyecto y también la más consciente: se prefirió partir de una especificación completa y validada a empezar a programar sobre requisitos a medias o sin confirmar con el *Product Owner*.

Se construyeron las matrices de trazabilidad entre requisitos, casos de uso e historias de usuario, y se revisó el documento completo con el *Product Owner* en una sesión de validación. La trazabilidad hizo visible un requisito sin respaldo: la idea inicial era que cada punto de votación representara un lugar físico con coordenadas y que el panel mostrara un mapa con los puntos y su estado. Sin embargo, el mapa no respondía a ningún objetivo del gestor y entraba en conflicto con el requisito de plataforma generalizada, ya que carece de sentido cuando los puntos son elementos lógicos sin ubicación; y cuando sí son físicos, el gestor ya sabe dónde están sus máquinas. Además, exigía una dependencia externa de mapas y datos de geolocalización que mantener. Era la parte visualmente más atractiva de la propuesta inicial y costó retirarla, pero la trazabilidad aportó el argumento objetivo, y el punto quedó definido por su nombre, su descripción y su enlace público. En la misma línea se tomó una segunda decisión de alcance: el requisito RF-16 hablaba de una duración de la votación, pero se decidió que el lanzamiento y el cierre de las rondas fueran acciones manuales del gestor desde el panel, ya que con ello se cubría la necesidad sin incorporar un planificador temporal al *backend*.

Se elaboró la primera versión del modelo de dominio, con cinco entidades: gestor, ítem, votación, punto de votación y voto. El punto tenía una votación asignada y un indicador de si estaba activo, y un voto era exactamente un ítem; se dudó entre ese indicador y una entidad aparte para representar el estado, y se optó por lo más simple. Las decisiones de arquitectura que justifica el capítulo 5, desde la separación en dos aplicaciones de una sola página hasta el código fijo de cada punto, se tomaron y documentaron en este momento, y con ellas quedaron fijados a nivel de diseño el borrado lógico, la separación entre entidades y objetos de transferencia y la estructura en capas de controlador, servicio y repositorio dentro de cada dominio.

En este *sprint* se añadieron las primeras líneas de código al repositorio. El *backend* se creó desde *Spring Initializr* con las dependencias de web, persistencia, validación, seguridad, migraciones y *PostgreSQL*; las dos aplicaciones *Angular* se crearon con la herramienta de línea de órdenes del *framework*. Aunque en este *sprint* no aparece ningún patrón de diseño, sí se fijaron dos convenciones que condicionan todo lo posterior: en el *backend*, un paquete por concepto de negocio y no por capa técnica, tal y como se describe en el capítulo 5; en los *frontends*, componentes autónomos con plantilla y estilos en el propio fichero.

También se escribió el primer fichero de *Docker Compose*, con *PostgreSQL* y un servicio de *backend* todavía vacío, con la intención de que la pila completa se levantara siempre con un solo comando y de que las pruebas manuales se hicieran sobre contenedores y no sobre el servidor de desarrollo. La integración continua, en cambio, se aplazó de forma explícita al final del proyecto: sin código ni pruebas no había nada que integrar. Con la especificación cerrada se replanificó el *backlog*: el contenido previsto para los *sprints* 2, 3 y 4 pasaba a los *sprints* 3, 4 y 5, y se aceptó que la fase de calidad y despliegue tendría que comprimirse.

7.4 Sprint 3: núcleo del backend

Este *sprint* absorbió el contenido previsto originalmente para el anterior. Se escribió la primera migración de *Flyway* con las tablas de gestores, ítems, votaciones y su lista ordenada de ítems, puntos y votos, y se tomaron tres decisiones que se mantuvieron hasta el final: índices parciales que ignoran las filas borradas, claves ajenas que impiden el borrado en cascada, puesto que nunca se hace un borrado físico, y el mapeador objeto-relacional en modo de validación, de forma que el esquema lo gobierna *Flyway* y cualquier divergencia con las entidades impide el arranque.

Al implementar el lanzamiento y la detención como un indicador en el punto quedó claro que las estadísticas se perderían en cada relanzamiento y que no habría forma de saber qué votación se había celebrado si el gestor la cambiaba después. Se consultó con el *Product Owner*, que confirmó que el histórico por lanzamiento era imprescindible, y se introdujo la ronda de votación como entidad propia: cada lanzamiento de una votación en un punto, con su inicio, su fin y su estado. Es la versión del modelo que recoge finalmente el capítulo 4. La garantía de que solo exista una ronda activa por punto se implementó como un índice único parcial en la base de datos y no solo en el código, en aplicación del principio, enunciado en el capítulo 5, de que las invariantes críticas viven en el gestor de base de datos. El rediseño del esquema costó unas pocas horas y fue la primera confirmación de que el modelo de dominio se termina de definir programando.

El resto del *sprint* se dedicó a la autenticación y a las operaciones CRUD. Los *tokens* se firman con HMAC mediante el soporte de servidor de recursos de *Spring Security*, sin librería adicional, y las contraseñas se almacenan con *bcrypt*. Los ítems, las votaciones y los puntos se implementaron con la misma estructura en cada paquete (entidad, repositorio, servicio transaccional, objetos de transferencia y controlador), siguiendo los patrones de capa de servicio y repositorio que describe Fowler [8]: el controlador solo traduce HTTP, el servicio contiene las reglas y las transacciones, y el repositorio codifica en el nombre de cada consulta el filtrado por propietario y la exclusión de lo borrado. Los recursos de otro gestor devuelven un 404 y no un 403, para no revelar su existencia. Los errores adoptaron el formato de detalles de problema de la RFC 9457 mediante un manejador global que extiende la clase base de *Spring*: esta define el algoritmo de conversión de excepciones en respuestas y el proyecto redefine únicamente los pasos que le interesan, como añadir el mapa de errores por campo en las validaciones, lo que constituye una aplicación directa del patrón *Template Method* [9]. La documentación interactiva de la API sirvió como herramienta de prueba manual durante todo el *sprint*, y con el primer *Dockerfile* del *backend* la pila de *Docker Compose* pasó a levantar base de datos y *backend*.

Las dificultades fueron menores pero instructivas: la librería de firma rechazaba un secreto demasiado corto con un error poco descriptivo, así que se añadió una comprobación explícita al arrancar, y la validación del esquema provocó varios arranques fallidos hasta que tipos y longitudes de columna coincidieron con la migración, molestia que se consideró útil porque cualquier deriva futura se detecta antes de atender la primera petición. La única desviación fue que las imágenes de los ítems no cupieron en la semana y se dejaron para hacerlas junto con la interfaz que las sube.

7.5 Sprint 4: panel de gestión

El panel de gestión, previsto inicialmente para el *sprint* anterior, se construyó en este. Se estableció la base de la aplicación con rutas cargadas de forma diferida, estado de cada página mediante señales y localización en español, y se creó el componente de armazón que envuelve las pantallas autenticadas con una barra lateral que, en pantallas estrechas, se convierte en un cajón deslizante. La autenticación se resolvió con un servicio de sesión que guarda el *token* en el almacenamiento del navegador, un interceptor HTTP que añade la cabecera de autorización a cada petición y cierra la sesión ante un 401, y dos guardas de ruta para las zonas privada y pública, junto con las páginas de inicio de sesión y registro (figura 7.1).

Sobre esa base se implementaron las tres parejas de lista y formulario. Los ítems se muestran en una rejilla de tarjetas con imagen y su formulario permite subirla con vista previa y validación en cliente de tipo y tamaño (figura 7.2), lo que obligó a completar el *backend* pendiente del *sprint* anterior: almacenamiento de ficheros con nombre alea-

torio, comprobación de que la ruta resuelta no sale del directorio raíz, servicio público de imágenes con caché y un error claro al superar el límite de tamaño. El formulario de votación se organiza en dos columnas, con los datos generales a un lado y, al otro, un buscador de ítems y la lista ordenable de los seleccionados (figura 7.3); los puntos muestran su votación asignada y su enlace público (figura 7.4). Una vez escritas las tres parejas se extrajeron los elementos repetidos en componentes compartidos: encabezado de página, estado vacío, indicador de carga, avisos y diálogo de confirmación, además de una utilidad central que traduce los detalles de problema del *backend* en mensajes y errores por campo, reutilizada por todos los formularios. Esta separación entre componentes de presentación, que reciben sus datos y no conocen la API, y páginas contenedoras, que cargan datos y mantienen estado, es la que describe el capítulo 5, junto con el cliente de API que actúa como fachada única del *backend* y oculta rutas y formatos al resto de la aplicación. Por último se escribieron los *Dockerfile* de los dos *frontends*, servidos con *nginx* configurado para reenviar `/api` al *backend*, de modo que la pila completa se levantaba ya con un solo comando.

Las dificultades fueron de integración. La librería de iconos declaraba compatibilidad solo hasta la versión anterior de *Angular* y su instalación fallaba; se comprobó que funcionaba y se fijó en el proyecto una instalación permisiva que se respeta también dentro de *Docker*. Enviar datos e imagen en una sola petición *multipart* exigió construir el cuerpo a mano, y las reglas de caché de *nginx* capturaban también las imágenes servidas bajo `/api`, que devolvían 404 hasta que se dio prioridad a ese prefijo. Como desviación, el lanzamiento y la detención de rondas desde el panel se pospusieron al *sprint* siguiente, porque es la interfaz de votación la que los necesita para probarse, y la contenerización de los *frontends* no estaba prevista aquí, pero permitió probar el panel exactamente como se desplegaría.

7.6 Sprint 5: interfaz de votación, rondas y códigos QR

La planificación situaba aquí el panel de estadísticas; en su lugar se construyó la funcionalidad de voto prevista para el *sprint* anterior. En el *backend*, el lanzamiento comprueba que el punto tiene una votación con ítems y que no existe otra ronda activa, la detención cierra la ronda, y se implementaron las reglas de bloqueo que el capítulo 5 enumera: mientras hay una ronda activa no se puede borrar la votación, sus ítems ni el punto, ni cambiar la lista de ítems o la votación asignada. La API pública se redujo a dos operaciones sin autenticación, consultar un punto por su código y votar, y un único servicio concentra todas sus reglas a modo de fachada: punto no borrado, ronda activa y papeleta no duplicada. Esta última comprobación se hace dos veces a propósito, con una consulta previa que permite devolver un mensaje claro y con la captura de la violación de la restricción única al guardar, para el caso de dos peticiones simultáneas con el mismo identificador. Es la misma defensa en profundidad que el diseño aplica a la ronda activa.

La interfaz de votación genera un identificador anónimo de dispositivo la primera vez que se abre, lo guarda en el almacenamiento del navegador y lo conserva también en memoria. La pantalla se modeló como una máquina de seis estados explícita (en carga, punto no encontrado, ronda inactiva, papeleta, voto completado y error), mantenida en una señal sobre la que la plantilla decide qué mostrar (figura 7.7). Se trata de una variante ligera del patrón *State*: se descartó la versión clásica con una clase por estado porque no existe comportamiento polimórfico que justifique tantas clases, y un valor con nombre por pantalla bastaba para evitar las combinaciones inconsistentes de indicadores booleanos. El diseño se hizo para el teléfono, con el botón de confirmación siempre visible y tarjetas grandes. En cuanto a los códigos QR, la idea inicial de generarlos en el servidor

se substituyó por su generación en el navegador del panel, donde la información necesaria ya está disponible, con descarga en PNG y un botón para copiar el enlace; el enlace que va dentro del código lo construye el *backend* a partir de la dirección pública configurada, y la permanencia que exige el requisito RF-13 se resuelve con el código fijo del punto, de modo que un mismo QR impreso vale para todas las votaciones futuras. Las acciones de lanzar y detener, con confirmación, se añadieron a la lista de puntos (figura 7.4).

Aparecieron algunas dificultades más en la implementación, pero ninguna de demasiada relevancia. La desviación respecto a lo previsto fue de forma más que de fondo: la pantalla principal resultó más simple de lo planeado, pero la aplicación necesitó más estados de los dos contemplados, y cada uno de ellos apareció al probar en un navegador real.

7.7 Sprint 6: tipos de votación y estadísticas

Este *sprint* debía dedicarse, según el plan, al refuerzo de la seguridad y a las pruebas de rendimiento, y tras la replanificación le correspondían las estadísticas. Sin embargo, la demostración del incremento anterior al *Product Owner* produjo dos peticiones inmediatas: poder elegir varias opciones y poder ordenarlas. La primera ya estaba contemplada en el requisito RF-24, pero el incremento solo implementaba el voto único; la segunda, el *ranking*, no aparecía en ningún requisito. Ambas afectaban al núcleo del modelo, donde un voto era un ítem y la restricción de unicidad recaía sobre él, y cuanto más tarde se abordaran, más caras resultarían, así que se decidió hacerlas en este mismo *sprint* junto con las estadísticas, que dependían del resultado.

El cambio de modelo introdujo la papeleta como participación de un dispositivo en una ronda, que contiene uno o varios votos con una posición opcional. La segunda migración de *Flyway* añade el tipo y el máximo de selecciones a la votación, crea las papeletas, migra los votos existentes agrupándolos por ronda y dispositivo, y traslada la restricción de unicidad del voto a la papeleta. Las reglas de cada tipo se encapsularon en el patrón *Strategy* que justifica el apartado 5.2.3, con una interfaz de tres responsabilidades (validar la configuración, validar una papeleta y puntuar los resultados) y tres implementaciones. En las interfaces se añadió el selector de tipo en el formulario de votación, con el campo de máximo visible solo para los votos limitados, y en la interfaz de votación el contador de selecciones con bloqueo al llegar al máximo y la numeración por orden de toque (figura 7.7).

Las estadísticas se construyeron sobre este modelo. El servicio compone la lista de candidatos con los ítems actuales de la votación, incluidos los borrados y marcados como tales, más los que recibieron votos y ya no pertenecen a ella, y delega la puntuación en la estrategia del tipo. Se organizan en tres niveles, que cubren los requisitos RF-30, RF-31 y RF-32. El primero es el listado de rondas celebradas, que admite dos filtros combinables, por punto de votación y por votación, cuyas opciones se derivan del propio listado para que aparezcan también los puntos y las votaciones ya borrados; la página de resultados de una ronda muestra barras de progreso, porcentajes y ganador destacado y, en los *rankings*, los puntos de Borda, la posición media y los primeros puestos (figura 7.6), y mientras la ronda está activa se refresca automáticamente. El segundo son los resultados agregados de una votación a lo largo de todos sus lanzamientos, accesibles desde el botón «Resultados» de cada votación, con el número de lanzamientos, las papeletas recogidas, la clasificación de los ítems y un selector de punto que restringe la suma a los lanzamientos celebrados en un punto concreto. El tercero es el historial de cada ítem, accesible desde el botón «Historial» de la lista de ítems y desde su nombre en cualquier clasificación, con sus participaciones, victorias y porcentaje medio, y una tabla con el puesto y el

resultado obtenidos en cada ronda en la que fue candidato. Se completó además la página de inicio, con contadores y las rondas en marcha, y una página de votaciones activas con lanzamiento y refresco automático (figura 7.5). Se descartó incorporar una librería de gráficos: las barras dibujadas con CSS, encapsuladas en un componente de clasificación que comparten los resultados por ronda y por votación, cubren la necesidad y mantienen el panel sin recursos externos.

Las dificultades se concentraron en la migración y en el recuento. La primera versión de la migración borraba y recreaba la tabla de votos, y se reescribió como una migración de datos real probada contra una base con votos del *sprint* anterior. En los *rankings* parciales hubo que decidir qué puntuación recibe un ítem que un votante no ordenó: se optó por cero puntos y por expresar el porcentaje sobre el máximo de puntos posible, y la fórmula se documentó en la propia interfaz. Los ítems votados y luego retirados de la votación desaparecían de los resultados hasta que se añadió la consulta que los recupera, y el listado de rondas, que lanzaba una consulta por ronda, se sustituyó por una única consulta agregada. Para los resultados por votación y el historial por ítem se valoró escribir consultas SQL de agregación específicas y se descartó: en su lugar se cargan por lotes las papeletas y los votos de todas las rondas implicadas, con dos consultas en total, y se puntúan con la misma función de la estrategia que resuelve una sola ronda, de modo que el recuento existe una sola vez en el código y los tres niveles dan los mismos números. En el *frontend*, el cambio de «un ítem» a «lista de ítems» se propagó con el compilador de TypeScript señalando cada punto afectado, lo que fue el argumento definitivo a favor de la configuración estricta. La desviación fue clara: dos *sprints* de contenido en uno, a costa de dejar la revisión de seguridad y todas las pruebas automáticas para el siguiente.

7.8 Sprint 7: pruebas, seguridad y despliegue continuo

Este *sprint* coincidió en buena parte con el plan original (despliegue continuo y pruebas finales), pero absorbió además la seguridad y el rendimiento que no cupieron en el anterior. Las tres estrategias y su registro se probaron de forma unitaria sin base de datos. Para las pruebas de integración se descartó una base de datos en memoria, porque no soporta los índices únicos parciales del esquema, y se adoptó *Testcontainers* con *PostgreSQL*, de modo que las pruebas corren contra la misma base de datos que producción y validan también las migraciones. Una clase base levanta el contenedor, registra un gestor y obtiene un *token* real, sin usuarios simulados, y las clases de prueba recorren por HTTP la autenticación, el aislamiento entre gestores, el ciclo completo de lanzar, votar, duplicar, detener y consultar estadísticas, los tres tipos de votación, y los resultados agregados por votación y el historial por ítem. En el panel se probaron con *Vitest* el listado de estadísticas y el componente de clasificación, sustituyendo el cliente de la API por un doble y comprobando lo que se muestra. Un inicializador opcional de datos de prueba aceleró las pruebas manuales, la carga y las demostraciones. La revisión de seguridad recorrió una lista de riesgos habituales contra el código, desde el recorrido de rutas en la lectura de imágenes hasta el aislamiento por propietario en todas las consultas; no aparecieron fallos nuevos, se homogeneizaron mensajes y se añadieron pruebas para los puntos sensibles. Las pruebas de carga se escribieron con *k6* y se ejecutan a mano contra la pila *Docker* local; su diseño y sus resultados se presentan en el capítulo 8.

Con el producto ya maduro se decidió cómo publicarlo. La opción rápida era una plataforma gestionada con plan gratuito, y se descartó porque esos planes duermen los servicios tras un tiempo de inactividad, lo que haría esperar medio minuto al primer votante, porque la dirección que va impresa en los códigos QR debe ser estable y propia, y porque se quería controlar la pila completa. Se optó por un servidor virtual pequeño

con dos subdominios y *Caddy* como *proxy* inverso con certificados automáticos, ya que HTTPS no era opcional: la generación del identificador del votante y el acceso al portapapeles solo funcionan en contextos seguros. El flujo de *GitHub Actions* anunciado en el capítulo 5 se configuró con tres trabajos encadenados, de forma que cada uno se ejecuta solo si el anterior termina bien. El primero ejecuta las pruebas del *backend* con *Testcontainers* y compila y prueba los dos *frontends*; el segundo construye las tres imágenes y las publica en el registro de contenedores de *GitHub* etiquetadas con el identificador del *commit*; el tercero se conecta por SSH al servidor, descarga las imágenes nuevas y reinicia los contenedores afectados. Se escribió también la variante de producción del fichero de *Docker Compose*, con imágenes publicadas y sin puertos expuestos salvo los de *Caddy*, junto con la configuración de este último.

Entre las dificultades, *Testcontainers* exige *Docker* también en el ejecutor de la integración continua, que lo incluye, pero no es posible ejecutar las pruebas durante la construcción de la imagen, así que la imagen se construye sin ellas y las pruebas corren antes, en el primer trabajo. Al no deshacer las transacciones, la base de datos del contenedor acumula filas entre pruebas; se aceptó el aislamiento por datos y se anotó como limitación. Y *nginx* cortaba las imágenes grandes antes de que llegaran al *backend*, así que se elevó su límite por encima del de la aplicación para que el error lo dé siempre esta con su mensaje. La desviación fue que el despliegue quedó a medias: el flujo pasaba las pruebas y publicaba las imágenes, pero el primer despliegue real con HTTPS operativo se completó en el *sprint* siguiente.

7.9 Sprint 8: estabilización y cierre

El último *sprint* estaba reservado como margen de estabilización y se consumió en parte con el despliegue pendiente. Se apuntaron los subdominios al servidor, *Caddy* obtuvo los certificados en el primer arranque, se configuraron los secretos de despliegue en *GitHub* y se realizó el primer despliegue completo a partir de un *push*. Se añadieron un guion de copias diarias de la base de datos y de las imágenes, y una guía paso a paso del despliegue. Se incorporó también la medición de cobertura con *JaCoCo*, cuyo informe señaló un caso sin probar, la edición de un punto con una ronda activa, para el que se añadió la prueba correspondiente; su configuración y sus cifras se analizan en el capítulo 8.

Con la dirección pública disponible se probó la aplicación en varios teléfonos escaneando códigos QR impresos, y de ahí salió la lista de defectos menores que se corrigieron: el cajón lateral del panel no se cerraba al navegar, en los *rankings* faltaba una forma rápida de deshacer todo el orden, tras un cierre de sesión automático no se volvía a la página solicitada al entrar de nuevo, y algunos textos de estado resultaban ambiguos en pantallas pequeñas. El *sprint* terminó con la revisión final de la documentación técnica del repositorio, la retrospectiva global y el cierre del tablero.

No apareció ningún patrón nuevo, pero al revisar el catálogo se documentaron también los patrones que se valoraron y descartaron, con el criterio de no añadir ninguna abstracción que no tuviera al menos dos implementaciones reales. Así, una fábrica abstracta para el almacenamiento de imágenes solo tendría sentido si existiera una alternativa al disco local; un decorador que cacheara la consulta pública de un punto, si la carga lo exigiera; un patrón *Command* con histórico de acciones, si hubiera que deshacerlas; un observador de eventos de dominio, si el panel recibiera notificaciones en tiempo real; y la versión clásica del patrón *State*, si las rondas tuvieran más estados con reglas propias. Es el mismo criterio con el que el capítulo 5 justifica que el *Strategy* de los tipos de votación sea el único patrón de objetos introducido de forma deliberada.

Dos incidencias del primer despliegue se documentaron para no repetirlas: el secreto de firma de los *tokens* era distinto entre el entorno local y el servidor, con lo que todas las sesiones del panel caducaron a la vez, y la contraseña de la base de datos se cambió después del primer arranque, cuando la imagen de *PostgreSQL* ya había inicializado el volumen con la anterior, de modo que el *backend* no conectaba. Quedaron fuera de forma explícita, y así se recoge en el capítulo 8, la ampliación de las pruebas unitarias de los *frontends*, las pruebas de extremo a extremo automatizadas y un umbral mínimo de cobertura.

7.10 Recorrido de uso de la aplicación

Las secciones anteriores describen cómo se construyó Krate; en esta se muestra el resultado tal y como lo utilizan sus dos actores, el gestor y el votante. Se sigue el orden habitual de una puesta en marcha: el gestor crea su cuenta, da de alta los ítems, compone una votación, crea un punto de votación, le asigna la votación, lanza una ronda y, una vez recogidas las papeletas, consulta las estadísticas; el votante, por su parte, solo escanea un código QR y elige. Se ilustran únicamente las pantallas principales de cada paso, y el resto se describe en el texto. Aunque el recorrido se presenta de forma lineal, ningún paso exige que el anterior esté cerrado: los ítems, las votaciones y los puntos pueden editarse en cualquier momento, con las únicas restricciones que impone una ronda activa y que se indican más adelante.

7.10.1. El gestor: de la cuenta a las estadísticas

El primer contacto del gestor con Krate es la página de inicio de sesión del panel de gestión (figura 7.1), desde la que se accede al formulario de registro, que solo pide un correo electrónico, un nombre y una contraseña. Al completarlo la sesión queda iniciada y el panel muestra su página de inicio, con los contadores de rondas activas, votaciones, ítems, puntos y rondas celebradas, que sirven a la vez de acceso a cada sección, y la lista de rondas en marcha, que en una cuenta recién creada aparece vacía junto a un enlace para lanzar la primera. En accesos posteriores basta con iniciar sesión; esta dura veinticuatro horas, tras las cuales el panel devuelve a esta pantalla y, una vez dentro de nuevo, conduce a la página que se estaba consultando. El cierre de sesión está siempre disponible en la barra lateral, que en pantallas estrechas se pliega en un cajón deslizante.

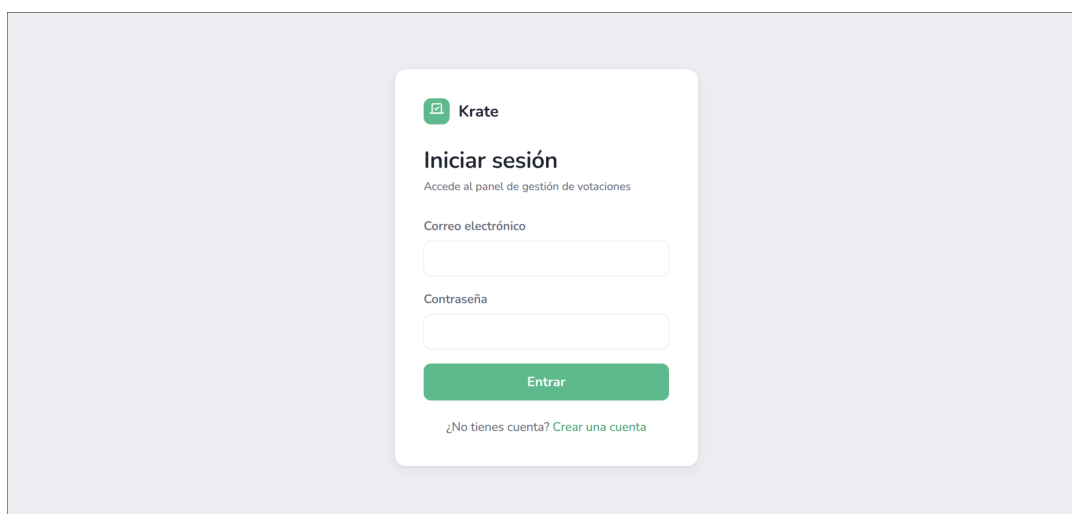


Figura 7.1: Página de inicio de sesión del panel de gestión, con el acceso al registro.

El siguiente paso es dar de alta las opciones que se van a someter a votación. La sección de ítems los presenta en una rejilla de tarjetas con su imagen, su nombre y su descripción (figura 7.2), y cada tarjeta ofrece las acciones de consultar el historial, editar y eliminar. El formulario de alta pide un nombre, una descripción breve y, de forma opcional, una imagen, que se muestra en vista previa antes de enviarla y se valida en el navegador en tipo y tamaño. Un ítem no pertenece a ninguna votación en concreto: forma parte del catálogo del gestor y puede aparecer en tantas votaciones como se quiera. El borrado es lógico, de modo que el ítem desaparece de la lista y deja de poder seleccionarse, pero conserva su historial en las estadísticas.

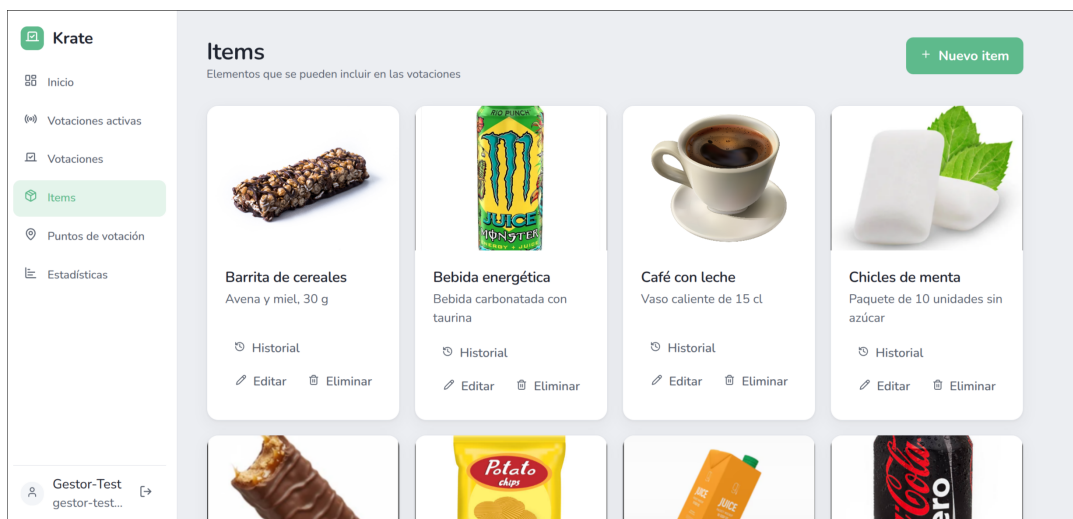


Figura 7.2: Catálogo de ítems del gestor, con las acciones de historial, editar y eliminar en cada tarjeta.

Con los ítems creados se compone la votación. La lista de votaciones (figura 7.3) muestra para cada una su descripción, el número de ítems, su estado y las acciones de ver resultados, editar y eliminar. El formulario se organiza en dos columnas: a un lado, el nombre, la descripción y el tipo de votación, con el máximo de selecciones que solo aparece cuando se eligen los votos limitados; al otro, un buscador sobre el catálogo de ítems y la lista ordenada de los seleccionados, cuyo orden es el que verá el votante en la papeleta. La votación es una plantilla reutilizable: define qué se pregunta y cómo se responde, pero no dónde ni cuándo, y eso es precisamente lo que aportan los puntos de votación.

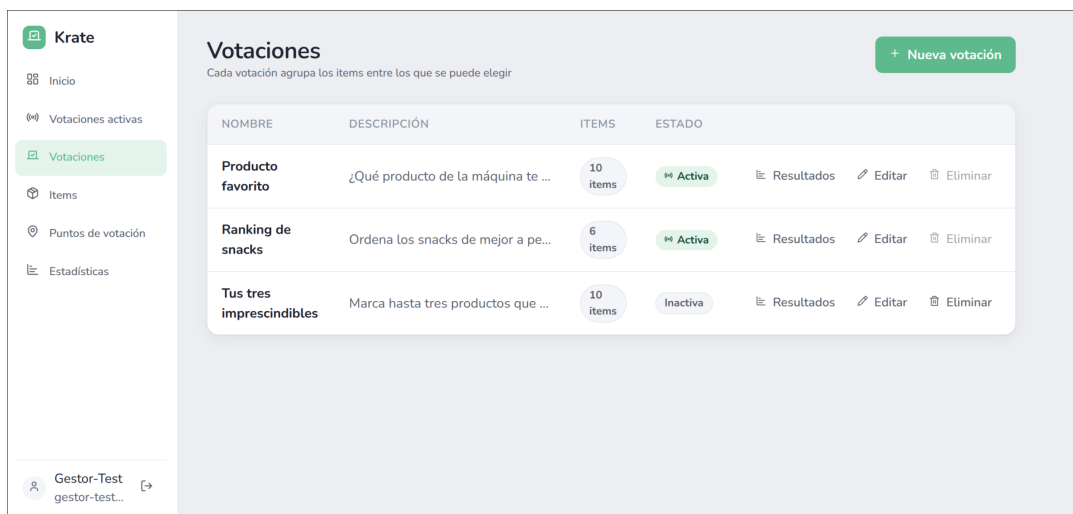


Figura 7.3: Lista de votaciones del gestor, con las acciones de resultados, editar y eliminar.

El punto de votación representa el lugar, físico o lógico, en el que se vota. Su formulario pide un nombre, una descripción y la votación asignada, que se elige entre las existentes y puede cambiarse después. Al crearlo, el sistema le asigna un código fijo y, con él, un enlace público de la forma `/p/{código}`, que es el que va dentro del código QR. La lista de puntos (figura 7.4) muestra cada uno con su votación asignada, su enlace con un botón para copiarlo, el acceso al diálogo del código QR, con descarga en PNG para imprimirlo, y las acciones de lanzar o detener. Como el enlace no cambia nunca, el gestor imprime el código una sola vez y lo coloca en la máquina, el aula o el mostrador; todas las votaciones que se asignen al punto en el futuro se servirán a través de ese mismo código.



Figura 7.4: Puntos de votación con la votación asignada, el enlace público, el código QR y las acciones de lanzar y detener.

Hasta aquí nada es visible para los votantes: el enlace del punto responde con la pantalla de votación desactivada. Lanzar una ronda es una acción explícita, disponible en la lista de puntos y en la página de votaciones activas (figura 7.5), que pide confirmación y comprueba que el punto tiene asignada una votación con al menos un ítem y que no hay ya otra ronda activa en él. A partir de ese momento la página de votaciones activas muestra la ronda en marcha con su punto, su votación, la hora de inicio, el número de papeletas recibidas, que se refresca automáticamente, y su enlace público; la página de inicio la recoge también entre las rondas en marcha. Mientras la ronda está activa el panel impide las operaciones que la dejarían inconsistente: eliminar la votación, sus ítems o el punto, y cambiar la lista de ítems, el tipo, el máximo de selecciones o la votación asignada; el intento se responde con un aviso que explica el motivo, aunque el nombre y la descripción sí pueden editarse. Detener la ronda, también con confirmación, la cierra de forma definitiva, devuelve el enlace a la pantalla de votación desactivada y conserva la ronda con sus papeletas para las estadísticas. En el mismo punto se pueden lanzar tantas rondas como se quiera, con la misma votación o con otra.

El recorrido termina en las estadísticas, organizadas en los tres niveles descritos en el *sprint* 6. La sección de estadísticas presenta el listado de todas las rondas celebradas, activas o cerradas, con dos filtros combinables por punto y por votación. Desde cada una se accede a su página de resultados (figura 7.6), que muestra el número de papeletas y la clasificación de los ítems con barras, porcentajes y el ganador destacado; en las votaciones de tipo *ranking* se añaden los puntos de Borda, la posición media y el número de primeros puestos de cada ítem. Si la ronda sigue activa, la página se refresca sola, lo que permite seguir la votación en directo. Los otros dos niveles se alcanzan desde las propias listas: el botón de resultados de una votación agrega todos sus lanzamientos, con un selector para restringir la suma a los celebrados en un punto concreto; el historial de un ítem,

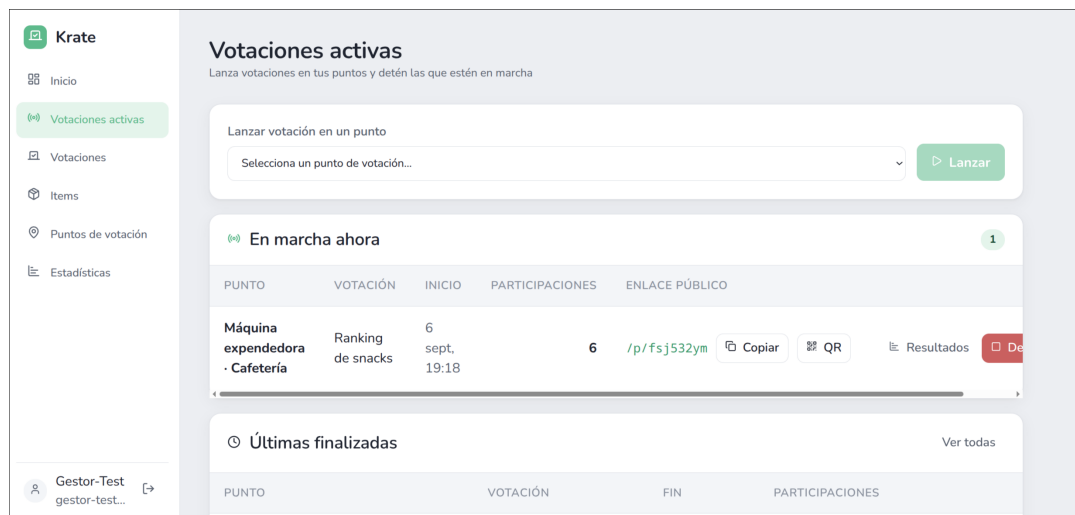


Figura 7.5: Página de votaciones activas con una ronda en marcha.

disponible en su tarjeta y en su nombre dentro de cualquier clasificación, resume sus participaciones, sus victorias y el puesto obtenido en cada ronda en la que fue candidato.

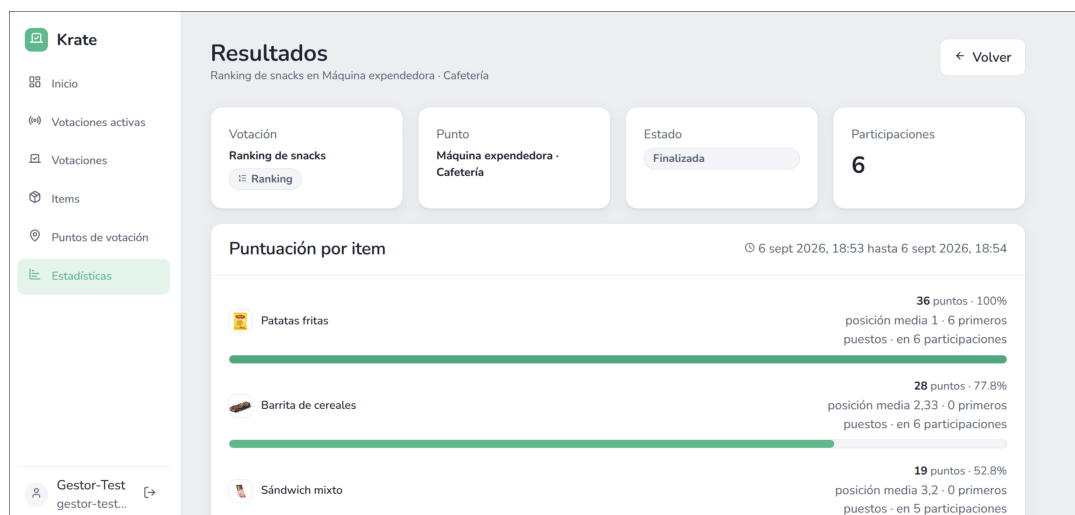


Figura 7.6: Página de resultados de una ronda de votación.

7.10.2. El votante: del código QR a la papeleta

La experiencia del votante se ha reducido deliberadamente a lo mínimo. Escanea el código QR con la cámara del teléfono, o abre el enlace si lo ha recibido por otro canal, y la interfaz de votación se carga en el navegador sin instalar nada ni identificarse. En ese primer acceso la aplicación genera un identificador anónimo de dispositivo y lo guarda en el navegador; es lo único que identificará al votante en adelante. Si el punto tiene una ronda activa aparece la papeleta (figura 7.7): el nombre de la votación y el del punto en el que se celebra, una instrucción adaptada a su tipo, las opciones en tarjetas grandes con su imagen y un botón de confirmación que permanece siempre visible en la parte inferior. En el voto único, tocar una opción sustituye a la anterior; en los votos limitados, cada toque marca o desmarca una opción, un contador indica cuántas se han elegido y al alcanzar el máximo el resto se desactiva; en el *ranking*, cada toque añade la opción al final del orden con su número de posición, tocarla de nuevo la retira y un enlace permite reiniciar el

orden completo. Al confirmar, la papeleta se envía y la pantalla de agradecimiento cierra el proceso.

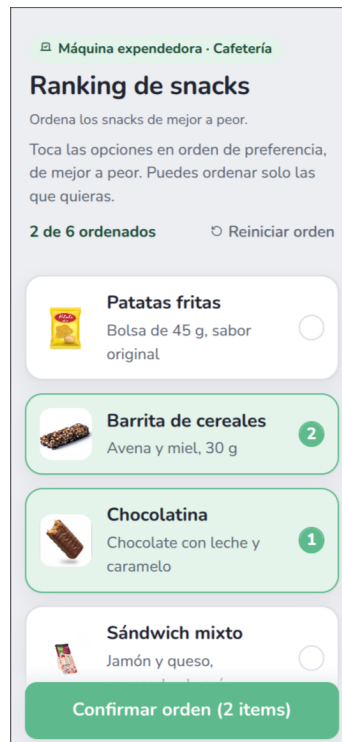


Figura 7.7: Interfaz de voto con la papeleta de una ronda activa.

A partir de ahí el votante puede encontrarse con otras pantallas, todas ellas de estado y con el mismo diseño de icono, título y mensaje breve. Si vuelve a abrir el enlace, o recarga la página, después de haber votado, la aplicación reconoce su identificador y muestra de nuevo el agradecimiento en lugar de la papeleta, que es el mecanismo con el que se disuade el voto repetido. Si accede al enlace de un punto sin ninguna ronda activa, porque todavía no se ha lanzado o porque ya se ha detenido, ve la pantalla de votación desactivada, con el nombre del punto y un botón para volver a comprobar, pensada para el caso habitual de un código impreso que permanece en su sitio entre una votación y la siguiente. Si el código no existe, porque el enlace está mal escrito o el punto se ha borrado, aparece la pantalla de enlace no válido. Ante un fallo de red se ofrece reintentar, y si la ronda se detiene entre la carga de la papeleta y su envío, o el dispositivo ya había votado desde otra pestaña, la aplicación no supone qué ha ocurrido, sino que vuelve a consultar el estado del punto y muestra la pantalla que corresponda.

7.11 Conclusiones del desarrollo

La tabla 7.1 resume, para cada *sprint*, lo que la planificación inicial del capítulo 3 preveía y lo que finalmente se hizo. Las cuatro fases se cumplieron en su orden, pero con un desplazamiento de un *sprint* a partir del segundo, causado por la decisión de completar la especificación de requisitos antes de programar. Ese retraso se recuperó en el *sprint* 6 al juntar los tipos de votación con las estadísticas, a costa de mover las pruebas al *sprint* 7, que era donde el plan original ya las situaba. En ningún caso se alargó la semana de trabajo: cuando faltó tiempo se movió alcance, y el margen de estabilización del último *sprint* cumplió precisamente la función para la que se había reservado.

Sprint	Planificado	Realizado
0	Entornos y herramientas de gestión	Lo previsto, sobre <i>Windows</i> y con <i>Jira</i>
1	Requisitos, <i>backlog</i> y modelo de dominio	Especificación extensa sin terminar; cambio a <i>Trello</i> y a <i>Linux</i>
2	<i>Backend</i> , base de datos, CRUD e integración continua	Trazabilidad, validación, modelo de dominio, arquitectura y <i>Docker</i> inicial
3	Panel de gestión	Núcleo del <i>backend</i> ; aparece la ronda de votación
4	Interfaz de voto y códigos QR	Panel de gestión e imágenes de ítems
5	Panel de estadísticas	Interfaz de voto, rondas, API pública y códigos QR
6	Seguridad y rendimiento	Tipos de votación y estadísticas
7	Despliegue continuo y pruebas finales	Pruebas, carga, seguridad, flujo de despliegue y elección del servidor
8	Estabilización	Despliegue final, cobertura, defectos y documentación

Tabla 7.1: Contenido planificado y realizado en cada *sprint*.

Este recorrido es, en el fondo, una demostración real de las prácticas ágiles adoptadas en el capítulo 3. El alcance y la especificación del proyecto no se fijaron una sola vez al principio, sino que se fueron modificando conforme el desarrollo aportaba información nueva: el mapa de puntos se retiró cuando la trazabilidad mostró que no respondía a ningún objetivo; la duración de las votaciones se sustituyó por un control manual; la ronda de votación apareció como entidad al implementar el lanzamiento; los tipos de votación entraron a petición del *Product Owner* tras una demostración; y la integración continua se aplazó hasta que hubo pruebas que ejecutar. Ninguno de esos cambios rompió la base del proyecto porque los *sprints* de una semana permitían reorganizar el trabajo en el siguiente ciclo, el *backlog* estructurado mantenía visible lo que seguía siendo prioritario y las revisiones al cierre de cada *sprint* convertían cada desviación en una decisión tomada con el cliente en lugar de en una improvisación. El único cambio de alcance de calado, los tipos de votación, vino del *Product Owner* y no de un requisito mal entendido, lo que a su vez confirma el valor de haber invertido dos *sprints* en la especificación.

Quedan, por último, tres lecciones que la retrospectiva consideró más valiosas. El modelo de dominio se termina de definir programando, y consultar al *Product Owner* en cuanto surge la duda, como ocurrió con la ronda, evita entidades mal definidas. Las pruebas automáticas deberían haber empezado antes: concentrarlas en el *sprint* 7 hizo que el cambio de modelo del *sprint* 6 se abordara sin red. Y el cuaderno de bitácora llevado diariamente en *Obsidian* ha aportado un valor enorme para la posterior redacción en detalle de este capítulo.

CAPÍTULO 8

Pruebas y calidad

Las pruebas son una de las partes más importantes a la hora de desarrollar un producto software de calidad. Son una de las principales herramientas contra las malas prácticas en el código y los errores humanos en la programación, y en muchas ocasiones simplemente permiten ver elementos y situaciones que se habían pasado por alto. Unas pruebas bien diseñadas y ejecutadas con precisión permiten que el mantenimiento y la ampliación del software sean tareas más sencillas en el futuro.

Los capítulos anteriores han ido remitiendo a este la descripción de las pruebas: el capítulo 5 anuncia las que ejecuta el flujo de integración continua, el capítulo 6 presenta las herramientas y deja para aquí su uso y sus resultados, y el capítulo 7 sitúa en el tiempo cuándo se escribieron. Las pruebas de Krate persiguen cuatro garantías, por orden de importancia: que el recuento sea correcto, porque una plataforma de votaciones vale lo que valen sus resultados; que las reglas de negocio se cumplan en la API y no solo en la interfaz, ya que el aislamiento entre gestores, la papeleta única, los bloqueos con ronda activa y el borrado lógico se aplican en el *backend*; que el esquema de la base de datos y el código coincidan, puesto que las migraciones y las restricciones de *PostgreSQL* forman parte del comportamiento; y que la parte pública aguante a muchos votantes a la vez, en particular que la papeleta única resista dos envíos idénticos en el mismo instante. De estas garantías se deriva el reparto del esfuerzo que se describe a continuación.

8.1 Estrategia y automatización

Nivel	Herramientas	Casos	Cuándo se ejecuta
Unitarias del <i>backend</i>	<i>JUnit 5</i> y <i>AssertJ</i> , sin <i>Spring</i>	15	En local y en el flujo de <i>GitHub Actions</i>
Integración del <i>backend</i>	<i>JUnit 5</i> , <i>Spring Boot Test</i> , <i>MockMvc</i> y <i>Testcontainers</i>	18	Igual que las anteriores; requiere <i>Docker</i>
Unitarias de los <i>frontends</i>	<i>Vitest</i> y <i>jsdom</i>	7	En local y en el flujo, tras compilar
Extremo a extremo	<i>curl</i> y <i>Chromium</i> sin interfaz sobre la pila <i>Docker</i>	Manual	Durante el desarrollo y antes del primer despliegue
Carga de la API pública	<i>k6</i>	2 escenarios	A mano contra la pila local, fuera del flujo

Tabla 8.1: Niveles de prueba de Krate, herramientas, número de casos y momento de ejecución.

La mayor parte del esfuerzo se ha concentrado en pruebas de integración del *backend* contra una base de datos *PostgreSQL* real, porque ahí residen las reglas de negocio y el contrato HTTP que consumen las dos aplicaciones *Angular*. La lógica de puntuación, que es pura y no depende de *Spring*, se prueba de forma unitaria con ejemplos numéricos. Los *frontends* son capas finas sobre la API y se validan con la compilación, con unas pocas pruebas unitarias de arranque y de la sección de estadísticas del panel, y con comprobaciones manuales sobre la pila de contenedores. La concurrencia se mide con una prueba de carga que se ejecuta a mano. La tabla 8.1 resume los cinco niveles.

En total, el *backend* cuenta con 33 casos automáticos repartidos en 11 clases de prueba, y los dos *frontends* con 7 casos. La fase de verificación de *Maven* ejecuta las unitarias y las de integración en una misma pasada y genera después el informe de cobertura, y en cada *frontend* la compilación precede a las pruebas. El trabajo de pruebas del flujo de *GitHub Actions* descrito en el capítulo 5 ejecuta esos mismos comandos en cada integración en la rama principal, y los trabajos de publicación de imágenes y despliegue dependen de él, de modo que una versión que no supera las pruebas nunca llega al servidor. Un inicializador opcional de datos, desactivado en producción, crea al arrancar un gestor con diez ítems, una votación de cada tipo y dos puntos, y es el estado de partida de la prueba de carga y de las comprobaciones manuales. Por último, en coherencia con lo expuesto en el capítulo 6, los casos de prueba se han escrito con ayuda de un asistente basado en modelos de lenguaje, que propuso la estructura de cada clase y buena parte de las aserciones; los ejemplos numéricos se calcularon a mano y cada caso se revisó antes de incorporarlo, ya que una prueba que comprueba lo incorrecto es peor que ninguna.

8.2 Pruebas unitarias del backend

Las tres estrategias de votación del apartado 5.2.3 y el registro que las resuelve son objetos sin estado que reciben listas y devuelven resultados, por lo que se prueban instanciándolos directamente, sin contexto de *Spring* ni base de datos, en menos de una décima de segundo. Cada estrategia se prueba en sus tres responsabilidades. En la validación de la configuración se comprueba que el máximo de selecciones es obligatorio en los votos limitados, con un mínimo de dos y sin superar el número de ítems, y que los otros dos tipos lo rechazan. En la validación de la papeleta se enumeran las combinaciones válidas e inválidas de cada tipo: vacía, con un ítem repetido o ajeno, por encima del máximo y, en el *ranking*, la parcial que ordena solo algunos ítems, que sí se admite. En la puntuación se contrastan los resultados con ejemplos calculados a mano y anotados en la propia prueba: con tres ítems y las papeletas [A, B, C], [B, A] y [A], el ítem A suma ocho puntos Borda, el 88,9 % de los nueve posibles, con una posición media de 1,33, B suma cinco y C uno; otro caso comprueba que gana quien tiene más puntos aunque otro ítem tenga mejor posición media.

La prueba del registro garantiza que existe una estrategia para cada tipo, que las votaciones sin tipo se tratan como de voto único y que, si falta una estrategia, la aplicación falla al arrancar y no en la primera votación. Con ello, cualquier cambio en el reparto de puntos o en el redondeo rompe una aserción con un número concreto, las reglas de papeleta quedan documentadas de forma ejecutable y el patrón *Strategy* queda protegido.

8.3 Pruebas de integración del backend

Las pruebas de integración arrancan la aplicación *Spring Boot* completa, con seguridad, persistencia, migraciones y controladores, y le envían peticiones HTTP simuladas

mediante *MockMvc*, sin abrir un puerto. La base de datos es un contenedor *PostgreSQL* 17 que *Testcontainers* levanta al inicio de la ejecución y destruye al final, con la misma configuración que en producción. Como se justificó en los capítulos 6 y 7, se descartó una base de datos en memoria porque no reproduce el índice único parcial ni la restricción única de papeleta, y porque con *PostgreSQL* real cada ejecución aplica las migraciones y valida que las entidades coinciden con las tablas. El precio es necesitar *Docker* y unos segundos de arranque; después el contexto se reutiliza y cada clase corre en décimas de segundo.

Regla de negocio	Clase de prueba
Aislamiento por gestor: un recurso ajeno responde 404	ItemsTest, VotingFlowTest, StatsFlowTest
Una sola ronda activa por punto	VotingFlowTest
Una papeleta por dispositivo y ronda	VotingFlowTest, VotingTypesFlowTest; bajo concurrencia, <i>k6</i>
Contenido de la papeleta validado por la estrategia	Clases unitarias; VotingTypesFlowTest
Bloqueos con ronda activa: borrar votación, ítems o punto; cambiar ítems, tipo, máximo o votación asignada	VotingFlowTest, VotingTypesFlowTest
Nombre y descripción editables con ronda activa	VotingFlowTest
Borrado lógico: fuera de los listados, código público en 404, estadísticas conservadas	ItemsTest, VotingFlowTest, StatsFlowTest
Los resultados por votación suman todos sus lanzamientos y el filtro por punto restringe la suma	StatsFlowTest
El historial de un ítem cuenta cada ronda en la que fue candidato, aun sin votos	StatsFlowTest
Punto cuya votación se borra: queda sin votación y no se lanza	VotingFlowTest
El lanzamiento exige una votación con ítems suficientes	VotingFlowTest, VotingTypesFlowTest
Imágenes solo JPEG, PNG y WebP, servidas públicamente	ItemsTest
Validación de entrada con un error por campo	AuthFlowTest, ItemsTest
Rutas de gestión protegidas por <i>token</i>	AuthFlowTest y, de forma indirecta, todas las demás

Tabla 8.2: Reglas de negocio transversales y clases de prueba que las ejercitan.

Tres decisiones explican la forma de estas pruebas. La autenticación es real: cada prueba registra un gestor a través de la API y utiliza el *token* que esta devuelve, sin usuarios simulados, de modo que en cada petición se ejercitan el filtro de seguridad, la firma del *token* y el codificador de contraseñas como en producción. No hay dobles ni pruebas por capas: cada caso atraviesa controlador, servicio, repositorio y base de datos, con lo que se pierde granularidad al diagnosticar, pero cada aserción sobre un código de estado confirma la cadena completa, incluida la traducción de errores. Y el aislamiento entre pruebas se consigue por datos y no deshaciendo transacciones: cada prueba crea su propio gestor y solo consulta lo que ese gestor ve, que es la misma regla de aislamiento por propiedad.

rio que garantiza la aplicación, y de paso la pone a prueba; la acumulación de filas en el contenedor es inocua por tratarse de un contenedor desechable.

Las siete clases de integración recorren el comportamiento observable de la API: el arranque contra la base de datos real, que es lo primero que falla si una migración y una entidad divergen; el registro, el inicio de sesión, los errores de validación y la exigencia de *token* en las rutas protegidas; el ciclo de vida de los ítems con imagen y el aislamiento entre gestores; el ciclo completo de una ronda, desde el lanzamiento y los bloqueos con un 409 hasta el voto, el rechazo del segundo voto del mismo dispositivo, el cierre y unas estadísticas que conservan los votos de los ítems borrados y sobreviven al borrado de la votación y del punto; los votos limitados y el *ranking*, que obtiene los mismos números que la prueba unitaria de Borda atravesando la base de datos; las estadísticas agregadas, con una misma votación lanzada en dos puntos cuyo resultado global y por punto se contrasta con cifras calculadas a mano, y el historial de un ítem que participa en una votación de voto único y en un *ranking*; y el inicializador de datos. La tabla 8.2 relaciona las reglas de negocio transversales del capítulo 5 con la clase que las ejercita; las únicas sin prueba automática propia son el límite de tamaño de las imágenes y el nombrado aleatorio de los ficheros subidos, que se comprobaron manualmente.

8.4 Pruebas de los frontends y comprobaciones manuales

Las dos aplicaciones *Angular* utilizan el ejecutor de pruebas que su interfaz de línea de órdenes configura por defecto, *Vitest* con *jsdom* como navegador simulado. Cada una tiene un fichero que monta el componente raíz, con tres casos de arranque en total cuyo valor es el de una prueba de humo: confirman que el árbol de inyección de dependencias se resuelve y que el paso de pruebas del flujo tiene algo que ejecutar y que puede fallar. El panel añade cuatro casos sobre la sección de estadísticas, repartidos en dos ficheros. Los del listado de rondas comprueban que las opciones de los dos filtros se derivan de las rondas cargadas, que elegir un punto vuelve a pedir el listado con ese filtro y muestra solo sus filas junto al botón de quitar filtros, y que quitarlos pide de nuevo el listado completo. Los del componente de clasificación comprueban el ganador destacado, la marca de ítem borrado, la unidad en singular o plural, la barra con el porcentaje, el enlace de cada nombre a su historial y que la posición media solo aparece en los *rankings*. En todos ellos el cliente de la API se sustituye por un objeto con funciones simuladas, sin ninguna petición HTTP, y las aserciones se hacen sobre el documento generado. La verificación del resto de páginas descansa en la compilación, que con las comprobaciones estrictas de TypeScript y la compilación anticipada de plantillas detiene la construcción ante una propiedad desconocida o un tipo incompatible, y en que los *frontends* no contienen reglas de negocio: muestran lo que la API devuelve, y los códigos de estado que interpretan son los que las pruebas de integración fijan.

Lo que ningún nivel automático ve, la configuración de *nginx*, las cabeceras del *proxy*, el TLS de *Caddy* y la apariencia de las pantallas, se comprobó a mano sobre la pila de contenedores completa, único entorno donde intervienen esas piezas. Con *curl* se confirmó que cada aplicación se sirve, que las peticiones a la API atraviesan *nginx* y que las imágenes subidas se sirven desde el volumen. Con *Chromium* en modo sin interfaz se renderizaron las pantallas servidas por *Docker*, inyectando la sesión del gestor y simulando los toques de la papeleta, para revisar la presentación en móvil y los estados de la pantalla de voto; es el mismo mecanismo con el que se tomaron las capturas del apartado 7.10 y el informe del anexo C. Antes del primer despliegue se levantó en local la variante de producción completa, con *Caddy* emitiendo certificados internos, y se comprobó que todo respondía a través de él sin ningún puerto interno publicado. *Playwright*, previsto inicial-

mente para automatizar estas comprobaciones, no llegó a incorporarse: los *frontends* son capas finas cuyo comportamiento queda casi por completo determinado por la API ya probada, y mantener un entorno de navegador automatizado tenía un coste que en el tiempo disponible no compensaba. A cambio, esas comprobaciones hay que repetirlas a mano cada vez que cambia la infraestructura o una pantalla, y por eso se reconocen como el siguiente paso natural.

8.5 Validación con el usuario

La validación de que el producto hace lo que el usuario espera no se realizó con una encuesta formal, sino de forma continua a través de las revisiones con el *Product Owner* al cierre de cada *sprint* que describe el capítulo 7. Cada revisión consistió en una demostración del incremento sobre la pila de contenedores, y sus resultados fueron decisiones de alcance: la validación de la especificación retiró el mapa de puntos y la demostración del quinto *sprint* produjo la petición de los votos limitados y del *ranking*. En el último *sprint*, con la dirección pública ya operativa, se probó la aplicación en varios teléfonos escaneando códigos QR impresos, y de esa sesión salió la lista de defectos menores que se corrigieron. Sirvió también como comprobación cualitativa de los requisitos de usabilidad del capítulo 4: el votante llega a la papeleta directamente desde el escaneo, elige y confirma, dentro del límite de interacciones del RNF-01, y ambas interfaces se adaptan a la pantalla del teléfono, como exige el RNF-02. No se dispone de una medida cuantitativa de la usabilidad, y una encuesta con usuarios reales queda como trabajo pendiente.

8.6 Pruebas de carga

El panel de gestión lo usa un gestor a la vez; la parte pública, en cambio, la usan muchos dispositivos simultáneamente cuando varios votantes escanean el mismo QR. Solo dos operaciones reciben esa concurrencia, la consulta del punto por su código y el envío de la papeleta, y a ellas se limita la prueba de carga, con dos objetivos: medir el tiempo de respuesta bajo carga sostenida y demostrar que la restricción de papeleta única resiste envíos simultáneos con el mismo identificador, algo que las pruebas de integración, secuenciales, no pueden probar.

El guion de *k6* comprueba antes de empezar que el punto tiene una ronda activa y ejecuta después dos escenarios en paralelo. En el de votantes, cien usuarios virtuales repiten sin pausa el flujo de la interfaz de votación: cada iteración genera un identificador de dispositivo nuevo, consulta el punto comprobando que está activo y que ese dispositivo aún no ha votado, elige ítems según el tipo de votación y envía la papeleta, verificando que se registra y que la respuesta devuelve los mismos ítems en el mismo orden. En el de duplicados, diez usuarios virtuales envían dos papeletas idénticas exactamente a la vez con el mismo identificador y comprueban que una se acepta y la otra se rechaza. Cuatro umbrales convierten la ejecución en una comprobación con resultado binario: el percentil 95 de la consulta del punto por debajo de 300 milisegundos, el del envío de la papeleta por debajo de 500, una tasa de errores inesperados inferior al 1 % y ningún par de envíos simultáneos con dos papeletas aceptadas. Los dos primeros son objetivos de experiencia de usuario fijados con margen; los dos últimos son condiciones de corrección y no lo admiten. Al terminar, el guion genera un informe HTML autocontenido en español, preferido a las gráficas genéricas del panel de *k6* y a un generador externo que se importa desde una dirección de internet.

Tipo	Ítems	Peticiones	Papeletas	Pares dupl.	p95 GET	p95 POST
Voto único	10	204.849	102.424	15.199	29,9 ms	31,4 ms
Votos limitados	10	199.227	99.613	14.789	30,8 ms	32,4 ms
<i>Ranking</i>	6	169.815	84.907	12.474	35,8 ms	38,2 ms

Tabla 8.3: Resultados de las tres ejecuciones de la prueba de carga, una por tipo de votación, con 100 usuarios virtuales votantes y 10 de duplicados durante 30 segundos.

Se realizaron tres ejecuciones, una por tipo de votación, contra el *backend* de la pila *Docker* local con los datos del inicializador. La tabla 8.3 recoge las cifras y el anexo C reproduce el informe completo de la ejecución del *ranking*. Los cuatro umbrales se cumplieron en las tres, sin errores inesperados ni duplicados aceptados. Los percentiles medidos están un orden de magnitud por debajo de los umbrales, y el *backend* sostiene en un portátil de desarrollo entre 5.653 y 6.821 peticiones por segundo, que equivalen a entre 85.000 y 102.000 papeletas en medio minuto, muy por encima de cualquier votación presencial real. La papeleta única resiste la concurrencia: de entre 12.474 y 15.199 pares de envíos simultáneos por ejecución, ninguno produjo dos papeletas, y es la restricción única de la base de datos, traducida a un 409 por el *backend*, la que lo garantiza. El *ranking* es el tipo más costoso, porque cada papeleta inserta una fila de voto por ítem ordenado, y su percentil 99 sube de los 34 a 37 milisegundos de los otros tipos a 65 a 68, aunque la comparación no es del todo homogénea al tener su votación seis ítems frente a diez. Se trata, además, de una prueba de capacidad y no de usuarios reales: el escenario de votantes no incluye tiempo de reflexión entre peticiones, así que cien usuarios virtuales representan una carga muy superior a cien personas votando.

Estos resultados permiten cerrar los requisitos de rendimiento del capítulo 4. El RNF-06, que exige respuestas de la API por debajo de 800 milisegundos en carga normal, y el RNF-07, que admite hasta 2.500 bajo carga elevada, se cumplen con un margen muy amplio: incluso el mayor tiempo observado en las tres ejecuciones, 239 milisegundos en una consulta del *ranking*, queda muy por debajo del primer límite. El RNF-05, en cambio, se refiere al tiempo de carga de la interfaz de votación completa, que incluye la descarga de la aplicación; la prueba de carga no lo mide, y solo se comprobó de forma cualitativa en los teléfonos de la sesión de validación. Quedan dos límites de la medida: las ejecuciones se hicieron contra el *backend* directo y no a través del *nginx* de la interfaz de votación, y la prueba queda fuera del flujo de *GitHub Actions* a propósito, porque sobre un ejecutor compartido las cifras no son reproducibles ni comparables entre ejecuciones y alargarían cada despliegue sin aportar una comprobación de corrección que la integración no dé ya.

8.7 Cobertura de código

La cobertura del *backend* la mide *JaCoCo* en cada verificación de *Maven*. Su agente se engancha a la máquina virtual en la que corren las pruebas, de modo que registra el código ejecutado tanto por las unitarias como por las de integración, y al final de la fase de verificación escribe el informe en tres formatos: uno navegable por paquete, clase y línea, uno XML para herramientas externas y uno CSV del que se extraen las cifras. Se excluyen del cálculo el punto de entrada de la aplicación, los objetos de transferencia, que son *records* sin lógica, y las propiedades de configuración, para que el porcentaje refleje el código con comportamiento. La versión del complemento se fija a mano porque la lista de materiales de *Spring Boot* no la gestiona, y no hay umbral mínimo configurado: el informe es informativo y un descenso de cobertura no hace fallar la construcción. La tabla 8.4 recoge la ejecución de referencia, del 6 de septiembre de 2026, sobre 45 clases.

Medida	Cobertura
Instrucciones	92,9 %
Ramas	83,0 %
Líneas	92,7 % (769 de 830)
Métodos	87,9 %

Tabla 8.4: Cobertura de código del *backend* medida por JaCoCo.

Por paquete, la cobertura de instrucciones va del 73,0 % del paquete de votos, cuyas entidades tienen accesores que ninguna prueba lee, y el 79,0 % del de ítems, al 97,8 % de la API pública y el 98,9 % del de estadísticas; la autenticación queda al 97,4 % y las estrategias de votación al 95,2 %. El paquete de ítems es el que más código con lógica deja sin recorrer, como la sustitución de una imagen por otra o los errores de escritura en disco, y es el primer candidato si se añade un umbral. El informe ha sido útil en dos sentidos opuestos. Por un lado, como anticipa el capítulo 7, sirvió para cerrar un hueco funcional: la edición de un punto y el bloqueo del cambio de votación asignada con una ronda activa no tenían prueba hasta que la cobertura del servicio de puntos lo hizo visible, y se añadió el caso correspondiente. Por otro, ilustra el límite de la propia medida: el servicio de *tokens* y la configuración de seguridad aparecen al cien por cien de líneas, porque cada petición autenticada los atraviesa, y sin embargo ninguna prueba comprueba qué ocurre con un *token* caducado o con un origen no permitido. La cobertura mide qué código se ejecuta, no qué escenarios se verifican, y una cifra alta no sustituye a la pregunta de qué casos faltan.

8.8 Limitaciones

Lo que no se prueba, o se prueba solo de forma indirecta, conviene dejarlo escrito junto con el siguiente paso en cada caso. Las pruebas de los *frontends* se limitan al arranque y a la sección de estadísticas del panel, y las piezas con lógica propia, el servicio de sesión, la guarda de rutas, el interceptor y la traducción de errores en el panel, y el identificador de dispositivo y la máquina de estados en la interfaz de votación, siguen siendo candidatas directas a pruebas con *Vitest* y el cliente HTTP simulado de *Angular*, sin navegador. No existe un umbral de cobertura; con las cifras actuales, un mínimo del 85 % de instrucciones y del 75 % de ramas dejaría margen para no romper la construcción por un cambio pequeño. No hay pruebas de extremo a extremo automatizadas, y *Playwright* sobre la pila de contenedores sería el paso natural, empezando por el flujo de lanzar, votar y consultar la estadística. Varias clases del *backend* se ejercitan solo de refilón, sin un caso dirigido: el servicio de *tokens* ante un *token* caducado o manipulado, el generador de códigos, el almacenamiento de imágenes y la configuración de CORS. La base de datos de pruebas acumula filas entre casos, lo que haría depender del orden de ejecución a cualquier prueba que consultara datos globales. Y la prueba de carga se ha ejecutado únicamente contra el *backend* directo y sin un escenario con tiempo de reflexión que se aproxime a votantes reales.

8.9 Conclusiones de las pruebas

Las pruebas han cumplido las cuatro garantías fijadas al principio del capítulo. El recuento de los tres tipos de votación está comprobado con números calculados a mano, tanto de forma aislada como atravesando la base de datos. Las reglas de negocio están verificadas en la API, contra el mismo gestor de base de datos que en producción y con las

mismas migraciones, de modo que un desajuste entre el esquema y el código se detecta antes de ejecutar ningún caso. La papeleta única, de la que depende la credibilidad de los resultados, resiste decenas de miles de envíos simultáneos sin una sola doble papeleta. Y el flujo de integración continua convierte las pruebas en una condición de despliegue y no en una comprobación opcional.

Respecto a lo previsto, cambiaron tres cosas. Las pruebas automáticas se concentraron en el séptimo *sprint* en lugar de acompañar al desarrollo desde el principio, y el capítulo 7 ya recoge que el cambio de modelo del *sprint* anterior se abordó sin esa red; es la lección más clara de este capítulo. *Playwright* no llegó a incorporarse, y las comprobaciones de extremo a extremo quedaron como un procedimiento manual que ha funcionado pero que no escala. Y la medición de cobertura, incorporada al final, resultó más útil de lo esperado: no como cifra, sino como mapa de lo que faltaba, que permitió cerrar un hueco real y entender a la vez que una cobertura alta no equivale a un conjunto de escenarios completo. En cuanto al rendimiento, los requisitos del capítulo 4 se cumplen con un margen tan amplio que ninguna de las optimizaciones valoradas durante el desarrollo, como la caché de la consulta pública descartada en el capítulo 7, resulta necesaria en el estado actual del producto.

CAPÍTULO 9

Conclusiones y trabajo futuro

Este Trabajo de Fin de Grado ha resultado un proyecto que muestra en su gran mayoría los conocimientos y aptitudes obtenidos en la rama de Ingeniería del Software. Con ese objetivo en mente se ha realizado un desarrollo ambicioso y complejo sobre un producto software sencillo como es Krate. La decisión de que Krate debía ser un producto sin grandes complejidades venía desde el principio del proyecto, puesto que si se hubiese buscado crear un producto software excesivamente complejo junto con todas las facetas de la ingeniería del software que se buscaban plasmar, el alcance del proyecto se habría disparado.

Los objetivos planteados en el apartado 1.2 se han cumplido con creces. Empezando por los cinco objetivos a nivel de producto, la solución final cumple con todos ellos e incluso se han añadido más funcionalidades que las básicas que se contemplaban en un principio, como la capacidad de realizar varios tipos de votaciones o la creación de ítems reutilizables a lo largo de las votaciones. Por otro lado, los seis objetivos a nivel de ingeniería también se han cumplido durante el desarrollo del proyecto.

Como mayores aprendizajes del trabajo se destaca la capacidad de entender un sistema y una arquitectura software complejos y completos, junto con los elementos que los rodean, como son el despliegue, las metodologías y la especificación de requisitos.

9.1 Relación con los contenidos del grado

Tal y como se destaca a lo largo de esta memoria, los objetivos ingenieriles están muy ligados con las enseñanzas de las asignaturas vistas en el grado, concretamente en la rama de Ingeniería del Software. A continuación se presenta una enumeración de las principales asignaturas que han inspirado y guiado el trabajo:

- **Análisis, Validación y Depuración de Software:** el capítulo 8 sale en su gran mayoría de los conocimientos y prácticas adquiridos en esta asignatura.
- **Análisis y Especificación de Requisitos:** el capítulo 4 lleva a cabo la mayoría de prácticas estudiadas en esta materia.
- **Calidad de Software:** parte de los estándares oficiales seguidos como guía de desarrollo se dieron a conocer en esta asignatura.
- **Diseño de Software:** los patrones aplicados fueron en su mayoría estudiados y practicados en dicha asignatura, sin olvidar las pruebas unitarias que también forman parte de los contenidos de esta.

- Integración e Interoperabilidad: la arquitectura REST que gobierna el *backend* del proyecto se estudia aquí.
- Mantenimiento y Evolución de Software: la gestión de versiones y otra parte de las pruebas también se estudiaron en esta materia.
- Proceso de Software y Proyecto de Ingeniería de Software: de estas asignaturas gemelas se extraen los conocimientos sobre las metodologías y prácticas ágiles, junto con la experiencia de llevar a cabo un proyecto a gran escala.

Y ya fuera de la rama destaca la asignatura de Ingeniería del Software, en la que se aprenden conceptos como diagramas de clases, modelos de dominio y la notación UML, que ha sido la regente en los distintos diagramas que aparecen a lo largo de la memoria. Los conocimientos no solo se quedan ahí: asignaturas como Bases de Datos y Sistemas de Información o Interfaces Persona Computador, entre otras, también han aportado conocimientos relevantes.

Como se puede comprobar, el proyecto ha abarcado prácticamente todas las asignaturas de la rama, junto con su asignatura de presentación. Y aun así ha habido huecos que se han tenido que llenar por cuenta propia. El más grande es la omisión de los procesos de integración continua y despliegue continuo. Estos dos conceptos son una parte fundamental de muchos procesos de desarrollo y en el grado apenas se han comentado. Y, en relación con esto mismo, aunque merece su propia mención, está el despliegue en general. Estos conceptos apenas han sido mencionados a lo largo de los cursos y han supuesto un reto al tener que enfrentarlos sin una formación previa.

9.2 Trabajo futuro

Pese a que el Trabajo de Fin de Grado termine aquí, el proyecto de Krate continuará su desarrollo. Quedan pendientes reuniones con el *Product Owner* para ver hasta dónde se puede escalar la funcionalidad de la aplicación.

Existen otros aspectos a mejorar, como son las pruebas realizadas sobre el *frontend* o la cobertura de las realizadas sobre el *backend*. A nivel de CI/CD se pueden automatizar bastantes más procesos, especialmente si se empieza a trabajar con agentes de inteligencia artificial para la realización y revisión periódica de estos. Y, por último, a nivel de interfaz y experiencia del usuario se pueden pulir las interfaces con las que se interactúa.

Bibliografía

- [1] Asamblea General de las Naciones Unidas. *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible*. Resolución A/RES/70/1, 25 de septiembre de 2015.
- [2] Benita Benítez. *La ciudadanía de la democracia ateniense*. Foro Interno, vol. 5, pp. 37–58, dic. 2005.
- [3] Barry W. Boehm. *Software Engineering*. IEEE Transactions on Computers, vol. C-25, n.º 12, dic. 1976.
- [4] Jean-Charles de Borda. *Mémoire sur les élections au scrutin*. Histoire de l'Académie Royale des Sciences, París, 1781.
- [5] Alistair Cockburn. *Writing Effective Use Cases*. Addison-Wesley, 2000.
- [6] Alan Cooper, Robert Reimann, David Cronin y Christopher Noessel. *About Face: The Essentials of Interaction Design*. 4.ª ed., John Wiley & Sons, sept. 2014.
- [7] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Tesis doctoral, University of California, Irvine, 2000.
- [8] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [9] Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [10] ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018(E): Systems and software engineering — Life cycle processes — Requirements engineering*. IEEE, Nueva York, nov. 2018.
- [11] Michael B. Jones, John Bradley y Nat Sakimura. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force, my. 2015.
- [12] G. Kumar y P. K. Bhatia. *Comparative Analysis of Software Engineering Models from Traditional to Modern Methodologies*. Fourth International Conference on Advanced Computing & Communication Technologies, pp. 189–196, Rohtak, India, 2014.
- [13] Carlos Eduardo Mena Keymer. *Democracia digital: características y desafíos*. Revista de Derecho Electoral, n.º 31, pp. 171–181, ene. 2021.
- [14] Mark Nottingham, Erik Wilde y Sanjay Dalal. *RFC 9457: Problem Details for HTTP APIs*. Internet Engineering Task Force, jul. 2023.
- [15] *Pacto Internacional de Derechos Civiles y Políticos*. 16 de diciembre de 1966. Consultado en <https://www.ohchr.org/es/instruments-mechanisms/instruments/international-covenant-civil-and-political-rights#article-25> a 6 de julio de 2026.
- [16] Jeff Patton. *User Story Mapping: Discover the Whole Story, Build the Right Product*. O'Reilly Media, 2014.

- [17] Portal ISO 25000. *ISO/IEC 25010: Modelo de Calidad del Software*. Consultado en <https://iso25000.com/index.php/normas-iso-25000/iso-25010> a 15 de julio de 2026.
- [18] Niels Provos y David Mazières. *A Future-Adaptable Password Scheme*. Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference, Monterey, California, jun. 1999.
- [19] Jonette M. Stecklein, Jim Dabney, Brandon Dick, Bill Haskins, Randy Lovell y Gregory Moroney. *Error Cost Escalation Through the Project Life Cycle*. NASA Johnson Space Center Technical Report JSC-CN-8435, 2004.
- [20] Adam Wiggins. *The Twelve-Factor App*. Consultado en <https://12factor.net/es/> a 27 de julio de 2026.

APÉNDICE A

Objetivos de Desarrollo Sostenible

Los Objetivos de Desarrollo Sostenible (ODS) son los diecisiete objetivos que la Asamblea General de las Naciones Unidas aprobó en 2015 dentro de la Agenda 2030 [1]. La Universitat Politècnica de València los ha incorporado a su estrategia y, como parte de ese compromiso, todo Trabajo de Fin de Grado incluye este anexo con el grado de relación del trabajo con cada objetivo (tabla A.1) y la alineación con los de mayor relación.

Objetivos de Desarrollo Sostenible	Alto	Medio	Bajo	No procede
ODS 1. Fin de la pobreza.				X
ODS 2. Hambre cero.				X
ODS 3. Salud y bienestar.				X
ODS 4. Educación de calidad.				X
ODS 5. Igualdad de género.				X
ODS 6. Agua limpia y saneamiento.				X
ODS 7. Energía asequible y no contaminante.				X
ODS 8. Trabajo decente y crecimiento económico.				X
ODS 9. Industria, innovación e infraestructuras.		X		
ODS 10. Reducción de las desigualdades.			X	
ODS 11. Ciudades y comunidades sostenibles.			X	
ODS 12. Producción y consumo responsables.		X		
ODS 13. Acción por el clima.				X
ODS 14. Vida submarina.				X
ODS 15. Vida de ecosistemas terrestres.				X
ODS 16. Paz, justicia e instituciones sólidas.	X			
ODS 17. Alianzas para lograr objetivos.				X

Tabla A.1: Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible.

El objetivo con el que Krate guarda una relación más directa es el ODS 16 y, en concreto, su meta 16.7, que busca garantizar decisiones inclusivas, participativas y representativas a todos los niveles. Krate permite a un grupo decidir votando, con una papeleta por dispositivo, un recuento en el que todos los votos valen lo mismo y reglas de integridad que garantiza la base de datos (capítulo 5). Votar sin cuenta ni instalación rebaja la barrera de participación, de ahí la relación menor con los ODS 10 y 11.

La relación con los ODS 12 y 9 es indirecta. El primero viene del caso de uso que dio origen al proyecto (apartado 1.1): máquinas expendedoras abastecidas con lo que votan sus usuarios, lo que ajusta la oferta a la demanda y reduce el producto que caduca sin venderse; además, el QR estático de cada punto evita reimprimir carteles cuando cambia la votación. El segundo se debe a que el trabajo es un producto tecnológico accesible desde cualquier móvil con navegador, desplegado en la nube con integración y despliegue continuos (capítulo 7).

APÉNDICE B

Glosario de términos

A continuación se exponen diferentes términos del dominio de la ingeniería del software que se han utilizado a lo largo de la memoria. Algunos de los términos recogidos son anglicismos de uso generalizado en la industria del software, para los cuales no existe una traducción de uso habitual en el sector.

API REST: Interfaz de programación de aplicaciones (*Application Programming Interface*) que expone las funcionalidades de un sistema a través del protocolo HTTP siguiendo el estilo arquitectónico REST: cada recurso se identifica mediante una URL, las operaciones se expresan con los métodos estándar del protocolo (GET, POST, PUT, DELETE) y las respuestas se devuelven en un formato de intercambio de datos, habitualmente JSON. Es el mecanismo de comunicación entre los *frontends* y el *backend* de este proyecto.

Backend: Capa de un sistema de software que se ejecuta en el lado del servidor y se encarga de implementar la lógica de negocio, gestionar el almacenamiento en bases de datos, procesar las peticiones de la API y garantizar la seguridad del sistema. Es la infraestructura interna que no es visible de forma directa para el usuario final, pero que da soporte técnico a todas las funcionalidades de la aplicación.

Backlog: Lista priorizada de tareas, características, requisitos o mejoras que el equipo de desarrollo debe abordar a lo largo del proyecto. Actúa como la única fuente de verdad para el trabajo que se realizará.

Borrado lógico: Técnica de persistencia en la que un registro que el usuario elimina no se destruye físicamente en la base de datos, sino que se marca como eliminado (habitualmente con una fecha de borrado) y se excluye de los listados y operaciones habituales. Permite conservar el histórico de datos que dependen de ese registro, como las estadísticas de votaciones pasadas.

Bug: Defecto, fallo o error en el diseño o en el código de un sistema de software que provoca que este produzca un resultado incorrecto, inesperado o que se comporte de una manera no deseada respecto a sus especificaciones iniciales.

CI/CD (Continuous Integration y Continuous Deployment / Integración y despliegue continuos): Práctica de ingeniería del software que automatiza la integración de los cambios en el repositorio compartido, la ejecución de las pruebas y la construcción y despliegue del producto. Cada cambio validado desencadena un flujo automático que garantiza que solo el código que ha superado las pruebas llega al entorno de producción.

Cobertura de código: Proporción del código fuente que las pruebas automáticas ejecutan, medida en instrucciones, ramas o líneas. Señala qué partes del programa no

ejercita ninguna prueba, pero no garantiza que los escenarios relevantes estén verificados.

Contenerización: Técnica de virtualización a nivel de sistema operativo que consiste en empaquetar una aplicación junto con todas sus dependencias, bibliotecas y archivos de configuración necesarios dentro de un entorno aislado e independiente llamado contenedor. A diferencia de la virtualización tradicional, comparte el núcleo del sistema operativo anfitrión, lo que garantiza que el software se ejecute de manera idéntica, ligera y predecible en cualquier infraestructura.

Contexto seguro: Condición que el navegador exige para exponer ciertas funcionalidades, como la generación de identificadores aleatorios criptográficos o el acceso al portapapeles: la página debe servirse por HTTPS o desde la propia máquina. Por este motivo la interfaz de votación requiere HTTPS en producción.

DNS dinámico: Servicio que asocia un nombre de dominio a una dirección IP que puede cambiar con el tiempo, de modo que el nombre se mantiene estable aunque la dirección varíe. Permite disponer de un subdominio público sin registrar un dominio propio.

DTO (Data Transfer Object / Objeto de transferencia de datos): Objeto simple e inmutable cuyo único propósito es transportar datos entre las capas de una aplicación o entre el servidor y el cliente. Permite que el contrato de la API sea independiente de las entidades internas de persistencia, de modo que ambos puedan evolucionar por separado.

Framework: Entorno de software reutilizable que define la arquitectura y el flujo de ejecución abstracto de un sistema. Se caracteriza por el principio de inversión de control (IoC), donde el propio *framework* gestiona el ciclo de vida de la aplicación e invoca el código personalizado del usuario para implementar funcionalidades específicas, garantizando la consistencia y eficiencia en el desarrollo.

Frontend: Capa de presentación de una aplicación de software con la que el usuario final interactúa de forma directa. Engloba la interfaz gráfica de usuario (GUI), el diseño visual, la navegación y la lógica de control del lado del cliente, y su propósito principal es transformar los datos provistos por el *backend* en una experiencia interactiva, intuitiva y accesible desde el navegador o dispositivo.

IDE (Integrated Development Environment / Entorno de desarrollo integrado): Aplicación que reúne en una misma herramienta el editor de código, la compilación, la depuración, la integración con el control de versiones y el análisis del código, con el fin de agilizar el trabajo del desarrollador.

Índice parcial: Índice de base de datos que solo se aplica a las filas que cumplen una condición. Un índice único parcial permite expresar restricciones como «una sola ronda activa por punto» directamente en el gestor de base de datos, de modo que se cumplen aunque dos peticiones lleguen a la vez.

JWT (JSON Web Token): Estándar abierto (RFC 7519) para transmitir información entre dos partes como un objeto JSON compacto y firmado digitalmente. El servidor emite el *token* tras autenticar al usuario y puede verificar en cada petición posterior que no ha sido alterado sin necesidad de guardar estado de sesión.

LLM (Large Language Model / Modelo de lenguaje de gran tamaño): Modelo de inteligencia artificial entrenado sobre grandes volúmenes de texto que genera y comprende lenguaje natural y código. Es la tecnología en la que se basan los asistentes conversacionales de programación.

- LTS (Long-Term Support / Soporte a largo plazo):** Versión de un producto software para la que el fabricante garantiza actualizaciones de seguridad y correcciones durante un periodo prolongado, normalmente varios años, frente a las versiones intermedias de vida corta. Se prefiere en producción por su estabilidad.
- Mantenibilidad:** Característica que representa la capacidad del producto software para ser modificado efectiva y eficientemente, debido a necesidades evolutivas, correctivas o perfectivas.
- Metodología:** Conjunto de procedimientos, técnicas, herramientas y soporte documental que guían el proceso de desarrollo de software. En el contexto actual, suele hacer referencia a marcos de trabajo tradicionales o ágiles.
- Migración de base de datos:** Guion versionado que describe un cambio en el esquema o en los datos de una base de datos. Las migraciones se aplican en orden y quedan registradas, de forma que cualquier entorno puede llevarse al mismo estado del esquema de manera reproducible.
- OpenAPI:** Especificación estándar para describir APIs REST en un formato legible por máquinas (JSON o YAML), a partir de la cual se pueden generar documentación interactiva, clientes y validaciones. Antiguamente conocida como *Swagger*.
- ORM (Object-Relational Mapping / Mapeo objeto-relacional):** Técnica y herramienta que traduce de forma automática entre los objetos de un lenguaje de programación orientado a objetos y las tablas de una base de datos relacional, evitando escribir a mano la mayor parte de las consultas SQL.
- Patrón de diseño:** Solución general, reutilizable y documentada a un problema recurrente en el diseño de software. No es código concreto sino una descripción de la estructura y las relaciones entre clases u objetos que resuelven el problema, con un nombre común que facilita la comunicación entre desarrolladores.
- Percentil:** Valor por debajo del cual queda un porcentaje dado de las medidas de un conjunto. En una prueba de carga, el percentil 95 del tiempo de respuesta es el tiempo que no superó el 95 % de las peticiones, y resume mejor la experiencia de los usuarios que la media, que unos pocos valores extremos pueden distorsionar.
- Product Owner:** Rol clave en los marcos de trabajo ágiles que actúa como enlace entre el equipo de desarrollo y los clientes o *stakeholders*. Es el responsable de maximizar el valor del producto.
- Proxy inverso:** Servidor intermedio que recibe las peticiones de los clientes y las reenvía a uno o varios servidores internos, devolviendo después la respuesta como si fuera propia. Permite centralizar el cifrado TLS, ocultar la topología interna de la red y servir varios servicios bajo un mismo punto de entrada.
- Prueba de carga:** Prueba que somete a un sistema a un volumen de peticiones simultáneas controlado para medir sus tiempos de respuesta, su tasa de errores y su capacidad, y comprobar así que se comporta correctamente bajo la demanda esperada.
- Prueba de integración:** Prueba automática que ejercita varios componentes juntos, como los controladores, los servicios y la base de datos, para comprobar que colaboran correctamente. En este proyecto, estas pruebas arrancan la aplicación completa contra una base de datos real y le envían peticiones HTTP.

- Prueba unitaria:** Prueba automática que verifica de forma aislada una unidad pequeña de código, como una clase o una función, sin depender de bases de datos ni servicios externos. Es rápida y, cuando falla, señala con precisión la pieza afectada.
- Retrospectiva:** Reunión que cierra cada *sprint* en las metodologías ágiles, en la que el equipo revisa cómo ha trabajado, qué ha funcionado y qué debe cambiar en el ciclo siguiente.
- SaaS (Software as a Service / Software como servicio):** Modelo de distribución de software en el que las aplicaciones se alojan y ejecutan en infraestructuras gestionadas por un proveedor externo y se ofrecen a los usuarios a través de internet, normalmente mediante suscripción. En este modelo, el proveedor es responsable del mantenimiento, actualización, seguridad y disponibilidad de la aplicación, mientras que los usuarios acceden al servicio sin necesidad de instalar o administrar el software localmente.
- SPA (Single Page Application / Aplicación de una sola página):** Aplicación web que se carga una única vez en el navegador y que a partir de ese momento actualiza su contenido de forma dinámica comunicándose con el servidor mediante una API, sin recargar páginas completas. Ofrece una experiencia similar a la de una aplicación nativa.
- Sprint:** Ciclo de tiempo breve y de duración fija (típicamente de una a cuatro semanas) durante el cual un equipo de desarrollo trabaja para completar una cantidad específica de trabajo, entregando un incremento de software potencialmente desplegable.
- Stakeholder:** Persona, grupo u organización que tiene una participación o interés en el desarrollo, ejecución o mantenimiento de un sistema.
- Tablero Kanban:** Herramienta de gestión visual utilizada para mapear y realizar el seguimiento del flujo de trabajo. Se divide en columnas que representan los diferentes estados del proceso (por ejemplo: «Por hacer», «En progreso» y «Completada»).
- TLS (Transport Layer Security):** Protocolo criptográfico que cifra y autentica las comunicaciones entre un cliente y un servidor en la red. Es la base de HTTPS, la versión segura del protocolo HTTP, y garantiza que los datos intercambiados no puedan ser leídos ni alterados por terceros.
- UML (Unified Modeling Language / Lenguaje Unificado de Modelado):** Estándar industrial de modelado visual utilizado en la ingeniería de software para especificar, visualizar, construir y documentar los componentes y el comportamiento de un sistema.
- Usabilidad:** Capacidad de un producto software para ser comprendido, aprendido, utilizado y resultar atractivo para el usuario cuando se emplea bajo condiciones específicas de uso.
- Usuario virtual:** En una prueba de carga, cada uno de los clientes simulados que ejecutan en paralelo el guion de peticiones. Su número determina la concurrencia a la que se somete el sistema y no equivale a un número de personas reales, ya que un usuario virtual sin pausas genera mucha más carga que una persona.
- VPS (Virtual Private Server / Servidor virtual privado):** Máquina virtual alquilada a un proveedor de alojamiento sobre la que el cliente tiene control total del sistema operativo, a diferencia de las plataformas gestionadas en las que solo se despliega la aplicación.

APÉNDICE C

Informe de la prueba de carga

Este anexo reproduce el informe que genera el propio guion de *k6* al terminar cada ejecución de la prueba de carga descrita en el capítulo 8. Corresponde a la ejecución sobre la votación de tipo *ranking* del 5 de septiembre de 2026, con cien usuarios virtuales en el escenario de votantes, diez en el de duplicados y treinta segundos de duración, contra el *backend* de la pila *Docker* local. Por su longitud, el informe se presenta en tres figuras consecutivas: la cabecera con la configuración y los indicadores principales, los umbrales y los tiempos de respuesta por operación, y las comprobaciones funcionales de cada escenario. Los informes de las ejecuciones de voto único y de votos limitados tienen la misma estructura, y sus cifras se recogen en la tabla 8.3.

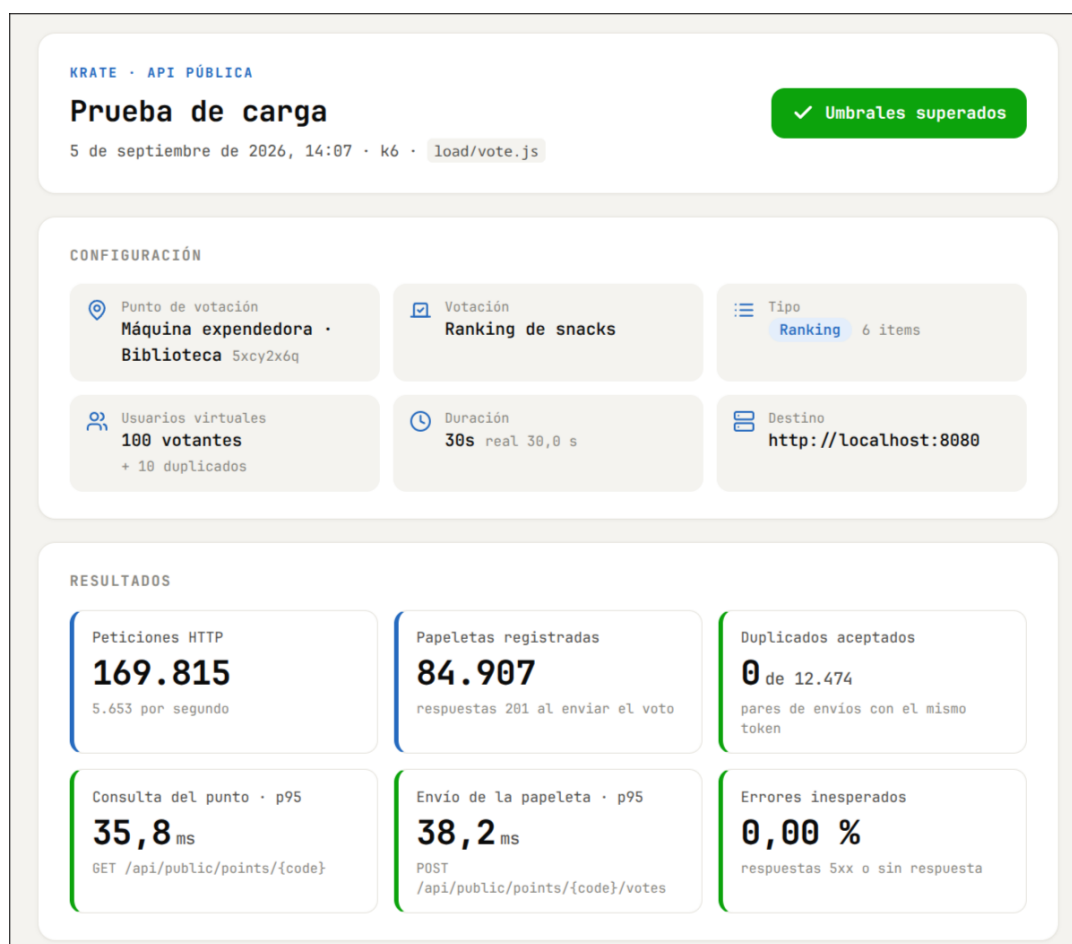


Figura C.1: Cabecera del informe con la configuración de la ejecución y los indicadores principales.

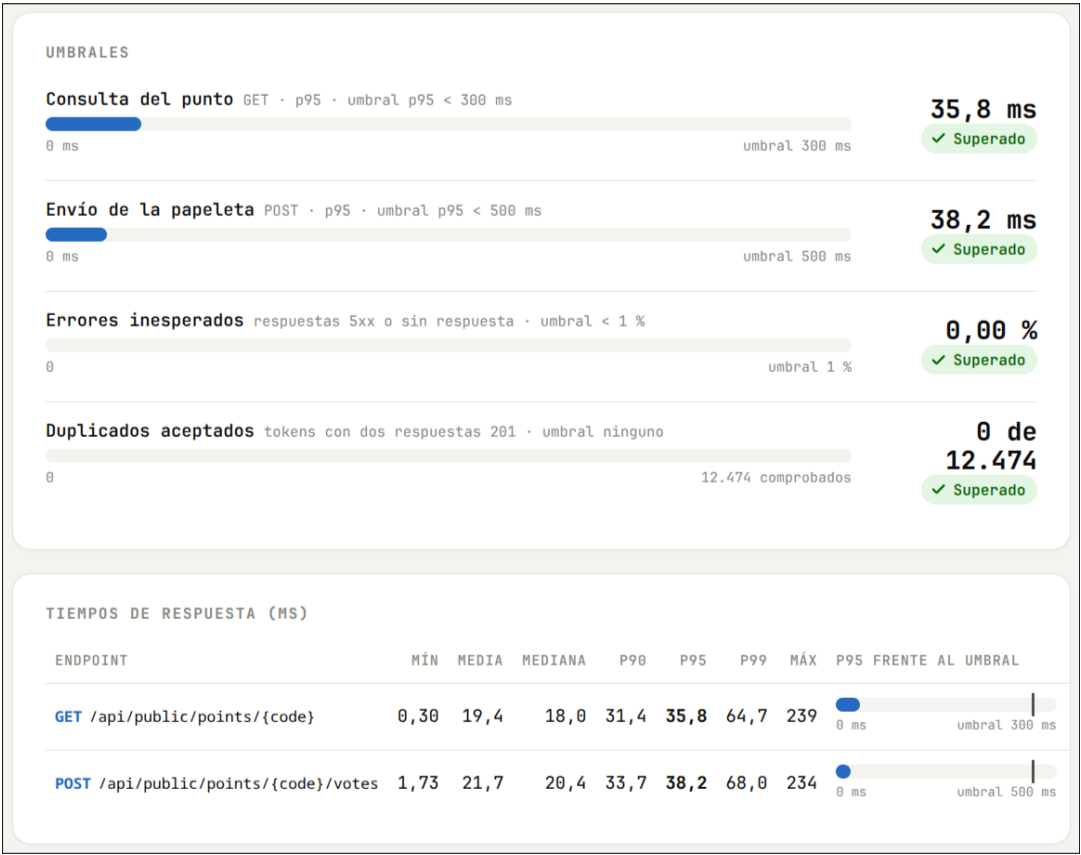


Figura C.2: Umbrales frente a los valores medidos y percentiles de tiempo de respuesta por operación.



Figura C.3: Comprobaciones funcionales de los escenarios de votantes y de papeletas duplicadas.